

Effective Approach for Mining Frequent Subgraph Patterns from Uncertain Graph Databases

by

Exam Roll: Curzon-1611

Registration No: 2014-416-606

Session: 2018-19

A thesis submitted in partial fulfilment of the requirements for the degree of
Master of Science in Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY OF DHAKA

March 21, 2021

Declaration

I, Riddho Ridwanul Haque, hereby declare that the work presented in this thesis titled “Effective Approach for Mining Frequent Subgraph Patterns from Uncertain Graph Databases” is the outcome of the investigation performed by me under the supervision of Dr. Chowdhury Farhan Ahmed, Professor, Department of Computer Science and Engineering, University of Dhaka. I also declare that no part of this dissertation has been or is being submitted elsewhere for the award of any degree or diploma.

Countersigned

Signature

.....

(Dr. Chowdhury Farhan Ahmed)

Supervisor

.....

(Riddho Ridwanul Haque)

Abstract

Due to their ability to model large collections of entities and store information about the links between them, graphs are widely used for representing networks and complex structures. Identifying the subgraph patterns that frequently appear in a graph database can help to extract useful information from the data and the patterns can be used for a variety of applications including social network analysis, analyzing biological networks and mining trajectory data. Most contemporary frequent subgraph mining approaches assume the graph data to be completely accurate and precise. Yet, in many practical scenarios, the data is imperfect, and the existence of edge relations between pairs of nodes in the graphs are uncertain. For instance, in datasets documenting vehicular movements, the trajectories are often uncertain due to inaccuracies in the data collection and compilation phases. Taking this uncertainty into account considerably increases the computational challenges faced during the mining process. However, doing so is crucial for accurately mining the complete list of relevant subgraph patterns. Existing frequent subgraph mining approaches that consider uncertainty rely on approximation schemes. These approaches are inefficient and fail to mine a sound and complete set of frequent patterns. In this paper, we present *UFreS*, a new algorithm for mining frequent subgraph patterns from uncertain graph databases. To make *UFreS* faster than its predecessors, we introduce a novel approach for determining the expected support of a subgraph. We further equip *UFreS* with various pruning and optimization techniques to expedite the entire mining process. Experimental evaluations conducted on real-world datasets provide empirical evidence to show that *UFreS* is efficient, scalable, and outperforms the existing approaches in terms of runtime, memory consumption and accuracy.

Contents

Declaration of Authorship	i
Abstract	ii
List of Figures	v
List of Tables	vii
1 Introduction	1
1.1 Motivation	2
1.2 Objective	7
1.3 Research Challenges	8
1.4 Research Contributions	9
1.5 Thesis Outline	9
2 Problem Definition and Literature Review	11
2.1 Preliminary Concepts and Problem Definition	11
2.2 Related Works	16
2.2.1 The Evolution of Frequent Pattern Mining	16
2.2.2 Existing Approaches for Frequent Subgraph Mining on Un- certain Graph Data	18
3 Our Proposed Approach	21
3.1 Expected support evaluation in an uncertain graph	21
3.2 Expected support evaluation in an uncertain graph database	32
3.3 Generating Candidate Subgraph Patterns	35
3.4 Simulation of UFreS	37
3.5 Time Complexity of UFreS	57
4 Experimental Results	59

4.1	Experimental Setup and Dataset Description	59
4.2	Runtime Comparisons	64
4.3	Memory Usage Comparisons	71
4.4	Accuracy of results	73
4.5	Scalability of UFreS	74
4.6	Summary of the Experimental Results	77
5	Conclusions	79
5.1	Research Summary	80
5.2	Future Research Scopes	81
	Bibliography	90
	List of Notations	90

List of Figures

1.1	A Sample Protein-Protein Interaction Network	3
1.2	Representing the trajectory of a Taxi as an Uncertain Graph	4
1.3	Representing Opinion Propagation Using Uncertain Graphs	5
2.1	A Sample Uncertain Graph Database	12
2.2	Implied Graphs of an example Uncertain Graph	13
2.3	A sample subgraph pattern and its corresponding embeddings in an Uncertain Graph	15
3.1	Edge-Embedding Graph corresponding to the Subgraph Pattern and Uncertain Graph in Fig. 2.3. The edge nodes are rectangular, and the embedding nodes are circular. The numbers above the edge-nodes represent the existential prob- ability of the edges, and numbers below the embedding nodes represent the <i>unique-prod</i> values of the respective embeddings.	24
3.2	Compressed version of the edge-embedding graph in Fig. 3.1	25
3.3	Eliminating an edge from an edge-embedding Graph	27
3.4	Evaluating the worst case when an edge is eliminated	30
3.5	Using Algorithm 2 to evaluate the Edge-Embedding Graph of Fig. 3.2	31
3.6	A Sample Uncertain Graph Database DB	37
3.7	Candidate Subgraph Pattern Generation by UFreS in DB	38
3.8	Edge-Embedding Graphs Generated for Candidate Pattern 1	39
3.9	Expected Support Evaluation for Candidate Pattern 2 in G_0	41
3.10	edge-embedding Graphs Generated for Candidate Pattern 3	42
3.11	Expected Support Evaluation of Candidate Pattern 3 in G_1	42
3.12	Expected Support Evaluation of Candidate Pattern 3 in G_2	43
3.13	edge-embedding Graphs Generated for Candidate Pattern 4	43
3.14	Expected Support Evaluation of Candidate Pattern 4 in G_1	44
3.15	edge-embedding Graphs Generated for Candidate Pattern 6	45
3.16	Expected Support Evaluation of Candidate Pattern 6 in G_0	46
3.17	Expected Support Evaluation of Candidate Pattern 6 in G_1	47
3.18	edge-embedding Graphs Generated for Candidate Pattern 10	48
3.19	Expected Support Evaluation of Candidate Pattern 10 in G_1	48

3.20	Expected Support Evaluation of Candidate Pattern 10 in G_0	49
3.21	edge-embedding Graphs Generated for Candidate Pattern 11	49
3.22	Expected Support Evaluation of Candidate Pattern 11 in G_0	50
3.23	Expected Support Evaluation of Candidate Pattern 11 in G_1	51
3.24	DB after deleting all 0-0 edges	52
3.25	edge-embedding Graphs Generated for Candidate Pattern 12	53
3.26	edge-embedding Graphs Generated for Candidate Pattern 13	53
3.27	edge-embedding Graphs Generated for Candidate Pattern 14	54
3.28	Expected Support Evaluation of Candidate Pattern 14 in G_0	55
3.29	edge-embedding Graphs Generated for Candidate Pattern 19	56
3.30	Frequent Patterns mined by UFreS for $\tau=0.90$ in the Sample Database	56
4.1	Distribution of Uncertainty values in the datasets	63
4.2	Runtime comparison between MUSE, UGRAP and UFreS	65
4.3	Comparison of time taken by the Expected Support Evaluation Schemes in the STRING Database [$\tau = 15\%$]	66
4.4	Comparison of time taken by the Expected Support Evaluation Schemes in the NTP Normal(0.8, 0.1) Dataset [$\tau = 45\%$]	66
4.5	Comparison of time taken by the Expected Support Evaluation Schemes in the NTP Normal(0.4, 0.05) Dataset [$\tau = 40\%$]	66
4.6	Comparison of time taken by the Expected Support Evaluation Schemes in the NTP Exp($\lambda = 4.0$) Dataset [$\tau = 40\%$]	67
4.7	Comparison of time taken by the Expected Support Evaluation Schemes in the Taxi Normal(0.8, 0.1) Dataset [$\tau = 15\%$]	67
4.8	Comparison of time taken by the Expected Support Evaluation Schemes in the Taxi Normal(0.4, 0.05) Dataset [$\tau = 15\%$]	67
4.9	Comparison of time taken by the Expected Support Evaluation Schemes in the Taxi Exp($\lambda = 4.0$) Dataset [$\tau = 12\%$]	68
4.10	Comparison of time taken by the Expected Support Evaluation Schemes in the AIDS Exp($\lambda = 4.0$) Dataset [$\tau = 35\%$]	68
4.11	Comparison of time taken by the Expected Support Evaluation Schemes in the AIDS Normal(0.8, 0.1) Dataset [$\tau = 35\%$]	68
4.12	Comparison of time taken by the Expected Support Evaluation Schemes in the AIDS Normal(0.4, 0.05) Dataset [$\tau = 30\%$]	69
4.13	No. of expected support evaluations required by MUSE, UGRAP and UFreS	70
4.14	Peak Memory Usage Comparison between MUSE, UGRAP and UFreS	72
4.15	Scalability of UFreS in terms of Runtime	75
4.16	Scalability of UFreS in terms of Memory	76

List of Tables

4.1	Dataset Attributes	62
4.2	Sample Precision and Recall Values of UGRAP and MUSE in our experiments. Values marked red represent instances where the pre-existing approaches produced false results	73

List of Algorithms

1	get_expected_support	26
2	evaluate_Edge_Embedding_Graph	30
3	expected_support_in_database	35
4	get_frequent_extensions	36

Chapter 1

Introduction

Social networks [1], text documents [3], vehicular trajectories [19] and biomolecular interactions [5] are only a few examples of the numerous application domains in which much of the relevant data can be conveniently represented using graphs. When dealing with large volumes of graph data, it is often useful to mine subgraph patterns that appear frequently within it. The mined patterns help users obtain pattern-centric insights into the nature of the graph data and identify recurring trends. Frequent subgraph patterns have been utilized in a wide variety of applications, ranging from graph indexing[7] and graph summarization [6] to vehicle routing [8], community detection in social networks [9], identifying sources of pollution and discovering propagation patterns [10], detecting communities from trajectory data [12], detecting abnormal trajectories [11], and developing descriptors for brain connectivity networks [13].

The importance of mining frequent subgraph patterns, along with the considerable computational challenges faced in doing so has made frequent subgraph mining a prominent area of data mining research, with various approaches being proposed to solve the problem in different types of graph data. Yan et. al. [14] proposed gSpan, an efficient approach for mining frequent subgraphs from transactional graph databases, i.e. databases where each transaction can be represented

as a graph. gSpan, and other similar approaches[15][16] mine frequent patterns from graph databases by assuming that the existence of each edge in the graph is certain. Yet, in the graphs of many real world datasets, the existence of the edges in them are probabilistic, that is, these edges may or may not exist in reality.

Databases consisting of such uncertain graphs appear quite frequently in various practical scenarios, and, as discussed in greater detail in Section 1.1, finding recurrent patterns in such databases can prove to be useful for many different applications. Due to their uncertain nature, the true structure of the graphs in these database remain unknown to us. In simpler terms, we do not know what these graphs actually look like. In such databases, the frequent subgraph patterns are those patterns that are *likelier* to appear frequently in the database. In this work, we propose *UFreS*, a fast, exact and scalable algorithm that mines frequent subgraph patterns from uncertain graph databases. Before defining the problem and describing the details of our approach, we first specify some of the practical use cases that motivated us to embark on this journey of mining frequent subgraph patterns from uncertain graph data.

1.1 Motivation

Uncertainty about the existence of edges is an unavoidable and an inseparable part of many real-world networks. This uncertainty arises in cases where we cannot unequivocally determine which is connected to what in a real-life network.

Oftentimes, this uncertainty occurs due to the method in which the data is collected. This is seen in the case of Protein-Protein Interaction (PPI) networks. Many different methods have been used to detect interactions between proteins in various organisms. Yet, none of these methods guarantee complete accuracy and may detect a large number of false interactions that do not exist in reality while

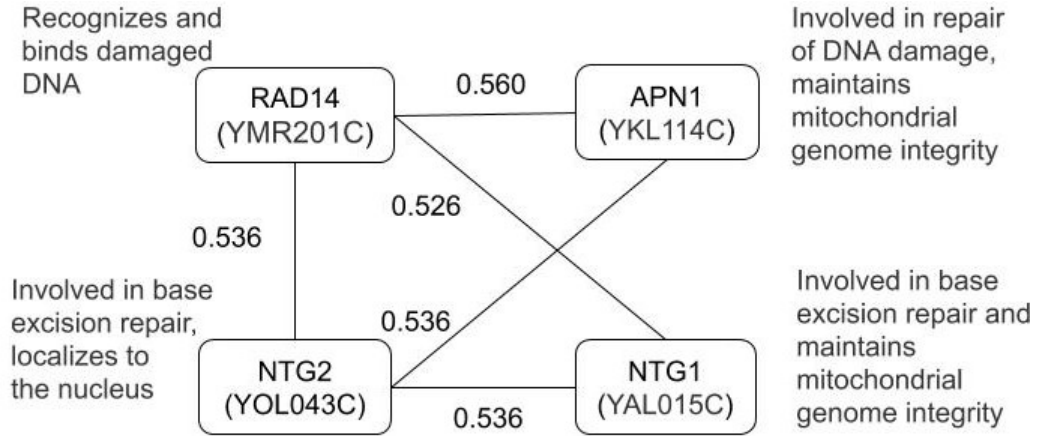


FIGURE 1.1: A Sample Protein-Protein Interaction Network

failing to detect a number of actual interactions. Approaches do exist, however, to determine the probability of each protein-protein interaction truly existing [5], and the resulting collection of interactions can be thought of as an uncertain graph, as shown in Fig. 1.1. The figure shows a sample subnetwork of the Protein-protein interactions occurring in *Saccharomyces cerevisiae* (Yeast), as obtained from the STRING database¹. Each protein contains a name (e.g. RAD14), an identifier (e.g. YMR201C) and a function (e.g. Binding damaged DNA). Interactions between proteins in the organisms have been detected with varying levels of confidence via experimentation.

If the proteins are represented as nodes in a graph and the interactions between them are considered as edges, the resulting network of interactions can be represented by a graph. The confidence levels associated with these interactions can be seen as the existential probabilities of the edges in the graph, thus creating an uncertain graph such as the one represented in Fig. 1.1. Frequent subgraph mining approaches have been used in PPI networks to identify functional modules and trace evolutionarily conserved patterns [17]. Yet, since uncertainties are almost inevitable in such biological networks, we aim to go beyond finding subgraph patterns that are just frequent, and find patterns that also have high confidence

¹string-db.org

values in terms of their probabilities of truly existing.

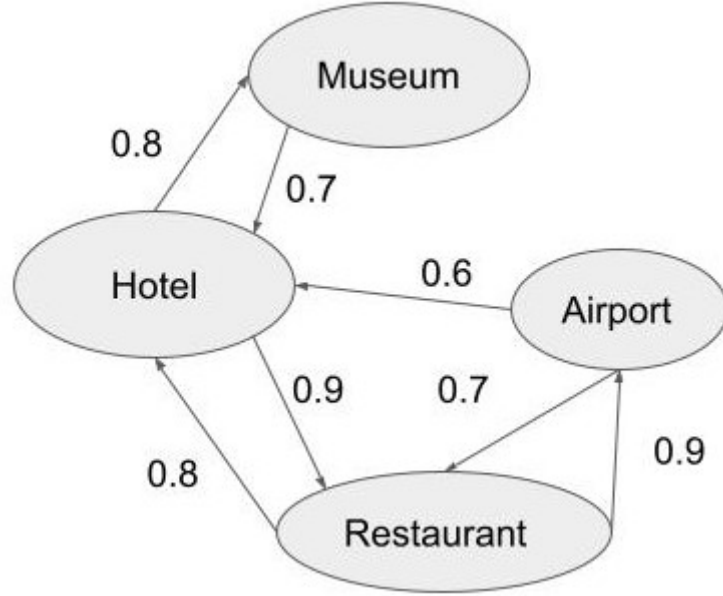


FIGURE 1.2: Representing the trajectory of a Taxi as an Uncertain Graph

Graphs can also be used for representing trajectories of vehicles and other objects. For instance, the dataset analyzed by Moreira et. al. [19] contains information about taxi trips in Porto. As shown in Fig. 1.2, if the sources and destinations of each trip of a taxi are treated as nodes, and an edge is added between these nodes for each trip, the set of trips by a taxi on a given day can be represented using a graph. Each node in the graph can be labelled according to the type of place it represents, (e.g. airports, museums, stadiums, hotels, restaurants, etc.). Yet trajectory data repositories, like most real-world datasets, can be error-prone [20], due to errors in the locations obtained using GPS [21], errors while compiling and compressing the data [22], and for a variety of other reasons. Thus, the locations of the starting and ending points of each trip obtained from the dataset may be incorrect, and the edge corresponding to that trip in the graph shall have an uncertainty associated with it. Mining frequent patterns in the resulting uncertain graphs can help in identifying mobility trends of public vehicles

in large cities.

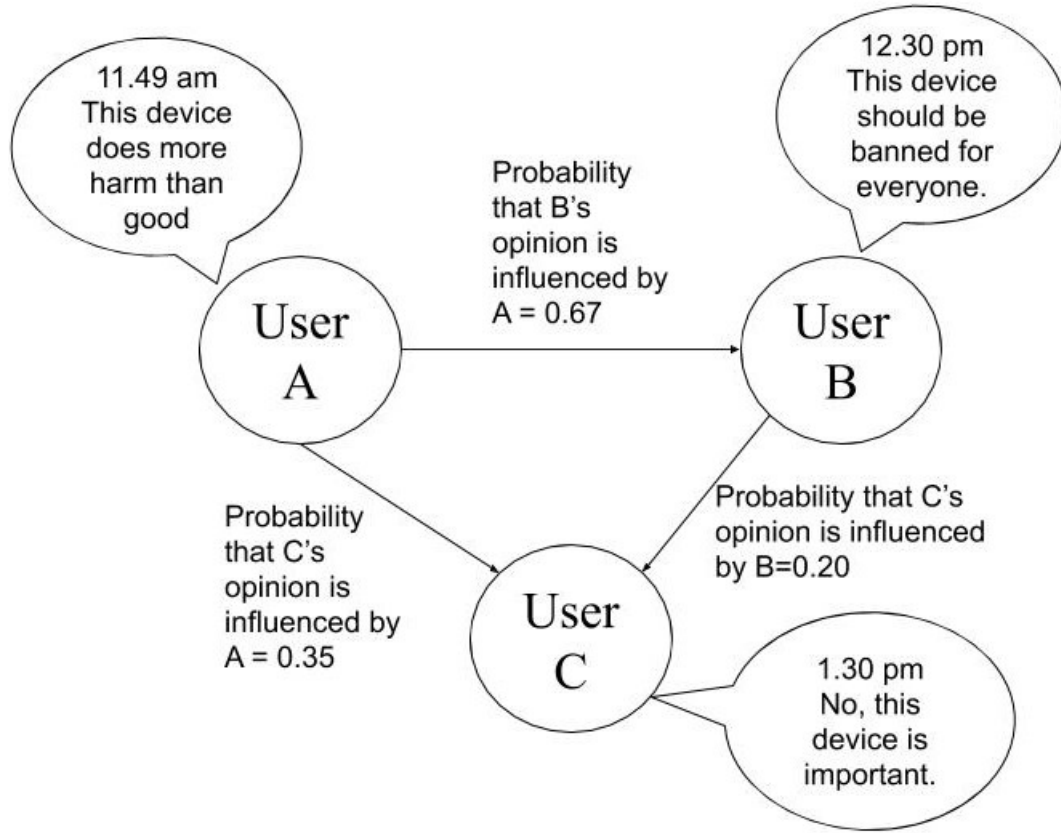


FIGURE 1.3: Representing Opinion Propagation Using Uncertain Graphs

In some cases, the uncertainty may be intrinsic to the phenomenon that the data represents. This is evidenced by the case of opinion propagation in social networks [18]. Even though we can find out who is connected to whom in a social network, whether or not someone's opinion has or will influence that of others remains probabilistic. Here, individuals in the network can be represented by nodes, and each probable event where one user's opinion might have influenced that of another user can be viewed as a probabilistic edge between their corresponding nodes. Social networks can thus be modelled as uncertain graphs, and finding relevant patterns in such networks can allow us to gain important insights as to how opinions are formed and propagated within online communities.

A small-scale example of this is shown in Fig. 1.3, which displays a particular event in which three users have posted their thoughts on the usage of a device. Here, user A posted first, and user B agreed with him. We do not know however, if B's strong aversion to the usage of the device is solely his own opinion, or if it was influenced by A's earlier post. User C, however, later posted a different opinion, and it remains uncertain if this is prompted by C's prior predilection for the device, or his aversion to the opinions posted by A or B. In this case, C's post may be influenced by both, either or neither of A and B. Once we represent such propagation of opinions in social networks as an uncertain graph, we can mine frequent and highly probable subgraph patterns to identify common behavioural traits displayed by users in online communities.

Interestingly, uncertainty in the data is often introduced willingly, typically for preserving private and sensitive information in public databases [23]. This causes originally deterministic graph data to become uncertain, and the consideration of uncertainty becomes vital when mining from such privacy-preserving datasets.

Aside from the cases mentioned, frequent subgraph patterns mined from uncertain graph data have been used for image classification [28], catching synchronized behaviour in networks [29], software defect management [31], feature selection for graph classification [30] and many other applications. However, current approaches for frequent subgraph mining from uncertain graph data are inefficient, and do not guarantee generating a complete and sound set of frequent patterns. The inaccuracy of these state-of-the-art approaches can often lead to misleading interpretations about the data, and hinders the widespread applicability of frequent subgraph mining in the presence of uncertainties. The inefficiency of the approaches makes it almost infeasible to use them to mine frequent subgraph patterns from large graph databases. The example use-case scenarios described so

far, as well as the inadequacies of the state-of-the-art approaches motivated us to explore and invent alternative solutions to the problem of frequent subgraph mining from uncertain graph data.

1.2 Objective

Our objective in this research venture was to find an exact and efficient approach to find frequent subgraph patterns in uncertain graph databases. Given a graph database, where each edge has an existential probability associated with it, we aim to mine all frequent subgraph patterns whose expected support satisfies a user-defined threshold. Chapter 2 provides further explanations regarding the technical terms relevant to our problem, and provides the theoretical foundation of our problem's definition.

Existing approaches that attempt to solve this problem include MUSE [24] and UGRAP [25]. However, the exact algorithm for computing the expected support of a subgraph pattern followed in both these approaches is not efficient enough for mining in large datasets. Both approaches thus resort to sampling-based inference techniques to approximately determine the value of the expected support, making their results unreliable. We aimed to propose the first ever feasible and efficient exact solution for evaluating the expected support of a subgraph pattern in an uncertain graph database. By extension, we wanted to use this solution to design an effective approach that mines a complete and sound set of frequent subgraph patterns despite the presence of uncertainties in large graph databases.

1.3 Research Challenges

When the uncertainty in edge relations are taken into consideration while mining frequent subgraphs, the problem of frequent pattern mining becomes even more challenging, with Zou et. al. [24] proving the problem to be $\#P$ -Hard.

The challenges we faced while designing our approach can be outlined as follows:

- The true form of an uncertain graph is unknown. Depending on the presence or absence of its edges, each uncertain graph can imply an exponential number of deterministic graphs. This makes evaluating the expected support of a subgraph pattern in an uncertain graph a non-trivial task. Zou et al. [24] proved that the problem of finding the expected support of a pattern in an uncertain graph can be reduced to the DNF counting problem. To solve this problem, the existing approaches MUSE [24] and UGRAP [25] both rely on FPRAS [26][54], a Monte-Carlo solution that approximates the expected support of a pattern. Since our objective was to design an efficient and exact algorithm, we needed to devise a method that could determine the expected support of a subgraph pattern efficiently without relying on any such approximation schemes.
- To ensure that our approach will scale well when the number of graphs in the database are high, we needed to find an efficient method to find the expected support of a pattern in the entire database. To do so, we aimed to design a scheme that allows us to reduce the number of expected support evaluations required to determine if a pattern is frequent or infrequent. Designing such a scheme that is both effective and efficient presents an interesting research challenge.

1.4 Research Contributions

In this thesis, we propose *UFreS* (Uncertain Frequent Subgraph miner), a novel approach for mining frequent subgraphs from such uncertain graph databases. As mentioned earlier, there are existing approaches that solve this problem, such as MUSE and UGRAP. However, the exact algorithm for computing the expected support of a subgraph pattern followed in these approaches is not efficient enough for mining in large datasets. Both approaches thus resort to sampling-based inference techniques to approximately determine the value of the expected support, making their results unreliable. Our work bridges this gap by proposing the first ever feasible and efficient exact solution for evaluating the expected support of a subgraph in an uncertain graph database. The main contributions of our work are as follows:

- We propose an efficient method for finding the exact value of the expected support of a subgraph pattern in uncertain graph data.
- We devise *UFreS*, a novel algorithm that finds out all frequent subgraph patterns in uncertain graph databases.
- We propose various techniques to make *UFreS* faster, and more scalable.
- We implement *UFreS* and experiment on real world graph datasets to show how it outperforms the existing solutions.

1.5 Thesis Outline

The remainder of this thesis is organized as follows:

- In Chapter 2, we formally describe the problem, and provide definitions of some terms that we shall repeatedly refer to while describing our approach.

Furthermore, we provide an overview of the relevant literature and summarize the techniques used by the existing solutions.

- In Chapter 3, we comprehensively describe our proposed approach *UFreS*, provide a simulation of how it accurately and efficiently mines a sound and complete set of frequent subgraph patterns from an example uncertain graph database, and discuss the theoretical upper bounds of its time complexity.
- Chapter 4 details the results of our experimental analysis, in which we compare the performance of our proposed approach *UFreS* with that of the existing approaches *MUSE* and *UGRAP*, based on their runtimes, accuracy, memory consumption.
- Chapter 5 discusses our conclusions, summarizes our research investigations and provides directions for future research.

Chapter 2

Problem Definition and Literature Review

In this chapter, we define the terms that we shall frequently refer to while describing our approach and formally define the problem in Section 2.1. Section 2.2 provides an overview of the related works, and describes the influential algorithms that have and continue to influence the domain of frequent pattern mining. We briefly summarize how the existing approaches that solve the problem of frequent subgraph mining from uncertain graph databases and discuss why these solutions fail to be completely reliable and feasible for usage in large datasets.

2.1 Preliminary Concepts and Problem Definition

We begin by defining the terms that we shall frequently refer to while describing our approach.

Definition 2.1 (Uncertain Graph). An uncertain graph G is described by the seven tuple $(V, E, L_v, L_e, l_v, l_e, \mu)$, where V denotes the set of vertices in the graph, E denotes the set of edges connecting them, L_v and L_e denote the set of node labels and edge labels respectively, the functions $l_v: V \rightarrow L_v$ and $l_e: E \rightarrow L_e$

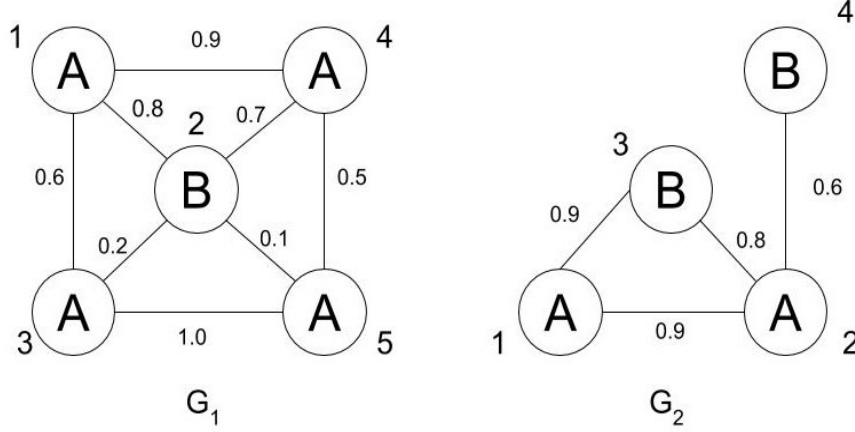


FIGURE 2.1: A Sample Uncertain Graph Database

assign labels to each node and edge in the graph respectively, and $\mu: E \rightarrow [0, 1]$ assigns an existential probability to each edge $e \in E$.

The graphs G_1 and G_2 in Fig. 2.1 are examples of uncertain graphs, with each node containing an identifier and being labelled either A or B. We consider all the edges to have the same label in Fig. 2.1. The existential probability is denoted alongside each edge. For instance, the existential probability of the edge between the nodes 1 and 2 in the graph is 0.8, indicating that the probability of the edge existing in reality is 0.8. We assume that the existential probability of all edges are mutually independent, a simplifying assumption that was also made during the design of the existing uncertain frequent subgraph mining approaches MUSE [24] and UGRAP [25]. The assumption has been proved to be reasonably true in various applications [5][28][29]. Due to the non-deterministic nature of the edges, each uncertain graph implies a set of deterministic graphs with differing probabilities.

Definition 2.2 (Implied Graphs). An implied graph $I(V, I_e, L_v, L_e, l_v, l_e)$ of a given uncertain graph $U(V, E, L_v, L_e, l_v, l_e, \mu)$ is a graph containing the same sets of vertices, labels and a subset of the edges of U (that is, $I_e \subseteq E$).

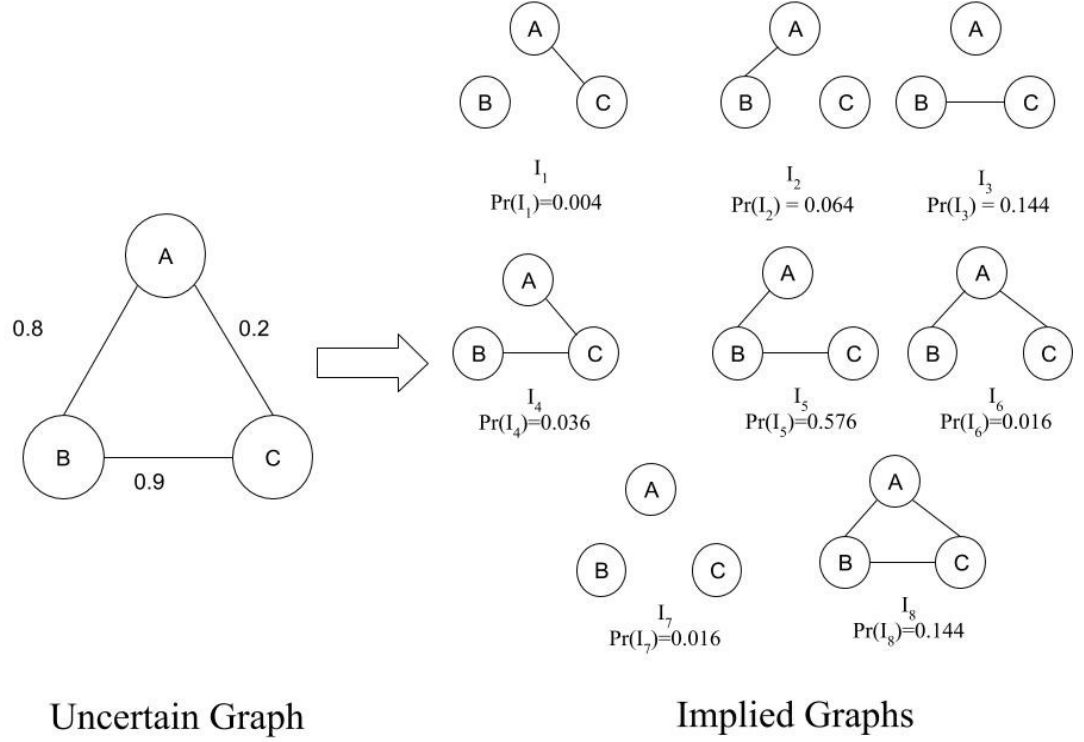


FIGURE 2.2: Implied Graphs of an example Uncertain Graph

An uncertain graph U with $|E|$ edges implies $2^{|E|}$ implied graphs with varying probabilities. Unlike uncertain graphs, the edges in an implied graph are deterministic and there are no probabilities associated with them. Under the assumption that edge probabilities are mutually independent, the probability with which an implied graph I (with the set of edges I_e) is implied by U (with the set of edges E) is defined by the following equation:

$$Pr(U \implies I) = \prod_{e_1 \in I_e} \mu(e_1) \prod_{e_2 \in (E - I_e)} (1 - \mu(e_2)) \quad (2.1)$$

For instance, in the implied graph I_6 , the edges connecting (A, B) and (A, C) of the uncertain graph U exists, and the edge connecting (B, C) does not. The events of (A, B) and (A, C) existing have probabilities of 0.8 and 0.2 respectively, and the event of (B, C) not existing has a probability of 0.1. Thus, the probability with which the uncertain graph U implies the implied graph I_6 , $Pr(U \implies I_6) =$

$$0.8 \times 0.2 \times 0.1 = 0.016.$$

On a seemingly different note, subgraph patterns may be defined as follows:

Definition 2.3 (Subgraph Pattern). A subgraph pattern S can be defined by the 6-tuple $(V, E, L_v, L_e, l_v, l_e)$, consisting of a set of vertices V , edges E , node and edge labels L_v and L_e , and functions l_v and l_e assigning labels to each node and edge in the graph respectively.

A subgraph pattern S is said to be *subgraph isomorphic* to a graph G , if a one-to-one function ϕ maps each node in S to a node with the same label in G , with the condition that if a pair of nodes (S_1, S_2) are adjacent to each other in S , $\phi(S_1), \phi(S_2)$ are adjacent to each other in G . In the context of subgraph isomorphism in uncertain graphs, the term subgraph embedding shall be used frequently in the remainder of the paper.

Definition 2.4 (Subgraph Embedding). A subgraph embedding $SE (V_E, E_E)$ of a subgraph $S(V_S, E_S, L_{vs}, L_{es}, l_{vs}, l_{es})$ in an uncertain graph $U(V, E, L_v, L_e, l_v, l_e, \mu)$ consists of a subset of the vertices and edges in U , i.e. $V_E \subseteq V$ and $E_E \subseteq E$, such that a reversible function ϕ maps each node and edge in V_S and E_S to a unique node in V_E and E_E respectively so that $\forall v \in V_S, l_{vs}(v) = l_v(\phi(v))$ and $\forall (v_1, v_2) \in E_S, l_{es}(v_1, v_2) = l_e(\phi(v_1), \phi(v_2))$.

As shown in Figure 2.3, a subgraph pattern may have multiple and *overlapping* embeddings in a graph. In this thesis, we are interested in mining subgraph patterns that are *likelier* to appear frequently in an uncertain graph database. An uncertain graph database D may consist of several uncertain graphs $\{G_1, G_2, G_3, \dots, G_n\}$. To quantify the likelihood of a subgraph pattern being frequent, we use the metric of expected support.

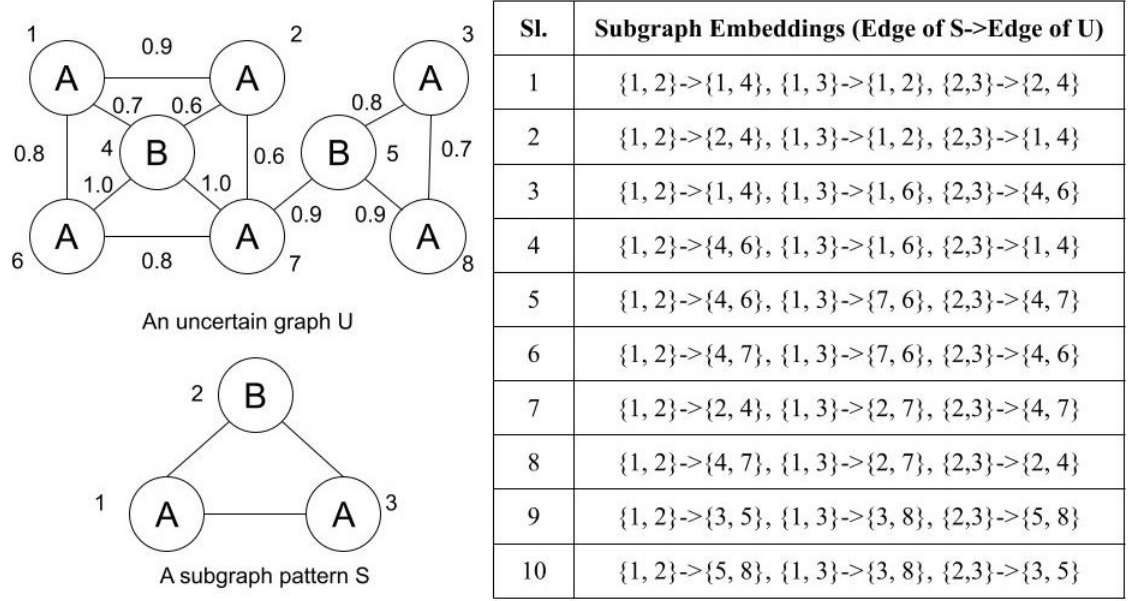


FIGURE 2.3: A sample subgraph pattern and its corresponding embeddings in an Uncertain Graph

Definition 2.5 (Expected Support). If I' is the set of implied graphs of an uncertain graph G which contains at least one embedding of a pattern S , the expected support of S in G is defined by the following equation:

$$ExpSup(S, G) = \sum_{I \in I'} Pr(G \implies I) \quad (2.2)$$

Extending this definition from the context of a single graph to a database consisting of multiple graphs, the expected support of S in an uncertain graph database D is defined as follows:

$$ExpSup(S, D) = \sum_{G \in D} ExpSup(S, G) \quad (2.3)$$

For instance, in Fig. 2.2, the subgraph containing one edge connecting two nodes labelled B and C respectively, is subgraph isomorphic to the implied graphs I_3, I_4, I_5 , and I_8 , and thus has an expected support of $0.144 + 0.036 + 0.576 + 0.144 = 0.9$ in the uncertain graph of Fig. 2.2. We define a subgraph pattern to be frequent

if and only if its expected support is greater than or equal to a user defined threshold τ . Thus, the problem can be formally stated as follows:

Definition 2.6 (Frequent Subgraph Mining From an Uncertain Graph Database). Given an uncertain graph database D and a user defined threshold τ , the problem of **frequent subgraph pattern mining from uncertain graph databases** requires finding all subgraph patterns S for which $\text{ExpSup}(S, D) \geq \tau$.

In the following Section, we provide a review of the existing literature related to frequent subgraph mining, and address how the current approaches solve the problem defined in Definition 2.6.

2.2 Related Works

In this Section, we first provide a brief walkthrough of the hallmark achievements in the field of frequent pattern mining, and discuss how research in this field slowly branched out to help us mine hidden nuggets of knowledge from different types of databases. Subsection 2.2.2 then discusses the shortcomings of the existing approaches that attempt to solve the problem of frequent subgraph mining in uncertain graph databases.

2.2.1 The Evolution of Frequent Pattern Mining

Frequent pattern mining has for long been a focused concern of data mining research. Agrawal et. al. [55] identified the downward closure property of frequent patterns, and used it to propose the Apriori algorithm for efficient frequent item-set mining. Although influential, the Apriori algorithm suffered from generating a large number of candidate patterns, a sizeable portion of which would turn out to be infrequent, resulting in wasted time and computational resources. This algorithm was soon improved by Han et. al. [34], who proposed a novel prefix-tree based data structure known as the FP-Tree, which allowed for mining frequent

itemsets without generating candidate patterns. The FP-Tree based solution was noted for introducing the pattern growth approach, which was later adopted and modified for various forms of frequent pattern mining. Frequent itemset mining approaches were later modified to fit into weighted [39][36], incremental [40][41], dynamic [42][43], high utility [53],[52] and uncertain [37][38][35] frameworks.

Approaches were also developed to mine frequent patterns from different types of data, including data streams [44], [45], sequential databases [46][47], time series databases [48] [49]), and, perhaps most notably in the context of our work, graph databases and large networks [50][51].

A number of approaches exist for mining frequent subgraph patterns from graphs, with gSpan [14] being one of the most influential pattern growth based approaches. gSpan starts by identifying frequent singled-edged subgraph patterns and adds edges to them one by one recursively, till the generated subgraph's support falls below the user-specified threshold. After each edge extension, the support of a subgraph pattern is computed, and to avoid repeatedly calculating the support of the same subgraph pattern, gSpan introduced the concepts of right-most path extension and the minimum DFS Code. However, gSpan does not take the uncertainty of edge relations into account. In our work, we use the recursive process introduced in gSpan to generate candidate subgraph patterns. In the following subsection, we visit the approaches that specifically address the problem of frequent subgraph mining in uncertain graph data, and explain their shortcomings, which we overcome through our own approach.

2.2.2 Existing Approaches for Frequent Subgraph Mining on Uncertain Graph Data

Evaluating the expected support of a subgraph pattern is a computationally challenging task. Zou et. al. [24] introduced MUSE, the first algorithm ever designed for mining frequent subgraph patterns from uncertain graph databases. They proved that determining the expected support is #P-Hard and proposed both an exact approach and an approximate solution to the problem. They showed that if a subgraph pattern S contains n embeddings ($SE_1, SE_2, \dots, SE_n$) in an uncertain graph G , then the expected support of S in G will be equal to the probability that at least one of the embeddings will exist in reality. This probability can be found using the principle of inclusion-exclusion as follows:

$$\begin{aligned}
 ExpSup(S, G) &= Pr(SE_1 \vee SE_2 \vee \dots \vee SE_n) \\
 &= \sum_{1 \leq i_1 \leq n} Pr(SE_{i_1}) - \sum_{1 \leq i_1 < i_2 \leq n} Pr(SE_{i_1} \cup SE_{i_2}) + \\
 &\quad \dots + (-1)^{n-1} \sum_{1 \leq i_1 < i_2 < \dots < i_n \leq n} Pr(SE_{i_1} \cup SE_{i_2} \cup \dots \cup SE_{i_n}) \quad (2.4)
 \end{aligned}$$

In eq. 2.4, as well as in all other similar equations to follow, when a subgraph embedding SE_i is used as a literal in a boolean expression, we will consider it to be true if the embedding exists in reality, and false otherwise. Here, $Pr(SE_{i_1} \cup SE_{i_2} \cup \dots \cup SE_{i_j})$ represents the probability of all the subgraph embeddings in the set $SE_{i_1}, SE_{i_2}, \dots, SE_{i_j}$ existing in reality. Since a set of embeddings will exist if and only if all the edges that constitute them are present, this probability can be found by multiplying the existential probabilities of those edges. For example, in Fig. 2.3, the edges that constitute the embeddings numbered 1 and 3 are $(\{1, 4\}, \{1, 2\}, \{2, 4\})$ and $(\{1, 4\}, \{1, 6\}, \{4, 6\})$ respectively. Thus, the chances that both these embeddings will exist is $Pr(SE_1 \cup SE_3) =$

$\mu(1, 4) \times \mu(1, 2) \times \mu(2, 4) \times \mu(1, 6) \times \mu(4, 6) = 0.3024$. It is to be noticed that although $\{1, 4\}$ appears in both embeddings, its probability will only be multiplied once during the calculations. Since eq. 2.4 requires the computation of 2^n such terms, this exact approach is too inefficient to be feasibly utilized when there are a large number of subgraph embeddings. Zou et al. prove that the problem of computing the expected support of a pattern can be reduced to the DNF Counting Problem. They use the FPRAS method [26] to approximately determine the expected support. However, using such a Monte Carlo approximation scheme leads to a number of drawbacks:

- To provide reasonably accurate results, the approximate algorithm generates a large number of sample implied graphs, making the process inefficient.
- If the parameters are set so as to reduce the number of samples tested, the estimated value of expected support may differ sharply from the actual expected value. This may lead to an infrequent subgraph pattern being incorrectly identified as frequent.
- Due to approximation errors, frequent patterns may be incorrectly classified as being infrequent. When this happens, since candidate patterns are generated by recursively adding edges to already mined frequent patterns, MUSE will not only incorrectly identify the subgraph pattern in question as infrequent, but will also fail to mine all the frequent subgraph patterns that could have been recursively extended from it.

For these reasons, resorting to approximate solutions for frequent subgraph mining is undesirable. Papapetrou et. al. [25] proposed UGRAP, which introduces concepts of edge and connectivity indexing, scheduling and early termination to improve upon MUSE in terms of runtime. Yet, UGRAP relies on the same exact and approximate support evaluation schemes as MUSE, and suffers from the same weaknesses. Chen et. al. [27] proposed FANTA, an approach that mines frequent subgraph patterns from uncertain single large graphs. FANTA uses similar exact

and approximate approaches as UGRAP and MUSE, but instead of working on databases containing numerous graph transactions, focuses on mining from large individual networks.

Thus, while several approaches exist for mining frequent subgraph patterns from uncertain graph data, they all rely on approximation schemes, and finding an efficient method to accurately determine the expected support remains necessary. *UFreS*, our proposed approach, addresses this issue, and mines a sound and complete set of frequent subgraph patterns.

Chapter 3

Our Proposed Approach

In this section, we describe our proposed approach, Uncertain Frequent Subgraph Miner (*UFreS*). We start by describing how *UFreS* determines the expected support of a candidate subgraph pattern in a graph in Section 3.1. We then show how we extend this idea to find the expected support of a subgraph pattern in a transactional graph database, containing numerous graphs. Afterwards, we show how *UFreS* generates the candidate patterns and uses the expected support evaluation module to mine the list of frequent subgraph patterns. We then provide a complete simulation of UFreS in an example uncertain graph database to help the reader better understand the nature and the working principle of the algorithm. Finally, we discuss theoretical upper bounds on the time complexity of our approach.

3.1 Expected support evaluation in an uncertain graph

As shown in eq. 2.4, when determining the expected support of a subgraph pattern S in an uncertain graph G with subgraph embeddings $SE_1, SE_2, \dots, SE_n$, contemporary approaches focus on determining the probability that at least one subgraph embedding will exist. We take an alternative approach and instead determine the

probability that *none* of these embeddings will exist. In other words, we determine the value of $\Pr(\neg SE_1 \wedge \neg SE_2 \wedge \dots \wedge \neg SE_n)$ and use De Morgan's law to compute the expected support, as shown in Eq. 3.1.

$$\text{ExpSup}(S, G) = \Pr(SE_1 \vee SE_2 \vee \dots \vee SE_n) = 1 - \Pr(\neg SE_1 \wedge \neg SE_2 \wedge \dots \wedge \neg SE_n) \quad (3.1)$$

A subgraph embedding will cease to exist if and only if any of the edges in it are absent. To find the value of $\Pr(SE_1 \vee SE_2 \vee \dots \vee SE_n)$, we shall use the following theorem:

Theorem 3.1. *If a subgraph pattern has a set of embeddings $SE = \{SE_1, SE_2, \dots, SE_n\}$, and if a subset $SE_i = \{SE_{i_1}, SE_{i_2}, \dots, SE_{i_m}\}$ of these embeddings consist of the same set of edges, i.e. are duplicates, then*

$$\begin{aligned} \Pr(\neg SE_1 \wedge \neg SE_2 \wedge \dots \wedge \neg SE_n) = & \Pr(\neg SE_1 \wedge \neg SE_2 \wedge \dots \wedge \neg SE_{i_1} \wedge \\ & \dots \wedge \neg SE_{i_2-1} \wedge \neg SE_{i_2+1} \wedge \dots \wedge \neg SE_{i_3-1} \wedge \neg SE_{i_3+1} \wedge \dots \wedge \neg SE_{i_m-1} \\ & \wedge \neg SE_{i_m+1} \wedge \dots \wedge \neg SE_n) \end{aligned} \quad (3.2)$$

Proof. The left hand side of eq. 3.2 includes all embeddings of the pattern. The right hand side includes all the embeddings *except* $SE_{i_2}, SE_{i_3}, SE_{i_4}, \dots, SE_{i_m}$. In other words, all the embeddings in SE_i except SE_{i_1} are *excluded* from the set of embeddings listed in the right hand side of the equation. Let us consider the following cases:

- If all the edges in SE_{i_1} exist, then the boolean value of SE_{i_1} will be true. Consequently, both sides of the equation will be false since they both require SE_{i_1} to be false.
- If any of the edges in SE_{i_1} is absent, then all the duplicate subgraph embeddings $SE_{i_1}, SE_{i_2}, \dots, SE_{i_m}$ will be non-existent and their corresponding boolean values will be false. That would mean that the truth values of both the left and the right hand side of eq. 3.2 would depend on the boolean

values of the literals in SE but not in SE_i , i.e. both the left and right hand sides of the equation shall be equal.

□

Theorem 3.1 indicates that if multiple subgraph embeddings contain the same set of edges, we only need to consider one of those embeddings during our expected support calculation in eq. 3.1. Referring to the embeddings in Fig 2.3, we can see that the embeddings numbered 1 and 2, 3 and 4, 5 and 6, 7 and 8, 9 and 10 consist of the same set of edges. According to theorem 3.1, while determining the expected support, we can find out the value of $Pr(\neg SE_1 \wedge \neg SE_3 \wedge \neg SE_5 \wedge \neg SE_7 \wedge \neg SE_9)$, instead of finding the probability of all 10 embeddings not existing. This duplicacy check among the embeddings is the first step in our approach.

After deleting the duplicate embeddings, we propose constructing a data structure that we shall call an **edge-embedding graph** with the embeddings that remain. An edge-embedding graph is a bipartite graph where each vertex represents either a subgraph embedding, or an edge in the uncertain graph that is included among at least one of the embeddings. We shall refer to the nodes representing embeddings and edges and embedding nodes and edge nodes respectively. An edge will exist in the edge-embedding graph between an embedding node and an edge node, if and only if the corresponding edge is included in the corresponding embedding. Being a bipartite graph, there will be no edges between a pair of embedding nodes or a pair of edge nodes. Fig. 3.1 shows the edge-embedding graph constructed from the non-duplicate embeddings in Fig. 2.3.

As seen in Fig. 3.1, for each edge node, we keep track of the existential probability of the edge. For each embedding node, we associate a variable which we shall refer to as *unique_prod*. Initially, the value of *unique_prod* shall be 1.0. The purpose of forming the edge-embedding graph and keeping track of the *unique_prod* variables is to efficiently find the value of $Pr(\neg SE_1 \wedge \neg SE_2 \wedge \dots \wedge \neg SE_n)$. For reasons

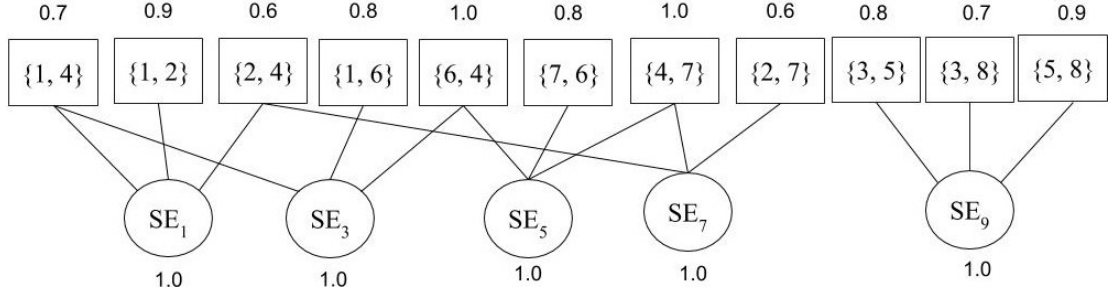


FIGURE 3.1: Edge-Embedding Graph corresponding to the Subgraph Pattern and Uncertain Graph in Fig. 2.3. The edge nodes are rectangular, and the embedding nodes are circular. The numbers above the edge-nodes represent the existential probability of the edges, and numbers below the embedding nodes represent the *unique_prod* values of the respective embeddings.

that will become apparent after we discuss the full procedure, we shall first employ a preprocessing step to reduce the number of vertices in the edge-embedding graph.

We shall eliminate the edge nodes in the edge-embedding graph that have a degree of 1, i.e. the edges that are included in only a single subgraph embedding. The *unique_prod* of the embedding node that the edge node is connected to shall be multiplied by the existential probability of the edge corresponding to the edge node. The edge connecting the two nodes shall also be eliminated from the edge-embedding graph. We refer to this step as *compressing* the edge-embedding graph, since the resulting edge-embedding graph stores essentially the same information, albeit using a more compact representation. After the compression phase is complete, the only edges remaining in the edge-embedding graph will be the ones that are present in two or more embeddings.

Fig. 3.2 shows the compressed version of the edge-embedding graph in Fig. 3.1. Since the edge $\{1, 2\}$ was only present in SE_1 , it is eliminated, and the *unique_prod* value of SE_1 is multiplied by the existential probability of $\{1, 2\}$, which is 0.9. Similar reasoning leads to the deletion of the edge nodes corresponding to $\{1, 6\}$, $\{7, 6\}$ and $\{2, 7\}$, which are only connected to SE_3 , SE_5 and SE_7 respectively. All three edge nodes connected to SE_9 are also eliminated, leaving

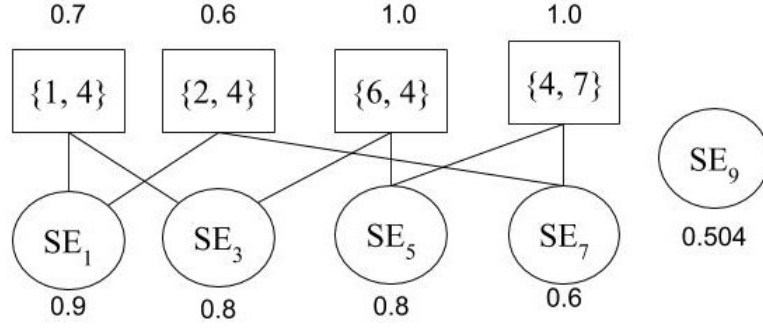


FIGURE 3.2: Compressed version of the edge-embedding graph in Fig. 3.1

its *unique-prod* value to be equal to $0.8 \times 0.9 \times 0.7 = 0.504$.

Fig. 3.2 shows that an edge-embedding graph may contain multiple connected components. For instance, in the edge-embedding graph in Fig. 3.2, SE_1 , SE_3 , SE_5 and SE_7 belong to the same component, while SE_9 belongs to another. Assuming the existential probability of all edges to be mutually independent, since embeddings in different components contain no shared edges, the existence of an embedding in one component will be independent of the existences of the embeddings found in other components. For example, the existence of SE_7 in Fig. 2.3 is independent of the existence of SE_9 , which is reflected by SE_7 and SE_9 being in separate components of the edge-embedding graph in Fig. 3.2. Thus, $Pr(\neg SE_1 \wedge \neg SE_3 \wedge \neg SE_5 \wedge \neg SE_7 \wedge \neg SE_9) = Pr(\neg SE_1 \wedge \neg SE_3 \wedge \neg SE_5 \wedge \neg SE_7) \times Pr(\neg SE_9)$, since the event of SE_1 , SE_3 , SE_5 and SE_7 not existing and that of SE_9 not existing are mutually independent. Generalizing this, if a subgraph pattern has n non-duplicate embeddings $\{SE_1, SE_2, \dots, SE_n\}$ which are partitioned into k components in an edge-embedding graph, if the number of embeddings in each component are $m_1, m_2, \dots, m_k$ respectively, and if $SE_{i,j}$ denotes the i -th embedding

in the j -th component, the following equation shall hold true:

$$\begin{aligned} Pr(\neg SE_1 \wedge \neg SE_2 \wedge \dots \wedge \neg SE_n) &= Pr(\neg SE_{1,1} \wedge \neg SE_{2,1} \wedge \dots \wedge \neg SE_{m_1,1}) \\ &\times Pr(\neg SE_{1,2} \wedge \neg SE_{2,2} \wedge \dots \wedge \neg SE_{m_2,2}) \times \dots \times Pr(\neg SE_{1,k} \wedge \neg SE_{2,k} \wedge \dots \wedge \neg SE_{m_k,k}) \end{aligned} \quad (3.3)$$

Eq. 3.3 shows that we can evaluate each connected component separately considering them as distinct edge-embedding graphs, and multiply the results of their evaluations together. The process of getting the expected support is summarized in algorithm 1. The return value in Line 7 of algorithm 1 corresponds to the right hand side of eq. 3.1.

Algorithm 1 get_expected_support

Input A subgraph pattern S , set of subgraph embeddings SE , an uncertain graph G

Output Expected Support of S in G

- 1: Remove subgraph embeddings with same sets of edges from SE .
 - 2: $EEG \leftarrow$ edge-embedding Graph formed from SE .
 - 3: $C \leftarrow$ Compressed version of EEG .
 - 4: product $\leftarrow 1$
 - 5: **for each** connected component $c \in C$ **do**
 - 6: product $\leftarrow$ product $\times$ evaluate_Edge_Embedding_Graph(c)
 - 7: **return** 1-product
-

To evaluate a given edge-embedding graph, we propose a recursive variable-elimination method. Suppose, an edge-embedding graph EEG contains m edge nodes $\{e_1, e_2, e_3, \dots, e_m\}$, and n embedding nodes $\{SE_1, SE_2, SE_3, \dots, SE_n\}$. The function `evaluate_edge_embedding_graph` needs to consider all 2^m cases formed by the combinations of the edges $\{e_1, e_2, e_3, \dots, e_m\}$ being present and absent in order to find the value of $Pr(\neg SE_1 \wedge \neg SE_2 \wedge \neg SE_3 \wedge \dots \wedge \neg SE_n)$. It does so by eliminating the edges one by one. Suppose, that we pick e_1 as the edge to eliminate. We shall consider two cases, the case where e_1 exists, and the one where it does not. In a world where e_1 certainly exists, we can remove its corresponding edge

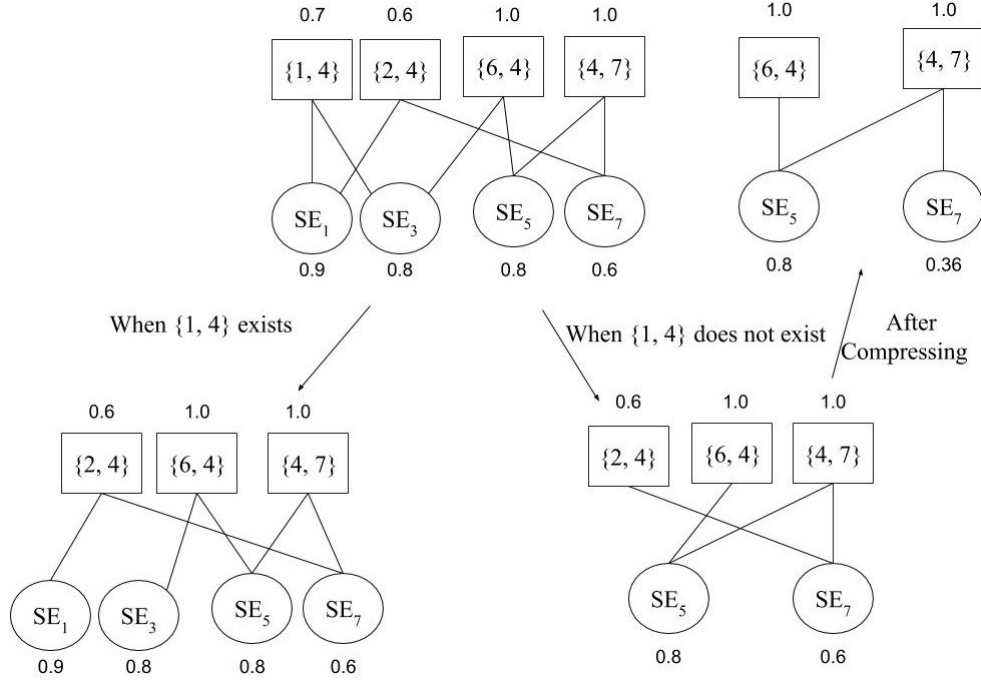


FIGURE 3.3: Eliminating an edge from an edge-embedding Graph

node from EEG along with its adjacent edges. e_1 's elimination from the edge-embedding graph represents that since its presence is now certain, it will have no further role in determining the absence of the remaining embeddings. We shall consider another alternative world where e_1 is certain *not* to exist. In such a world, e_1 is absent, meaning that we can be certain that the embeddings which e_1 were a part of will also be absent. Thus, we can eliminate all the embedding nodes from EEG which are adjacent to e_1 . These cases are illustrated in Fig. 3.3 which shows the transitions of the major connected component of the edge-embedding graph of Fig. 3.2 when the edge $\{1, 4\}$ is eliminated.

The edge-embedding graphs generated for the alternative worlds are then recursively evaluated, eliminating edge nodes (and at times their adjacent embedding nodes) one by one, till one of the following base cases are reached:

- There are no embedding nodes remaining. This means that we have reached a world where all prior subgraph embeddings are certain not to exist. Since

we are ascertaining the probability that no embedding will exist, an event that is now a certainty, we should return 1.0.

- There are no edge nodes remaining. This would mean that there remains only one embedding node in the graph. If there were more than one embedding nodes, they would be in different components, and in accordance with eq. 3.3, they would be evaluated separately. Thus for the embedding that remains, we are now in a world where all the edges it shared with the other embeddings are certain to exist. The only case in which the embedding in question will not exist is if any of the edges unique to it are non-existent. It is to be noted that the *unique_prod* of the embedding node accumulates the product of the existential probabilities of all the edges unique to it, and thus represents the probability that all of these unique edges will exist. The probability that one of these edges will not exist is denoted by the value $(1-\textit{unique_prod})$ and is returned.

The `evaluate_Edge_Embedding_Graph` function will continue to eliminate the existing edges and transition to alternative worlds recursively, till one of the two aforementioned base cases is reached. In the worst case, the algorithm will visit 2^m different states, where m is the number of edges belonging to two or more subgraph embeddings. Since we continue to recursively compress the edge-embedding graph and evaluate its connected components separately, the true number of states visited is usually much lower than this conservative worst case estimate. The order in which we eliminate the edges can influence the number of splittings and compressions taking place, and can thus play an important role in reducing the number of states that need to be visited and can influence the algorithm's runtime. Since finding the optimal order for edge elimination is computationally challenging, we use an alternative greedy approach.

If there exists an edge node in the edge-embedding graph whose existential probability is 1.0, i.e. the edge is certain to exist, we eliminate that edge node first. In this case, we need not consider the world where the edge does not exist.

Otherwise, we eliminate the edge for which the number of states that we will need to explore further in the worst case shall be minimized. When we eliminate an edge, we transition to a world where the edge certainly exists, and to a world where it certainly does not. If the compressed version of the edge-embedding graph in the world where the edge is present contains p connected components with $p_1, p_2, p_3, \dots, p_p$ edge nodes respectively and if that of the world where the edge is absent contains q connected components with $q_1, q_2, q_3, \dots, q_q$ edge nodes respectively, then the number of states that shall be visited in the worst case to evaluate these graphs shall be $2^{p_1} + 2^{p_2} + 2^{p_3} + \dots + 2^{p_p} + 2^{q_1} + 2^{q_2} + 2^{q_3} + \dots + 2^{q_q}$. We shall evaluate the worst case that occurs if each of the edges are eliminated, and then eliminate the edge which presents the best worst case scenario.

Fig. 3.4 shows an example worst case evaluation for eliminating edges from an example edge-embedding graph. In it, the worst case for eliminating e_3 is similar to that of the worst case for eliminating e_1 and is thus not separately shown in the figure. In such a case, our worst-case minimizing selector will select the edge node corresponding to e_2 for elimination.

Algorithm 2 summarizes the entire process for evaluating an edge-embedding graph. Lines 1-4 represent the base cases. If a base case has not been reached, we select an edge to eliminate (line 5) and construct edge-embedding graphs for the worlds where e exists (line 6) and where it does not (line 7). We separately evaluate the connected components of both graphs (lines 10-13) and return the weighted sum of the results of their evaluations, with weights assigned according to the probability of e being present and absent respectively (line 14). Fig. 3.5 shows

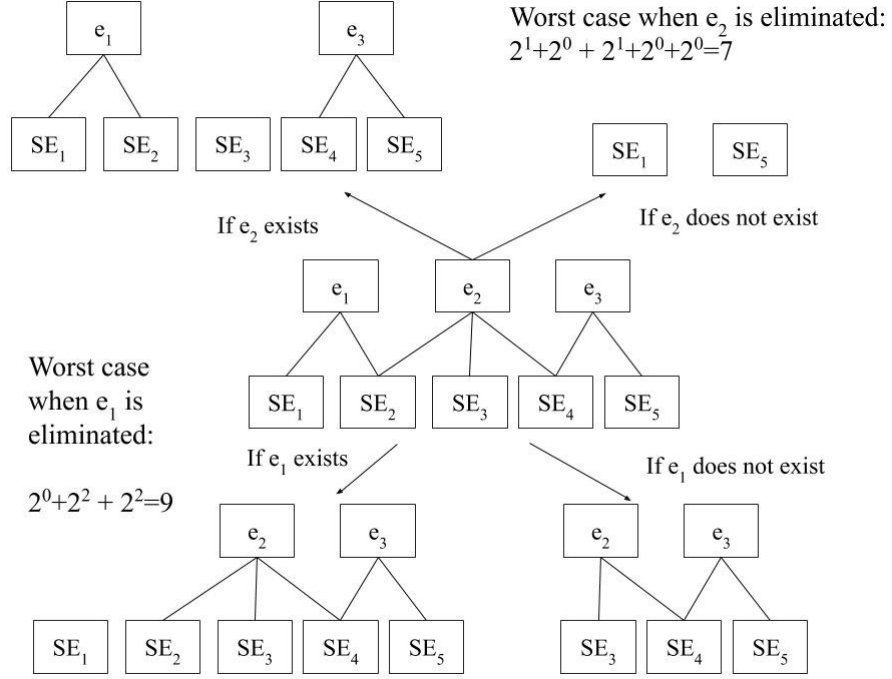


FIGURE 3.4: Evaluating the worst case when an edge is eliminated

Algorithm 2 evaluate_Edge_Embedding_Graph**Input** An edge-embedding graph G **Output** The probability that none of the embeddings in G will exist

- 1: **if** (number of embedding nodes is 0) **then**
- 2: **return** 1
- 3: **if** (number of edge nodes is 0) **then**
- 4: **return** $1 - \text{unique_prod}$ of embedding node
- 5: $e \leftarrow$ Select edge for elimination
- 6: $PG \leftarrow$ Compressed edge-embedding Graph if e exists
- 7: $UG \leftarrow$ Compressed edge-embedding Graph if e does not exist
- 8: $p_product \leftarrow$ Probability that e exists.
- 9: $u_product \leftarrow$ Probability that e does not exist.
- 10: **for each** connected component $c \in PG$ **do**
- 11: $p_product \leftarrow p_product \times \text{evaluate_Edge_Embedding_Graph}(c)$
- 12: **for each** connected component $c \in UG$ **do**
- 13: $u_product \leftarrow u_product \times \text{evaluate_Edge_Embedding_Graph}(c)$
- 14: **return** $p_product + u_product$

how the edge-embedding Graph of Fig. 3.2 can be evaluated using Algorithm 2.

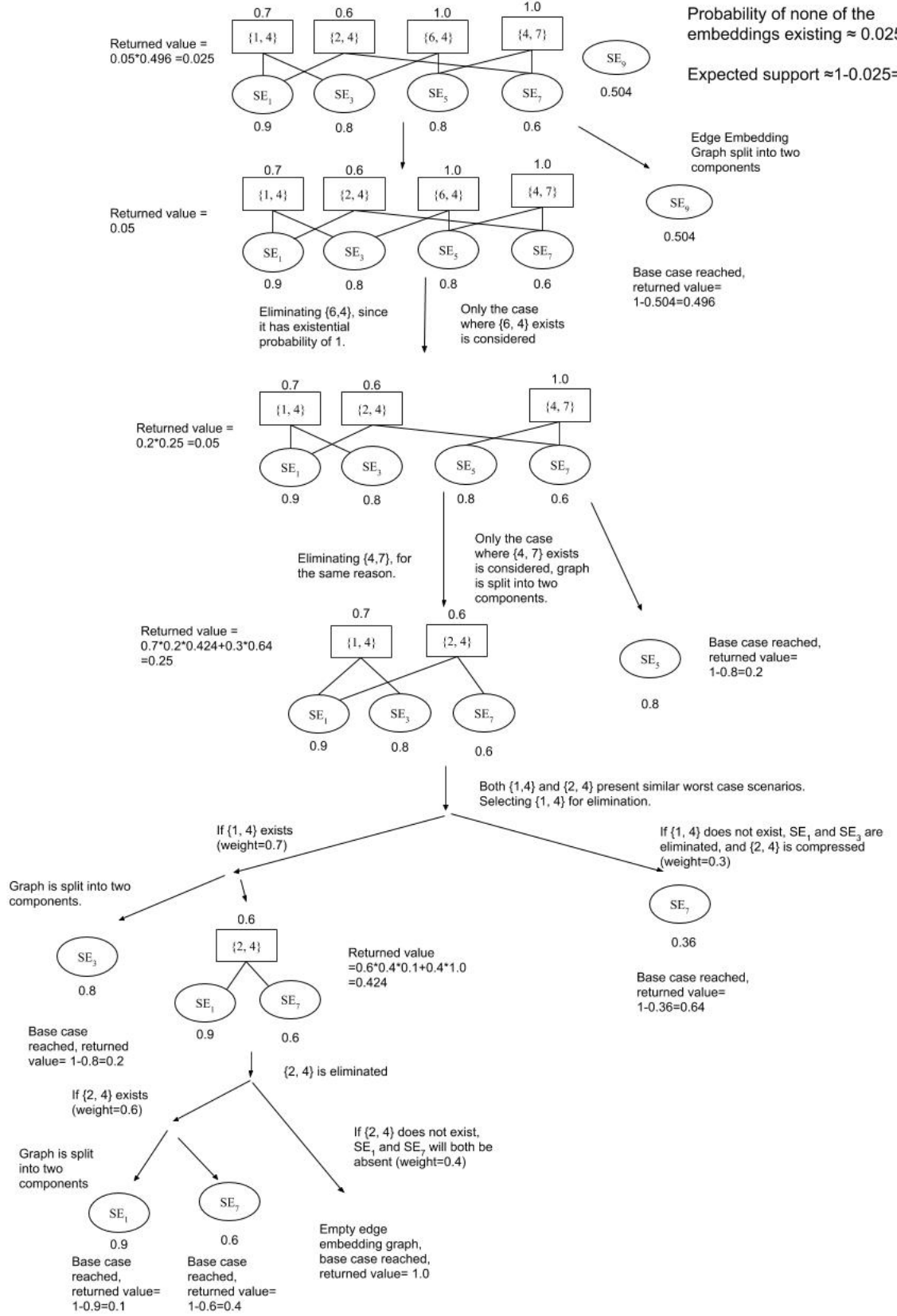


FIGURE 3.5: Using Algorithm 2 to evaluate the Edge-Embedding Graph of Fig.

In this way, edge-embedding graphs help *UFreS* to efficiently overcome the challenge of determining the expected support of a subgraph pattern in an uncertain graph. We note that real world graph databases consist of not one, but a collection of such uncertain graphs. In the following section, we turn our attention to describing how *UFreS* efficiently determines the total expected support of a subgraph pattern in such collections of uncertain graphs, and avoids unnecessary computations to improve the performance of the algorithm in terms of runtime.

3.2 Expected support evaluation in an uncertain graph database

While the previous section describes how we can compute the expected support of a subgraph pattern in a single uncertain graph, our eventual goal is to find the expected support of a pattern in an uncertain graph database, containing numerous individual graphs. This can be done trivially by using algorithm 1 to find the expected support of the pattern in each graph of the database, and summing the values altogether. However, to expedite the process we propose using the following two optimizations:

- **Early Termination:** The objective of our approach is to find the set of subgraph patterns whose expected support satisfies a given threshold, not to calculate the exact value of the expected support. While looping over the graphs in the database, and adding the expected support of the subgraph pattern in each graph, if the sum exceeds the threshold when only a fraction of the graphs in the database have been evaluated, we can skip calculating the expected support for the graphs that remain, since we can already deem the current pattern as frequent. To keep a similar mechanism for terminating early in the case of infrequent subgraphs, we find an upper bound of the expected support of the pattern in each graph before we run the loop for finding the exact expected support. If a subgraph pattern S was generated

by extending an edge to a pattern P , then the upper bound of the expected support of S in an uncertain graph G can be defined by eq. 3.4:

$$\overline{ExpSup(S, G)} = \min(ExpSup(P, G), \sum_{i=1}^n Pr(SE_i)) \quad (3.4)$$

Here, $SE_1, SE_2, SE_3, \dots, SE_n$ denote the subgraph embeddings of S in G . The first term of the upper bound denotes the expected support of the pattern that S was extended from. The process of recursively extending frequent patterns to generate candidate patterns is explained in the following section. For now, we assume the expected support of P in the database will be evaluated before evaluating the expected support of S . If the expected support of P in G was not calculated due to early termination while evaluating P , $\overline{ExpSup(P, G)}$ can be used in place of the first term. The first term can be treated as an upper bound due to the downward closure property, which essentially states that $ExpSup(P, G) \geq ExpSup(S, G)$ since the set of edges in S is a superset of the set of edges in P . The expected support of an empty pattern (i.e. a pattern containing no nodes or edges) is considered to be 1. If S is a single-edged pattern, this first term can be ignored. The second term in eq. 3.4 is actually the first term of the right hand side of eq. 2.4. From eq. 2.4, it can be proved trivially that $\sum_{i=1}^n Pr(SE_i) \geq ExpSup(S, G)$. We choose to compute $\sum_{i=1}^n Pr(SE_i)$ as the complexity of calculating it is linear to the size of the embeddings.

While looping over all the graphs in a database D , and summing up the expected value of the pattern in each graph, if the value of the current sum added with the sum of the upper bounds of the expected support of the pattern in the graphs that remain to be evaluated falls below the minimum support threshold, we can terminate early, concluding that the pattern is infrequent.

- **Scheduling:** To increase the chances of terminating early and save as much time as possible, we aim to evaluate the more *promising* graphs earlier. The graphs that are more *promising* to us are the ones that will give us a high value of expected support (i.e. have a high benefit) and take less time to evaluate (i.e. bear a low cost). Although these values cannot be computed before actually determining the expected support, we propose using the upper bound of the expected support defined in eq. 3.4 as a proxy value for the benefit. As the proxy value for the cost, we can use the number of states that will be visited for evaluating the expected support in a graph in the worst case. As described earlier, the number of states visited in the worst case while evaluating the expected support is dependent on the number of edge nodes in each component of the edge-embedding graph formed. If the number of edge nodes in the connected components of an edge-embedding graph constructed for a pattern S in a graph G are $m_1, m_2, m_3, \dots, m_c$ then the number of states visited in the worst case scenario is $Worst_Case(S, G) = 2^{m_1} + 2^{m_2} + \dots + 2^{m_c}$. Using these proxy values, we evaluate a cost benefit ratio (CBR) for a subgraph pattern in each graph of the database using eq. 3.5.

$$CBR(S, G) = \frac{Worst_Case(S, G)}{ExpSup(S, G)} \quad (3.5)$$

We evaluate the expected support in the graphs of the database in the ascending order of their CBR values.

The overall process of evaluating the expected support of a pattern incorporating the concepts of scheduling and early termination is shown in algorithm 3. After calculating the upper bound of the graphs in the database D (lines 1-2), we sort the graphs in the order in which we shall evaluate them (line 3). We loop over each graph in the database and store the sum of the expected support of the pattern (lines 5-6). If the sum exceeds the user defined threshold, we can directly return true, indicating that the pattern is frequent (lines 7-8). If at any stage,

Algorithm 3 expected_support_in_database

Input A subgraph pattern S , an uncertain graph database D , set of embeddings SE , threshold τ

Output The truth value of $\text{Exp_Sup}(S, D) \geq \tau$

```

1: for each graph  $G \in D$  do
2:   Calculate  $\overline{\text{ExpSup}}(S, G)$ 
3: Sort the graphs in  $D$  in ascending order of CBR
4:  $\text{sum} \leftarrow 0$ 
5: for each graph  $G \in D$  do
6:    $\text{sum} \leftarrow \text{sum} + \text{get\_expected\_support}(S, SE_G, G)$ 
7:   if  $\text{sum} \geq \tau$  then
8:     return true
9:   if  $\text{sum} + (\text{sum of upper bounds of the unevaluated graphs in } D) < \tau$  then
10:    return false

```

the current sum, and the upper bounds of the expected support of the graphs that have not been evaluated yet sum up to a value lower than the threshold, we conclude that the subgraph is infrequent and return false (lines 9-10).

Having defined how we can determine if a candidate subgraph pattern is frequent or not, we now turn our focus to how the candidate patterns are generated in the first place. The following section describes this process briefly.

3.3 Generating Candidate Subgraph Patterns

Our method of generating candidate subgraph patterns is akin to the approach used in gSpan [14]. Given an uncertain graph database D and a minimum support threshold τ , we initially find all the single-edged subgraph patterns that contain embeddings in at least τ graphs of D . These are the initial candidate patterns generated. Each candidate pattern is tested using algorithm 3 to check if they are frequent or not. The frequent patterns are included among the output, and a single edge is added to a node in their rightmost path to generate new candidate patterns. The newly generated candidate patterns are then evaluated recursively. To make sure that we do not evaluate the same pattern more than once, we use

the concept of minimum DFS code introduced in gSpan.

If a pattern is deemed to be infrequent, we do not extend it further. This is due to the *downward closure property*, which states that all the supersets of an infrequent pattern will be infrequent. Thus, all extensions of an infrequent pattern can be pruned from our search space. The downward closure property has been proved to hold for patterns in uncertain graph databases[24]. Algorithm 4 shows how the `get_frequent_extensions` function recursively generates all frequent extensions of S (including S itself).

Algorithm 4 `get_frequent_extensions`

Input A subgraph pattern S , a set of subgraph embeddings SE , an uncertain graph database D , a minimum support threshold τ

Output The set of frequent subgraph patterns extended from S (including S itself)

```

1: if DFSCode( $S$ )  $\neq$  minimum possible DFS Code of  $S$  then
2:   return  $\phi$ 
3: if expected_support_in_database( $S$ ,  $D$ ,  $SE$ ,  $\tau$ ) is false then
4:   return  $\phi$ 
5: frequent_patterns  $\leftarrow S$ 
6: for each  $S_{ext} \leftarrow \{\text{Rightmost path extensions of } S\}$  do
7:    $SE_{ext} \leftarrow \{\text{Embeddings of } S_{ext}\}$ 
8:   frequent_patterns  $\leftarrow$  frequent_patterns  $\cup$  get_frequent_extensions( $S_{ext}$ ,  $SE_{ext}$ ,  $D$ ,  $\tau$ )
9: return frequent_patterns

```

As shown in algorithm 4, we first check if the edges in a pattern S are ordered according to the minimum possible DFS code of S . If not, we discard the pattern and do not extend it further to avoid duplicate subgraph generation (lines 1-2). The concept of the minimum possible DFS Code remains the same as in [14], so we do not elaborately describe it here. Next, we check if the pattern S is frequent and avoid extending it any further if it is not (lines 3-4). Otherwise, we include the pattern in our set of frequent patterns (line 5), and extend the pattern along its rightmost path (line 6), find the embeddings of the extensions (line 7) and recursively find all frequent patterns that contain the extension as a subset (line

8). Finally, we return the list of frequent patterns mined (line 9).

In the following section, we demonstrate how UFreS works on a small example uncertain graph database, to make the working principle of the algorithm more comprehensible for the reader.

3.4 Simulation of UFreS

In this section, we show how UFreS mines all subgraph patterns having an expected support value of 0.90 or higher in the sample uncertain graph database shown in Figure 3.6.

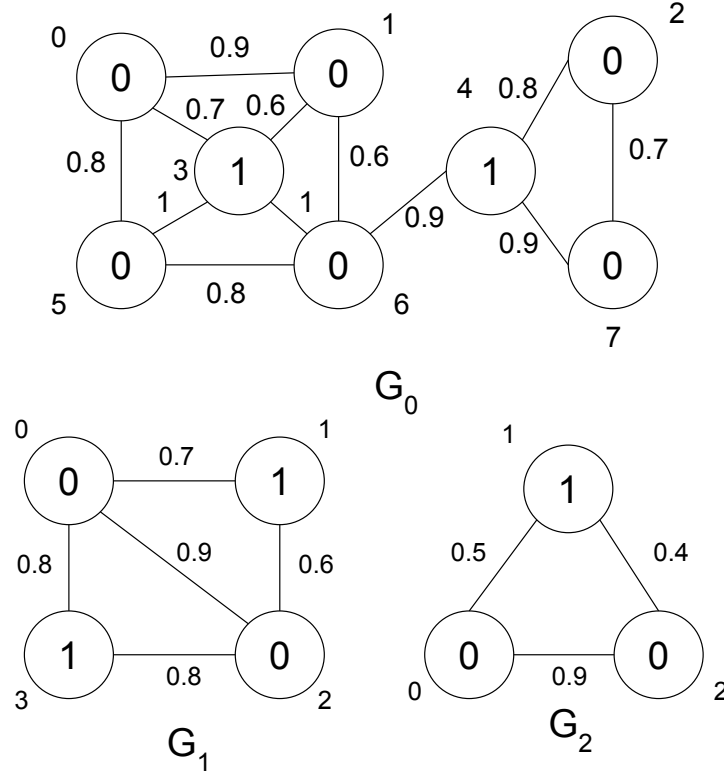


FIGURE 3.6: A Sample Uncertain Graph Database DB

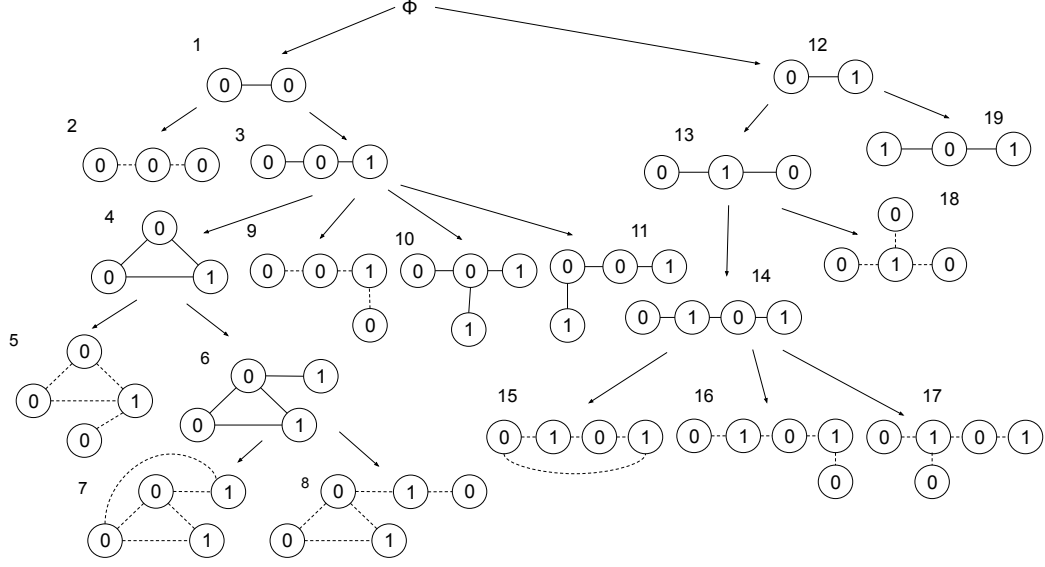


FIGURE 3.7: Candidate Subgraph Pattern Generation by UFreS in DB

As seen in the figure, the sample graph database DB consists of three uncertain graphs, G_0 , G_1 and G_2 respectively. Our goal is to mine all frequent subgraph patterns that have an expected support of 0.90 or higher in DB.

As discussed in Section 3.3, UFreS generates the candidate patterns following the rightmost extension principle and avoids duplicate subgraph generation using the DFS Coding scheme introduced in gSpan. For the given database DB, and the given threshold of 0.90, UFreS performs a depth first traversal of the Subgraph Generation Tree shown in Figure 3.7, recursively adding edges to the rightmost path of the frequent subgraphs, and evaluating the expected support of the canonical extensions.

In Fig. 3.7, the edges of the candidate patterns that were deemed to be infrequent are denoted via dashed lines. These patterns are not extended further due to the downward closure property. The numbers affiliated with the patterns in Fig. 3.7 represent the order in which UFreS generates the patterns, and evaluates their expected support. In the remainder of this Section, we demonstrate how

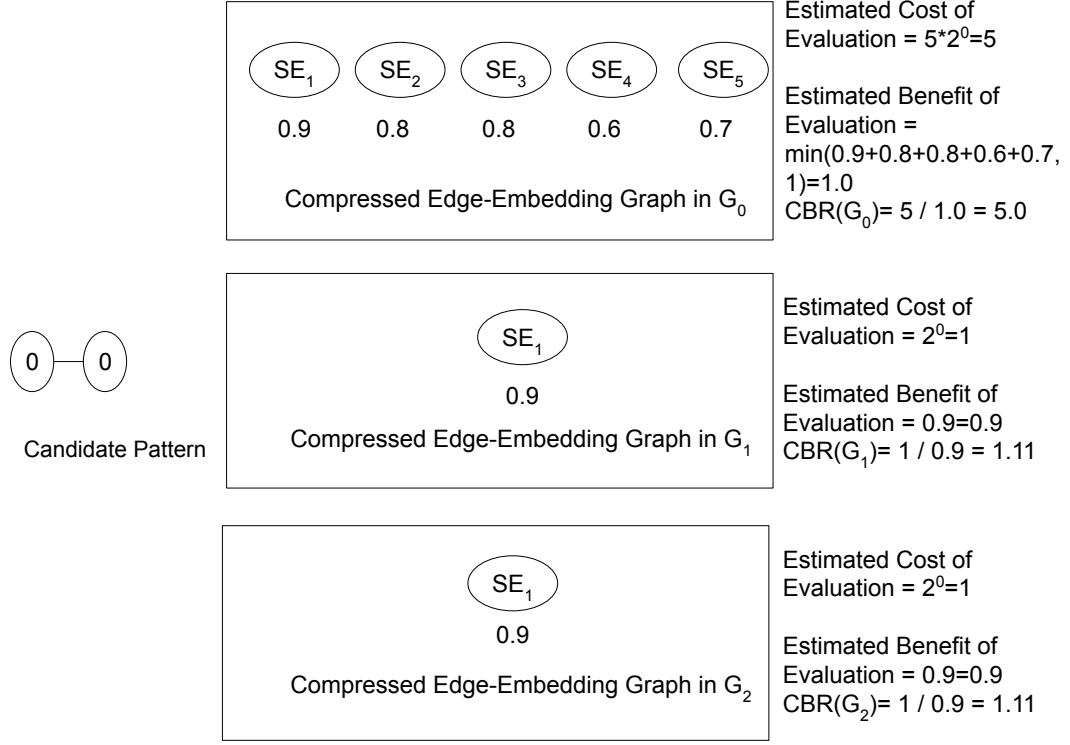


FIGURE 3.8: Edge-Embedding Graphs Generated for Candidate Pattern 1

the expected support of each pattern in the example database is calculated using edge-embedding graphs, early termination and scheduling.

The first candidate pattern generated is a single edged pattern, which produces relatively simple edge-embedding graphs in all three graph databases. As shown in Fig. 3.8, the Cost Benefit Ratio (CBR) of G_0 is the highest among all three graphs, with G_1 and G_2 having equally low CBR values. Intuitively speaking, although the edge-embedding graph of G_0 promises greater benefit, the edge-embedding graphs of G_1 and G_2 promise relatively quicker evaluation. UFreS thus schedules the evaluation of the edge-embedding graph of G_1 and G_2 before the evaluation of G_0 . Since the edge-embedding graph of G_1 consists of a single embedding node only, the expected support of the pattern in G_1 can be trivially determined to be 0.9, which satisfies the minimum support threshold of 0.90. Thus, the first candidate pattern is frequent, and early termination dictates that

the remaining edge-embedding graphs of G_0 and G_2 need not be evaluated.

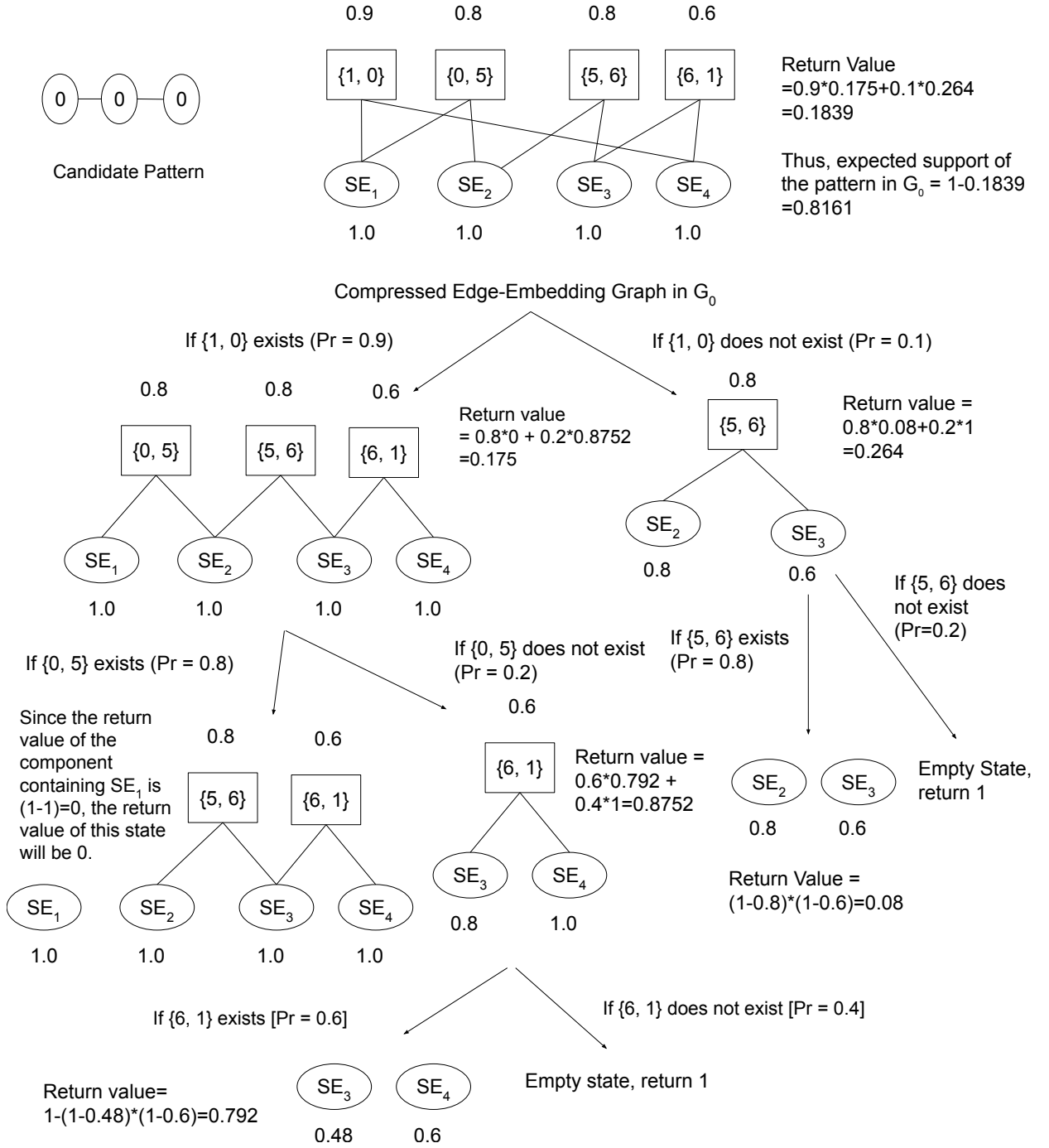
Candidate pattern 2 of Fig. 3.7 only has embeddings in G_0 . Thus, we can ignore the scheduling stage for this particular candidate and proceed to evaluate its expected support in G_0 . The expected support evaluation procedure is shown in Fig. 3.9. The expected support of the subgraph is determined to be 0.8161, which is less than the minimum threshold 0.90, and the candidate pattern is thus deemed infrequent. Accordingly, this pattern is not extended further.

The evaluation of the third candidate pattern helps to demonstrate the performance improvement gained by scheduling and early termination. As seen in Fig. 3.10, the edge-embedding graph of G_0 seems to be rather frightening and computationally expensive to evaluate, while those of G_1 , and G_2 provide a better cost-benefit ratio. We thus schedule the evaluations of the edge-embedding graphs in the order of G_1 , G_2 and G_0 .

As shown in Fig. 3.10, the evaluation of the edge-embedding graph of G_1 gives an expected support value of 0.89562, which is almost equal to the threshold value $\tau = 0.9$ itself.

Fig. 3.12 shows that the evaluation of the edge-embedding graph of G_2 gives a further expected support value of 0.63, making the total expected support = $0.89562 + 0.63 = 1.52562 > 0.9$. Thus, the third candidate pattern is deemed frequent, and the evaluation of the complex edge-embedding graph of G_0 is avoided.

A similar situation arises during the evaluation of the candidate pattern 4, as seen in Fig. 3.13. Once again, the edge-embedding graph of G_0 is seemingly harder to evaluate than that of G_1 and G_2 . G_1 promises better benefits for lower cost, and thus we evaluate its edge-embedding graph before the rest.

FIGURE 3.9: Expected Support Evaluation for Candidate Pattern 2 in G_0

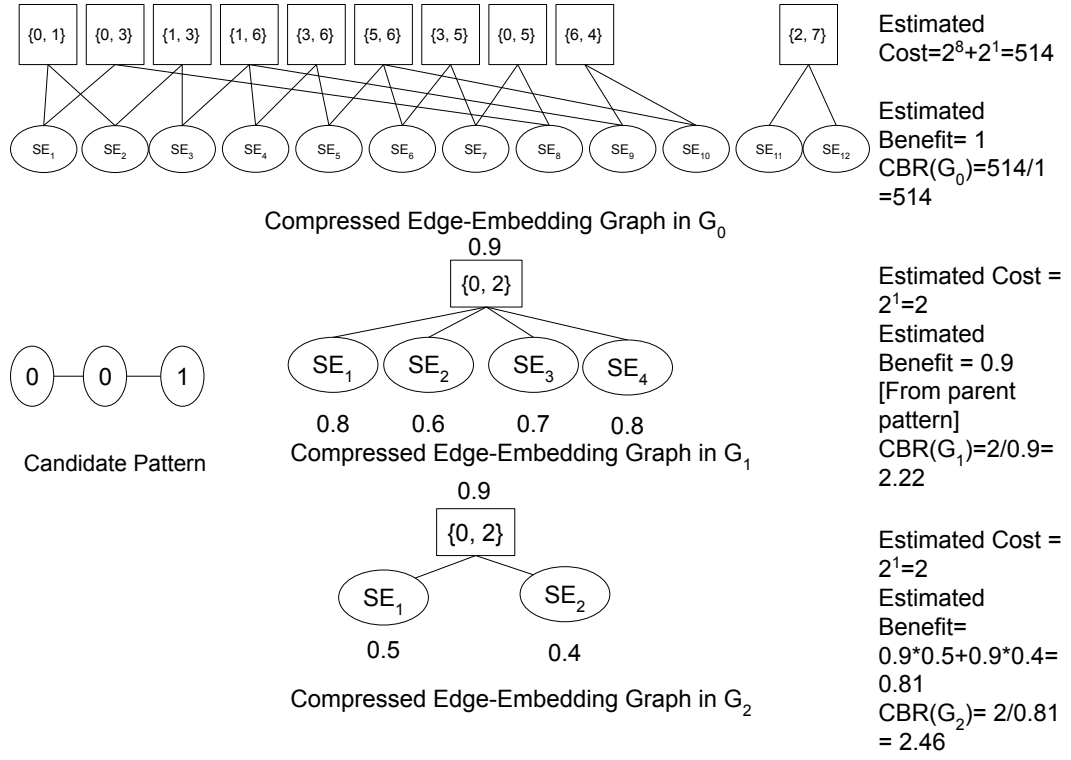
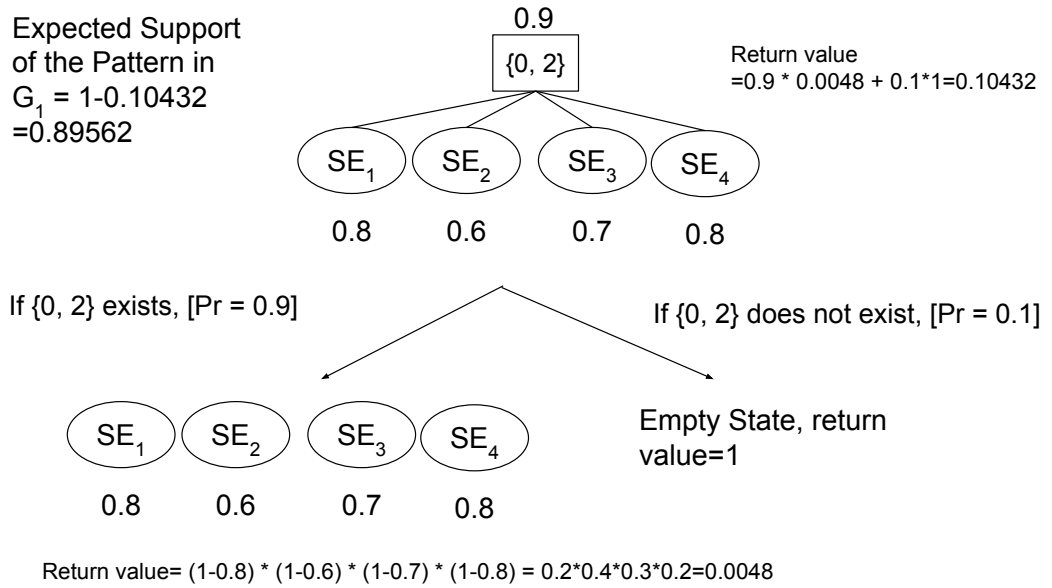


FIGURE 3.10: edge-embedding Graphs Generated for Candidate Pattern 3

FIGURE 3.11: Expected Support Evaluation of Candidate Pattern 3 in G_1

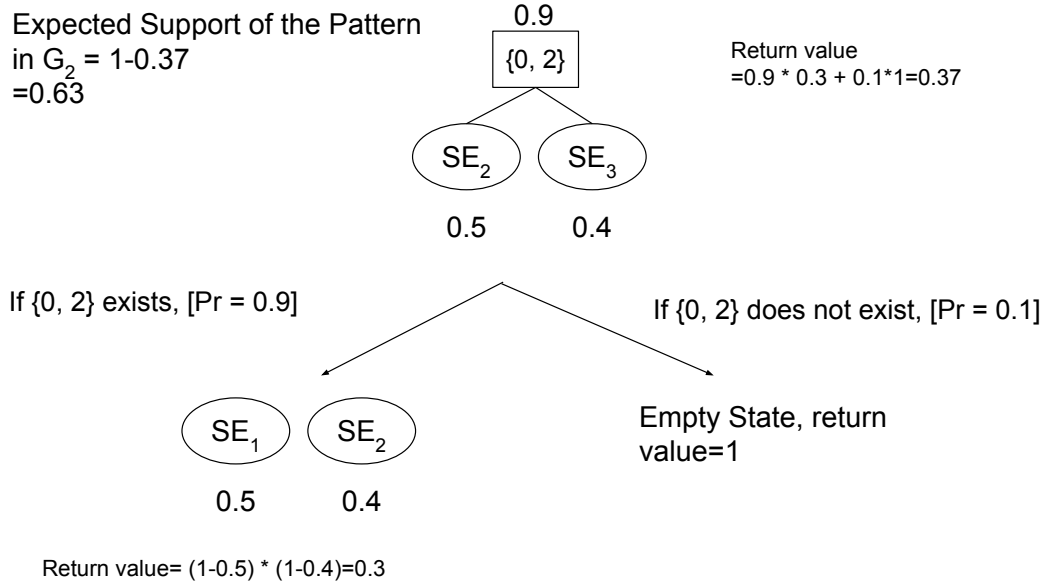
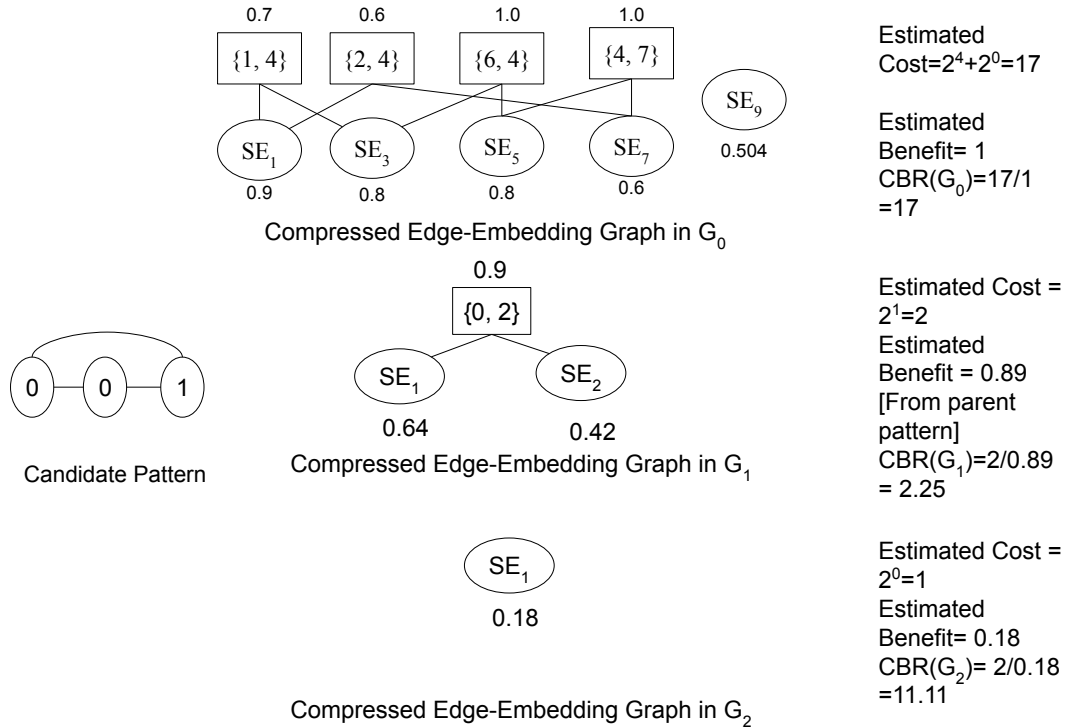
FIGURE 3.12: Expected Support Evaluation of Candidate Pattern 3 in G_2 

FIGURE 3.13: edge-embedding Graphs Generated for Candidate Pattern 4

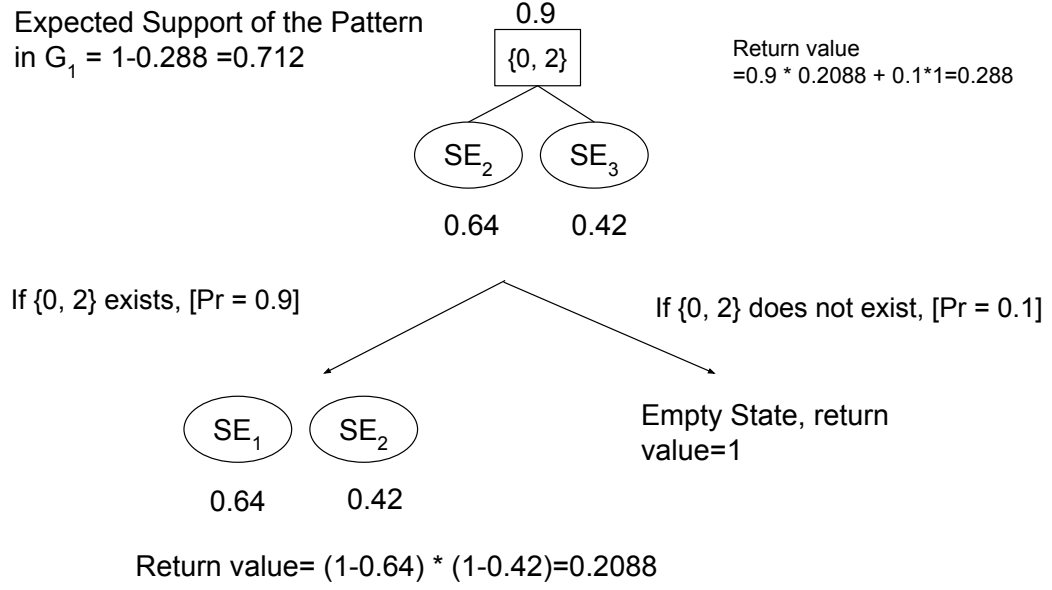
FIGURE 3.14: Expected Support Evaluation of Candidate Pattern 4 in G_1

Fig. 3.14 shows that the evaluation of the edge-embedding graph of G_1 gives a result of 0.712. The evaluation of G_2 is trivial, since it consists of a single embedding node only, and gives a result of 0.18. The total expected support of Candidate Pattern 4 in G_0 is $0.712 + 0.18 = 0.892$. Since this total expected support is not equal to the minimum threshold value of 0.90, we cannot terminate early, and proceed to evaluate the expected support of the pattern in G_0 . The evaluation of the edge-embedding graph of G_0 is shown in Fig. 3.5, and the resulting expected support of the pattern in G_0 is determined to be 0.975. As a result, the total expected support of the pattern in the database becomes $0.892 + 0.975 = 1.867$, which satisfies the threshold, and the pattern is thus determined to be frequent.

To avoid making the chapter unreadably long, we do not demonstrate the expected support evaluation of the infrequent extensions in detail.

For the sixth candidate pattern, the edge-embedding graph generated is

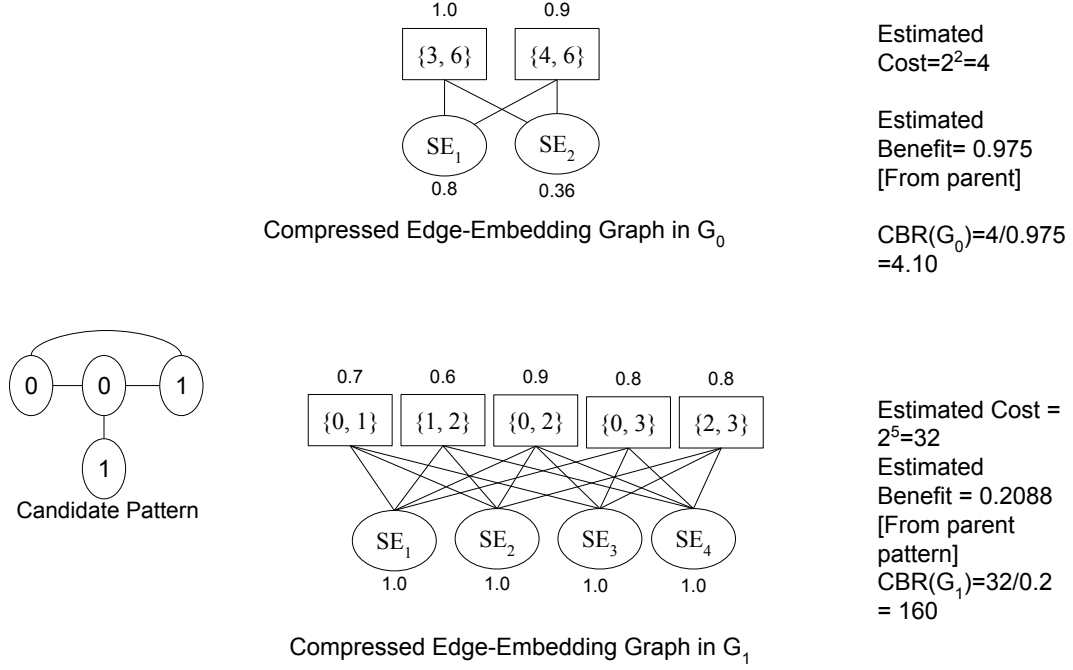


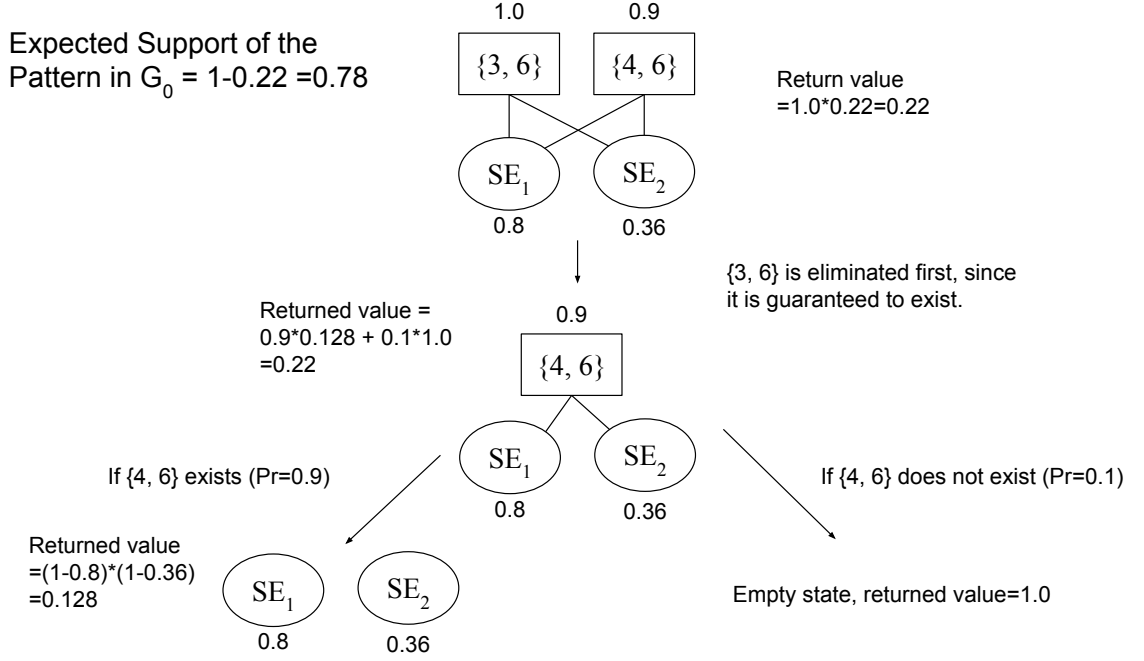
FIGURE 3.15: edge-embedding Graphs Generated for Candidate Pattern 6

shown in Fig. 3.15. It is to be noted that since candidate pattern 6 has no embedding in G_2 , no edge-embedding graph is created for the pattern in G_2 . Sorting in the ascending order of CBR values, the edge-embedding graphs are evaluated in the order G_0, G_1 .

Fig. 3.16 shows that the expected support of the candidate pattern 6 in G_0 is 0.78. Since this value does not satisfy the threshold by itself, we do not terminate early and proceed to evaluate the edge-embedding graph of G_1 .

Fig. 3.17 shows that the expected support of candidate pattern 6 in G_1 is 0.7164, making the total expected support $0.78 + 0.7164 = 1.4964$. Since this value is greater than 0.90, the pattern is frequent.

After candidate pattern 6, the next frequent pattern generated by UFreS is the pattern numbered 10 in Fig. 3.7. The edge-embedding graphs generated for

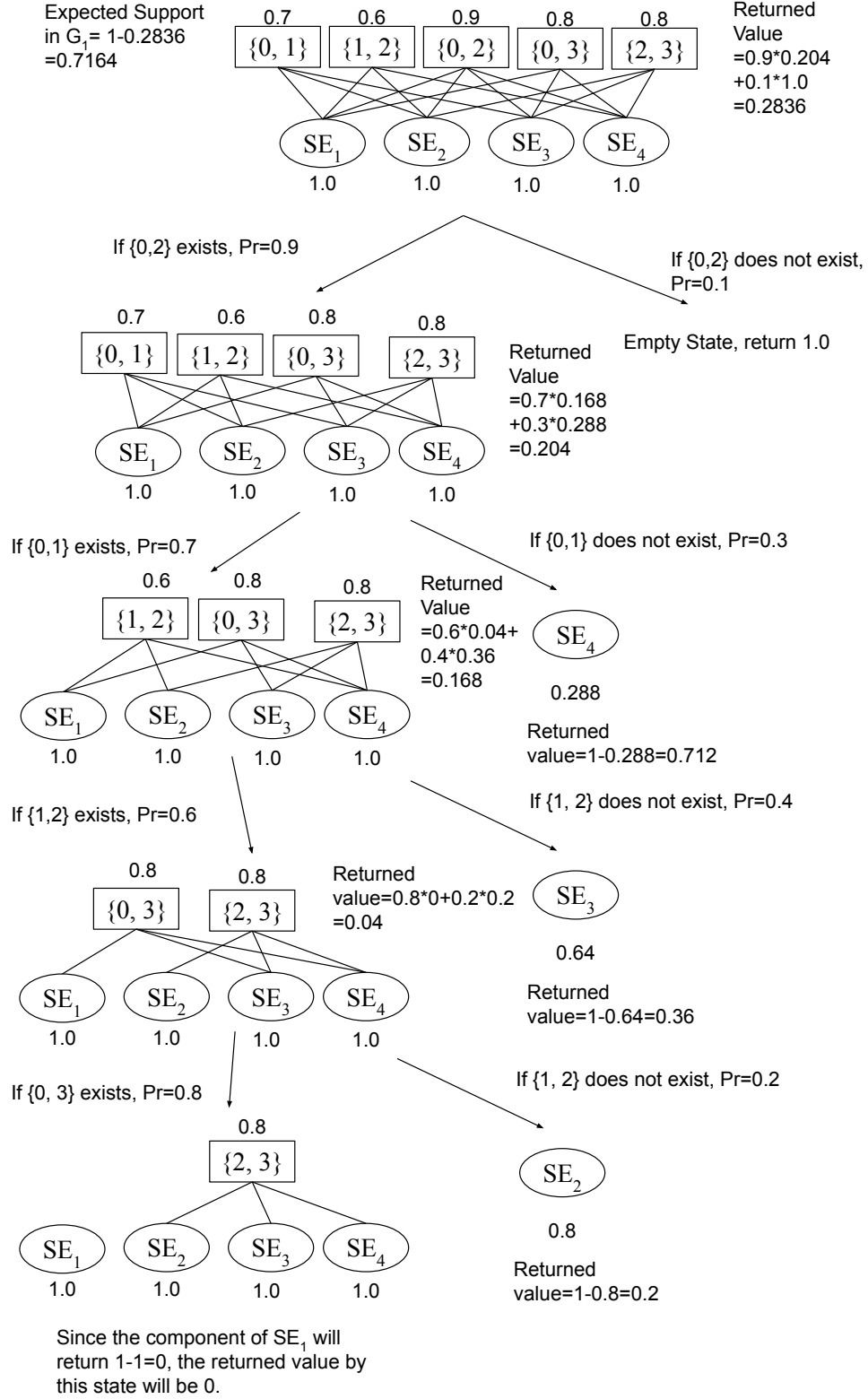
FIGURE 3.16: Expected Support Evaluation of Candidate Pattern 6 in G_0

this subgraph pattern are shown in Fig. 3.18. Since the pattern only has embeddings in G_0 and G_1 , and the cost benefit ratio of G_1 is lower, we first evaluate the edge-embedding graph of G_1 .

Fig. 3.19 shows that the pattern has a support of 0.69 in G_1 . Since this value does not satisfy the conditions for early termination, we proceed to evaluate the edge-embedding graph of G_0 .

Fig. 3.20 shows that the expected support of the pattern in G_0 is 0.828, making the total expected support of Candidate pattern 10 to be $0.69 + 0.828 = 1.518 > 0.90$. Hence, the pattern is frequent. Since this pattern has no canonical extension that has at least one embedding in the graphs of DB in Fig. 3.6, the next pattern UFreS evaluates is the pattern numbered 11 in Fig. 3.7.

Fig. 3.21 shows that candidate pattern 11 has embeddings in G_0 and G_1 ,

FIGURE 3.17: Expected Support Evaluation of Candidate Pattern 6 in G_1

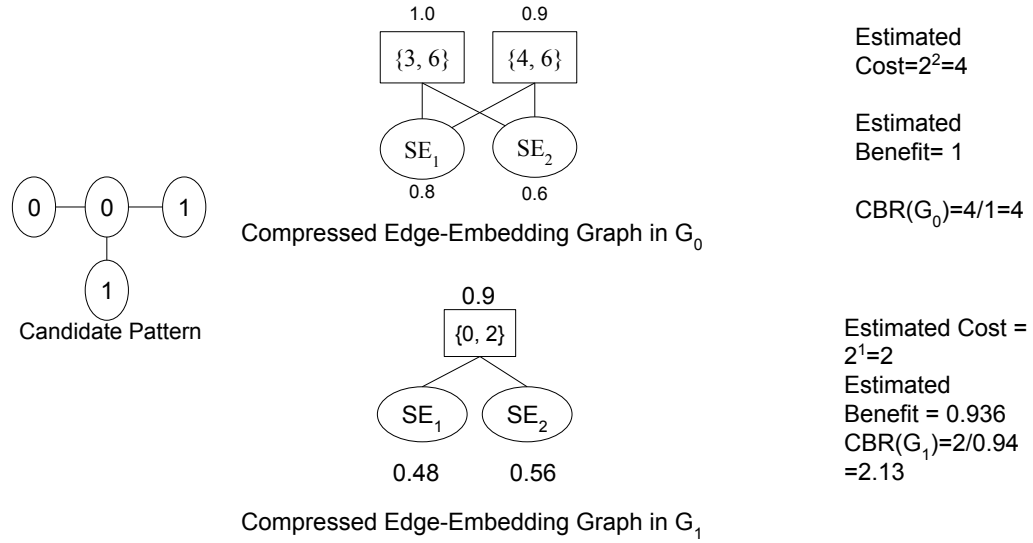
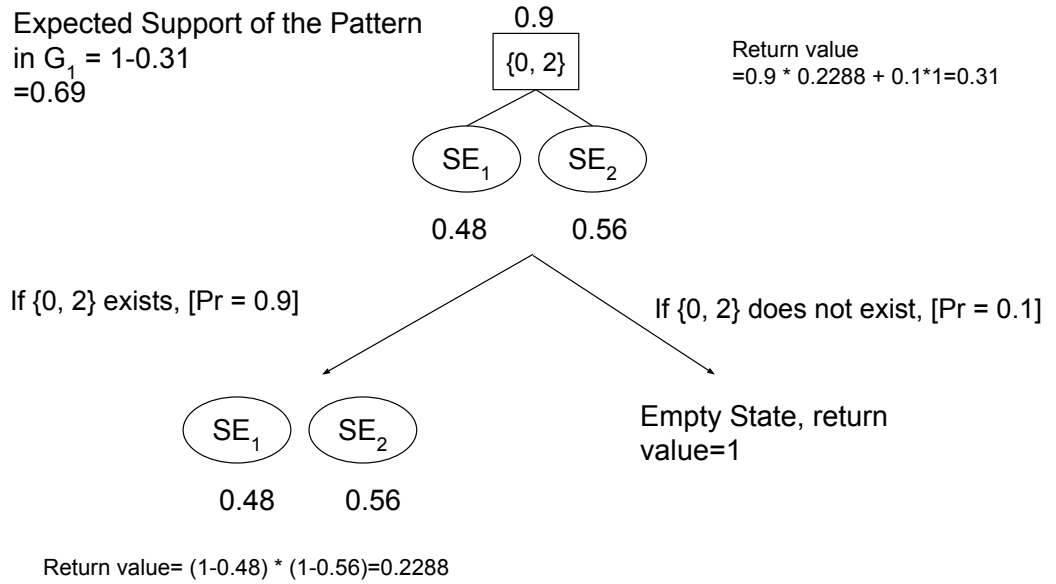


FIGURE 3.18: edge-embedding Graphs Generated for Candidate Pattern 10

FIGURE 3.19: Expected Support Evaluation of Candidate Pattern 10 in G_1

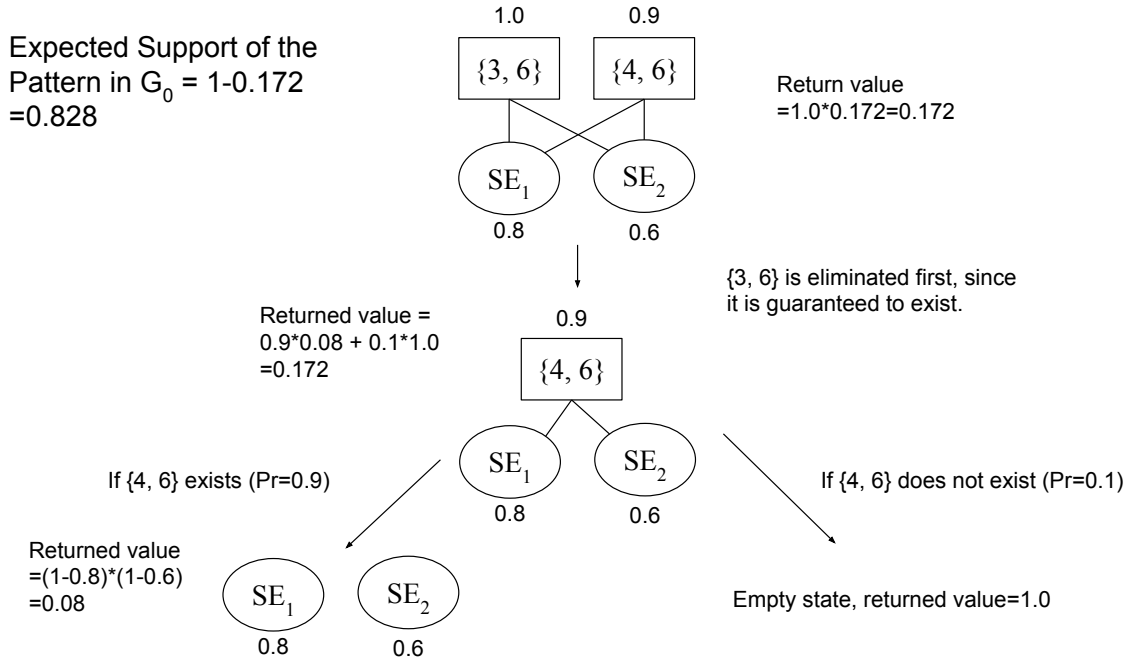
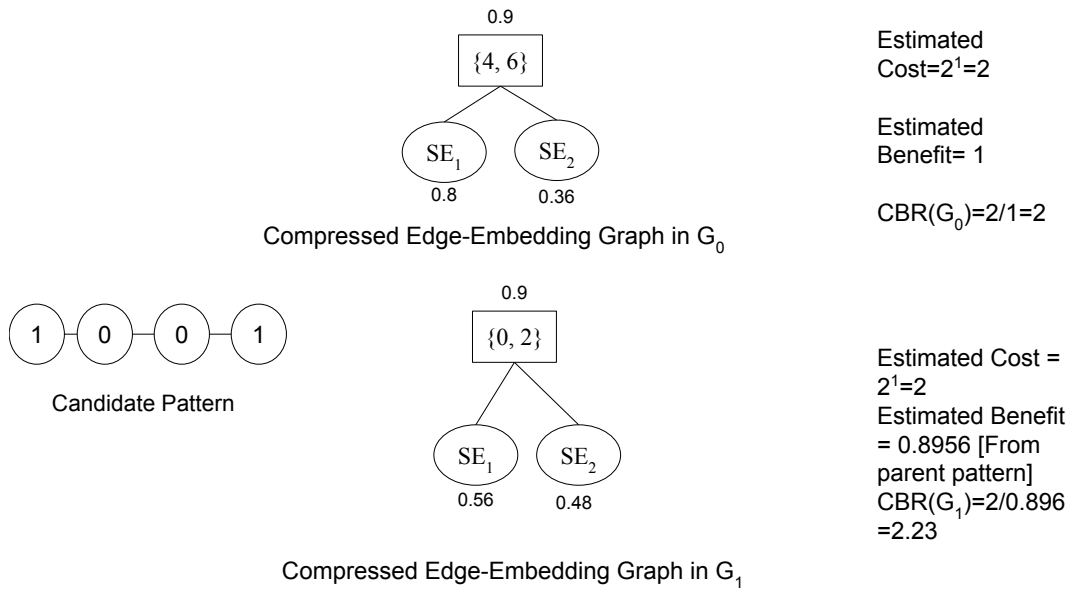
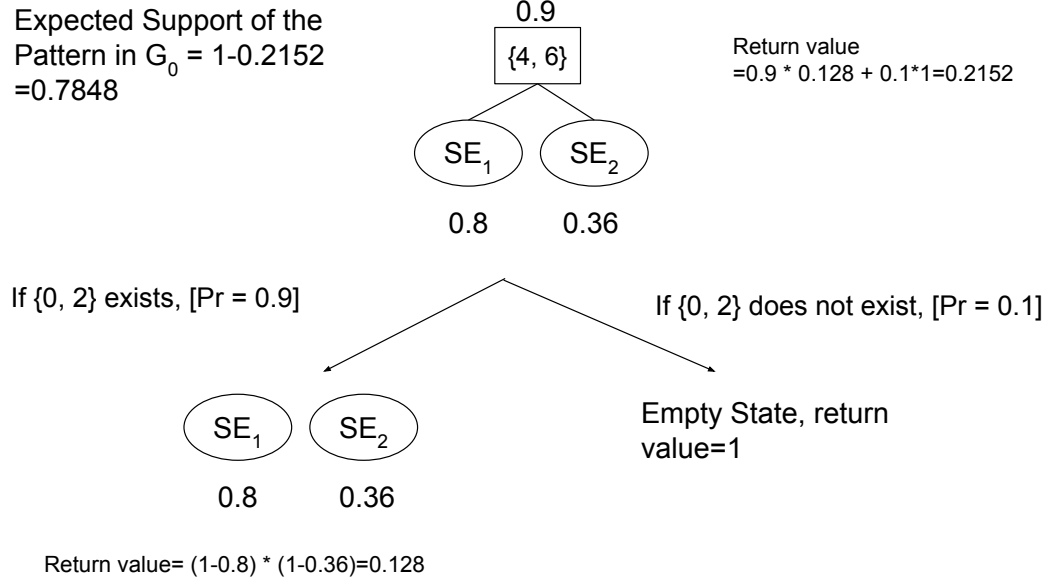
FIGURE 3.20: Expected Support Evaluation of Candidate Pattern 10 in G_0 

FIGURE 3.21: edge-embedding Graphs Generated for Candidate Pattern 11

FIGURE 3.22: Expected Support Evaluation of Candidate Pattern 11 in G_0

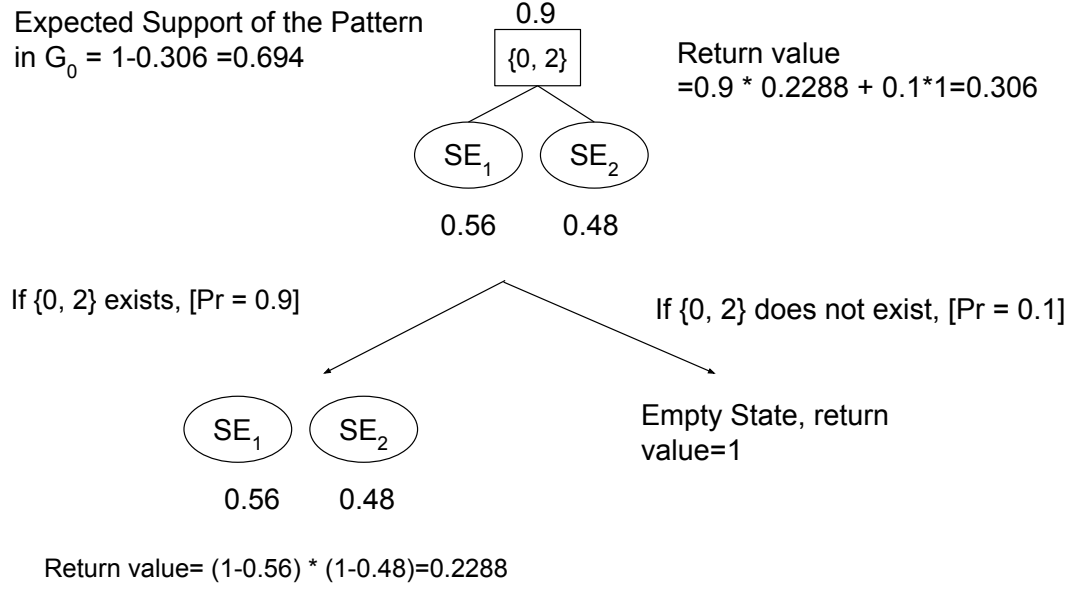
and the edge-embedding graph in G_0 is evaluated first, due to having a lower cost-benefit ratio.

Fig. 3.22 shows that the expected support of the pattern in G_0 is 0.7848. UFreS then evaluates the expected support of the pattern in G_1 .

Fig. 3.23 shows that the expected support of the pattern in G_1 is 0.694, making the total expected support of the pattern $= 0.7848 + 0.694 = 1.4788$. Thus, the pattern is frequent.

As seen in Fig. 3.7, the candidate pattern numbered 11 is the last pattern extended from the single-edged pattern 0-0 to be evaluated in this example. Thus, we can now remove all edges in the graph connecting two nodes which are both labelled 0, and the resulting graph is seen in Fig. 3.24.

The next pattern to be evaluated is the pattern numbered 12 in Fig. 3.7. The

FIGURE 3.23: Expected Support Evaluation of Candidate Pattern 11 in G_1

edge-embedding graphs of the pattern in each graph of the database are shown in Fig. 3.25. The edge-embedding graph of G_2 is evaluated first, since it has the lowest cost-benefit ratio. Being a single-edged subgraph, the evaluation of the pattern is relatively simple.

The expected support of the pattern in G_2 is $1 - (1 - 0.5) \times (1 - 0.4) = 1 - 0.5 \times 0.60 = 0.70$. The next edge-embedding graph to be evaluated, according to the order of the CBR values in Fig. 3.25 is G_1 . Having four embeddings located in separate components, its expected support is $1 - (1 - 0.7) \times (1 - 0.8) \times (1 - 0.8) \times (1 - 0.6) = 0.9952$. Thus, the combined expected support is now 1.6952, and the pattern is thus frequent. Due to early termination, we can skip the evaluation of the edge-embedding graph of G_0 .

The next candidate pattern that *UFreS* evaluates is the pattern numbered 13 in Fig. 3.7. The edge-embedding graphs generated for this candidate is shown in Fig. 3.26. As seen in the figure, the edge-embedding graph seen in G_0 seems

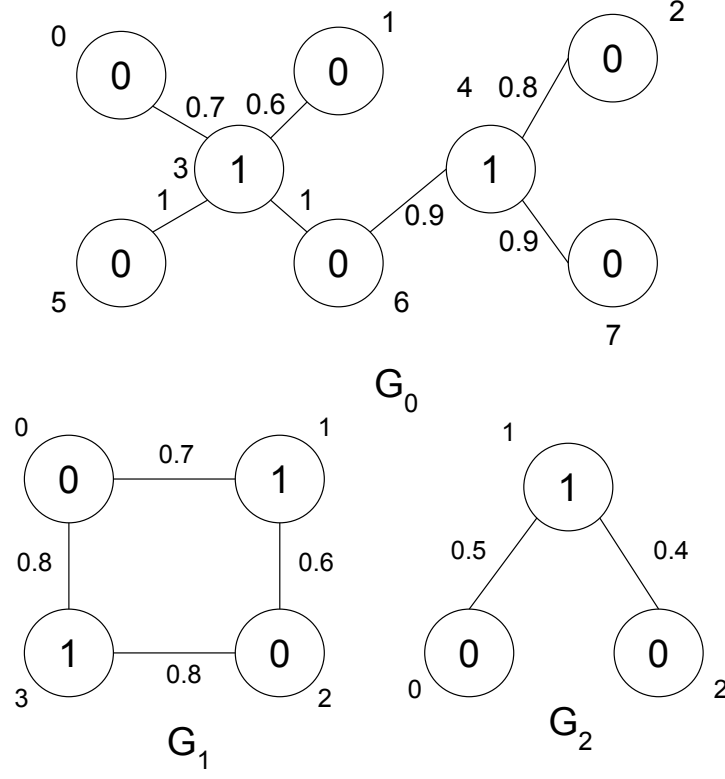


FIGURE 3.24: DB after deleting all 0-0 edges

computationally challenging to evaluate, due to the large number of embeddings the pattern has in G_0 . This is another case where scheduling and early termination comes to our advantage. Due to its higher CBR value, the edge-embedding graph of G_0 is evaluated last, and UFreS first determines the expected support of the pattern in G_1 , and then that of the pattern in G_2 .

The expected support of the pattern in G_1 is $1 - ((1 - 0.64) \times (1 - 0.42)) = 1 - (0.36 \times 0.58) = 1 - 0.2088 = 0.7912$. The expected support of the pattern in G_2 is $1 - (1 - 0.20) = 0.20$. This means that the combined expected support of the candidate pattern numbered 13 in the graphs G_1 and G_2 is $0.7912 + 0.20 = 0.9912$, which satisfies the minimum support threshold. Thus, we can determine that the pattern in question is frequent, and do not need to evaluate the edge-embedding

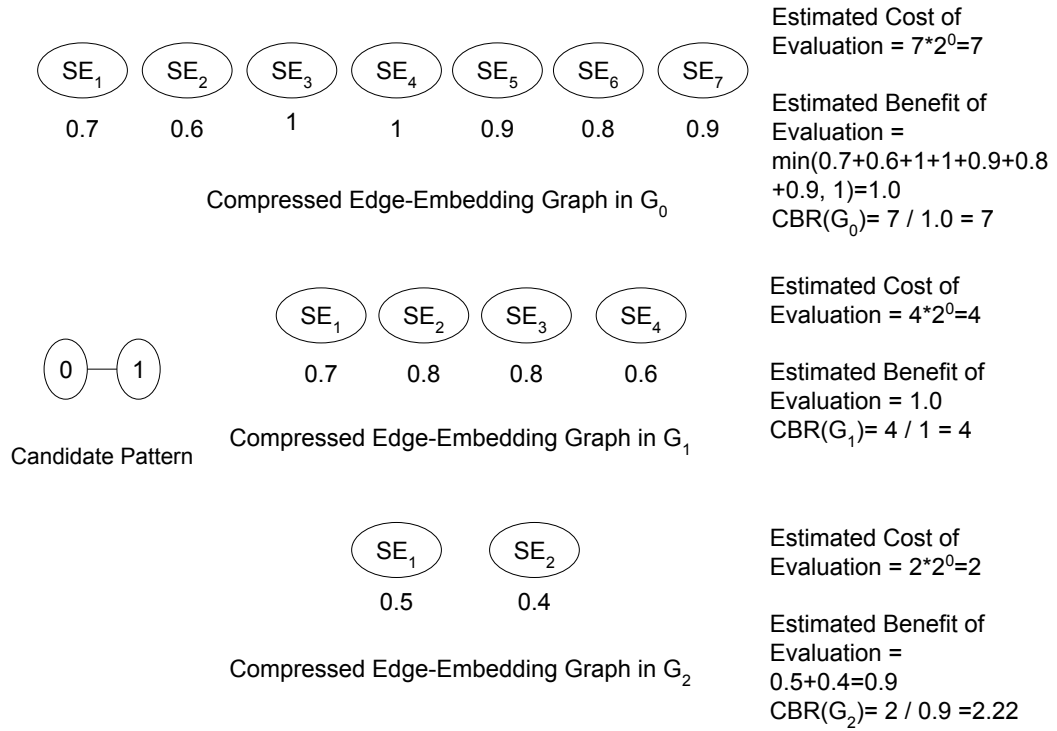


FIGURE 3.25: edge-embedding Graphs Generated for Candidate Pattern 12

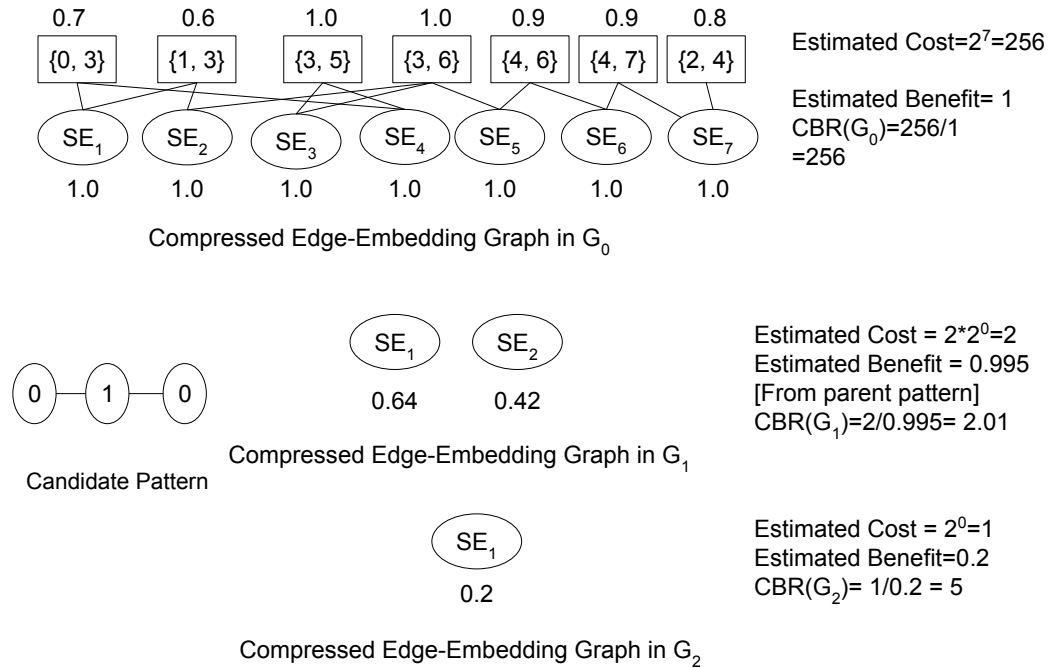


FIGURE 3.26: edge-embedding Graphs Generated for Candidate Pattern 13

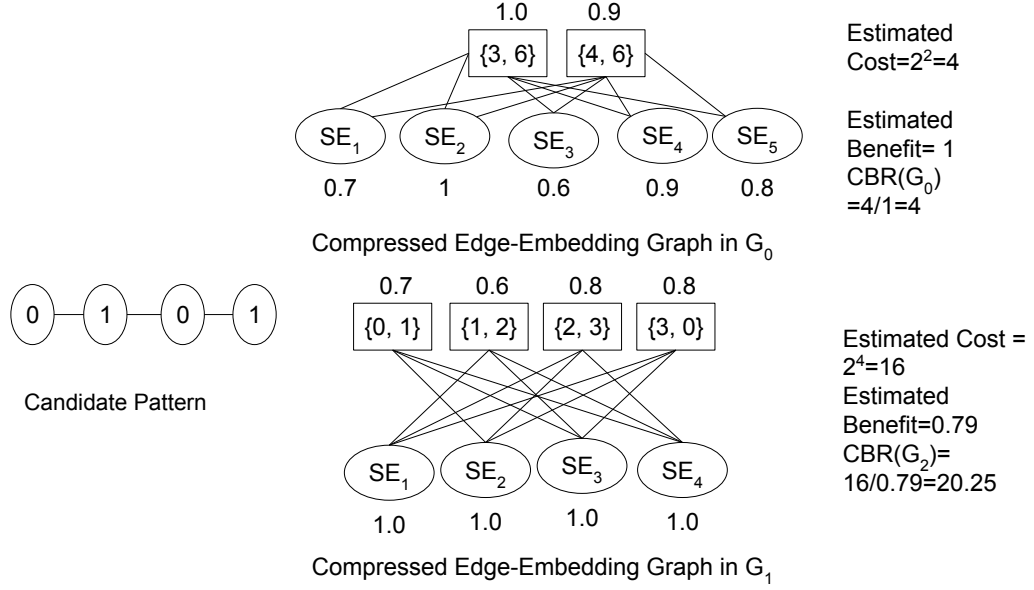


FIGURE 3.27: edge-embedding Graphs Generated for Candidate Pattern 14

graph of G_0 .

The candidate pattern numbered 14 in Fig. 3.7 is the next pattern to be evaluated. It has embeddings in graphs G_0 and G_1 , and the resulting edge-embedding graphs are shown in Fig. 3.27.

Due to its lower cost-benefit ratio, the edge-embedding graph of G_0 is evaluated first. The result of the evaluation, as demonstrated in Fig. 3.28, shows that the expected support of the pattern in G_0 is 0.90, which is equivalent to the minimum required expected support in the entire database. Thus, the pattern in question is frequent, and no further evaluation of edge-embedding graphs is required for this particular pattern.

The remaining frequent pattern whose evaluation remains to be demonstrated is that of the candidate pattern numbered 19 in the subgraph generation tree of

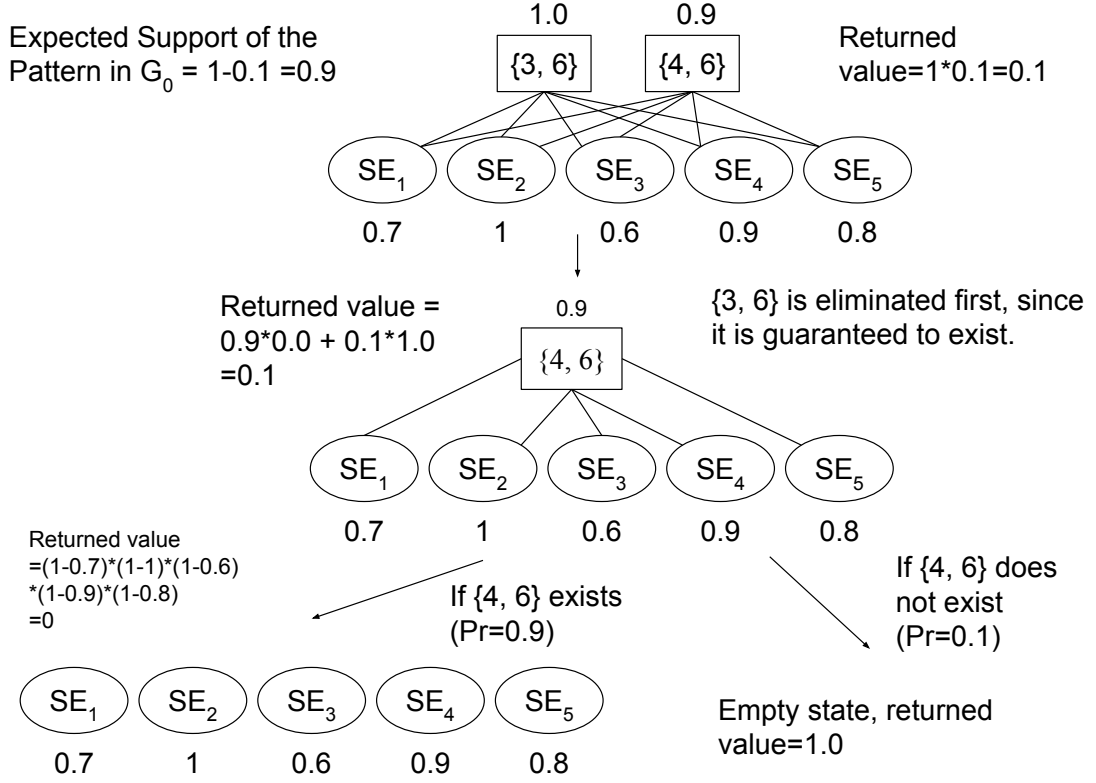
FIGURE 3.28: Expected Support Evaluation of Candidate Pattern 14 in G_0

Fig. 3.7. The edge-embedding graphs generated for this candidate subgraph pattern are shown in Fig. 3.29.

Evaluating the edge-embedding graphs generated for the candidate pattern 19 is relatively straightforward, since they do not contain any edge nodes. The candidate pattern has an expected support of 0.9 in G_0 . Evaluation of the edge-embedding graph of G_1 is skipped due to early termination, since the minimum support threshold has already been met.

In this way, by performing a depth first traversal of the candidate patterns shown in Fig. 3.7, and evaluating their expected support one by one, UFRoS mines the frequent subgraphs shown in Fig. 3.30 from the example uncertain graph database shown in Fig. 3.6 for the minimum expected support threshold

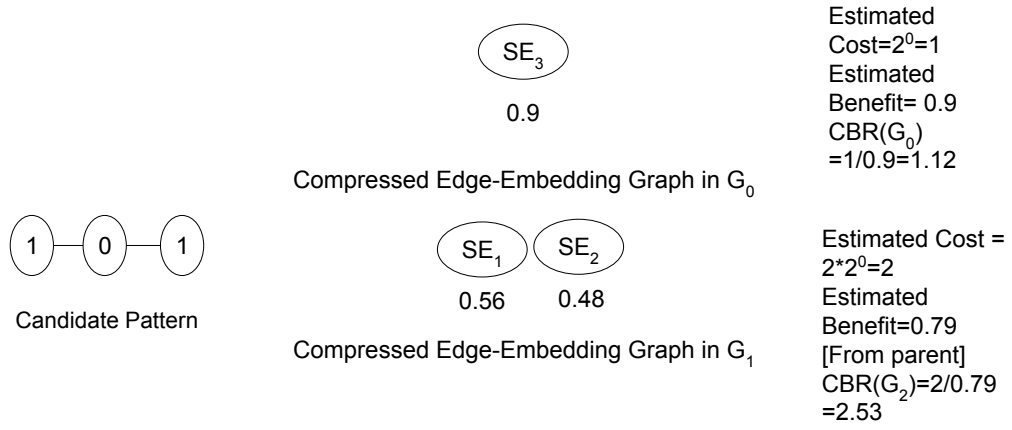
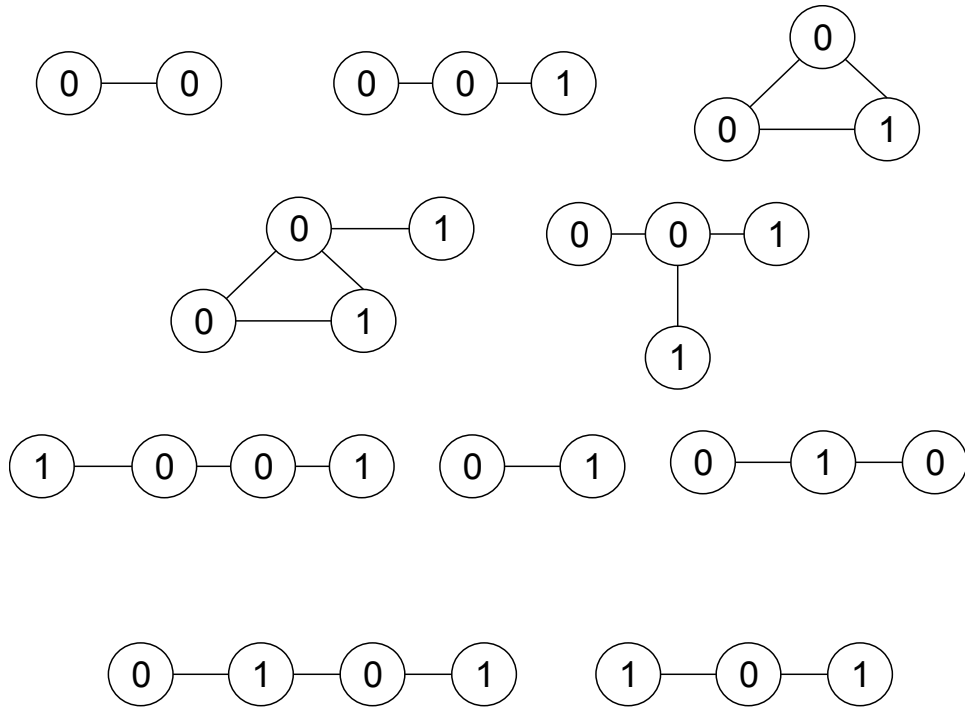


FIGURE 3.29: edge-embedding Graphs Generated for Candidate Pattern 19

FIGURE 3.30: Frequent Patterns mined by UFreS for $\tau=0.90$ in the Sample Database

value of $\tau = 0.90$. Having explained the working principle of UFreS, we now turn our focus towards analyzing the theoretical upper bound of the time complexity of our approach. In the following Section, we briefly discuss what factors impact the runtime of UFreS.

3.5 Time Complexity of UFreS

The time complexity of our proposed approach for evaluating an edge-embedding graph, as described in Algorithm 1, is equal to the product of the number of states visited by the recursive function described in Algorithm 2, the number of edge nodes and the size of the edge-embedding graph. A loose upper bound of the worst case time complexity can be defined as $O(2^m * m * (p * |SE_c|))$, where m is the number of edges contained in several embeddings, p is the number of edges in the subgraph pattern and $|SE_c|$ is the number of embeddings in a single component of the edge-embedding graph. Here, 2^m represents the number of states visited in the worst case. In each state, for selecting which edge to eliminate next, we loop over each edge node (with worst case complexity $O(m)$). For each edge node e , we traverse the edge-embedding graph once to determine the number of edge nodes in the resulting components of the edge-embedding graph if e is eliminated. Traversing the edge-embedding graph requires us to visit each node and each edge in it. Since the number of edges is higher than the number of nodes, the time complexity of the graph traversal step is dominated by the number of edges in the edge-embedding graph. To determine the number of edges in an edge-embedding graph, let us consider that each embedding node is connected with p edge nodes, making the total number of edges in the graph is $p * |SE_C|$. Multiplying these complexities together gives us the worst case complexity we discussed earlier.

Since we continually keep compressing and splitting the edge-embedding graph, and non-arbitrarily select the edge to eliminate next, the true runtime of the algorithm is far less in practice than implied by this upper bound. This

loose upper bound of the worst case time complexity is also better than that of the computational complexity of the algorithm implied by the eq. 2.4, whose inclusive exclusive approach has a worst case time complexity of $O(2^{|SE|}E)$ where $|SE|$ is the total number of embeddings of a subgraph pattern and E is the total number of edges included in them.

The runtime of the overall mining process is dependent on the number of candidate patterns examined, which is in turn dependent on the threshold the user specifies. Usually, the lower the user defined threshold, the higher the number of candidate subgraphs explored and the greater the runtime. Having seen that the time complexity of finding the expected support of a pattern in a graph is $O(2^m * m * p * SE_c)$, if the number of candidate patterns examined is C and the number of graphs in the database is $|D|$, the overall time complexity can be defined as $O(C * |D| * 2^m * m * p * SE_c)$, since we shall have to find the expected support of C patterns in all $|D|$ graphs in the worst case. This is a loose upper bound, and the optimization techniques mentioned greatly help to reduce the runtime.

For instance, although we consider that UFreS might have to evaluate the expected support of each candidate pattern in each uncertain graph of the database, a lot of these evaluations will be avoided in practice due to early termination. Scheduling further helps to boost the performance enhancements gained by early termination, making the number of expected support evaluations much lesser in practice. Furthermore, although we consider that we might have to visit 2^m states for each edge-embedding graph evaluation, this is once again an even that is unlikely to occur in practice due to the compression, splitting and edge selection taking place in each step. In practice, the overall runtime of UFreS is much lower than what the reader may anticipate upon reading the aforementioned worst case time complexity. This is evidenced by the results of our empirical analysis on real world datasets, described in the following chapter.

Chapter 4

Experimental Results

In this chapter, we discuss the results of our experiments on real world datasets, using which we compared the performance of our own approach with that of the existing approaches. In the first section, we provide details about our experimental setup and describe the datasets that we used for our experiments. In the sections that follow, we show how UFreS outperforms the existing approaches in terms of runtime, memory consumption and accuracy of results respectively. Finally, we provide empirical evidence to show the scalability of UFreS.

4.1 Experimental Setup and Dataset Description

We compared the performance of our proposed approach *UFreS* with that of the existing approaches *UGRAP*[25] and *MUSE*[24] on a total of ten different variants of four real world datasets. All three algorithms were implemented in C++, and all experiments were run on a PC with an Intel Core i7 11th Gen processor, 8GB RAM, 512 GB SSD space and 12MB of Cache in a Windows 10 Operating System. Runtime, peak memory consumption and accuracy of the results formed the basis of our comparisons. The following datasets were used for our analysis:

- **STRING**[33]: An open-source uncertain graph database documenting Protein-Protein Interactions in various organisms, where the nodes and edges represent proteins and their interactions respectively. The labels of the nodes in the database represent the COG functions of the proteins, and the edge probabilities represent the confidence levels with which the existence of each interaction has been identified. The PPI network of each organism comprises an uncertain graph in the database. We used the networks of 14 different mammals in our experiments, namely baboons (*Papio anubis*), cats (*Felis catus*), chimpanzees (*Pan troglodytes*), cows (*Bos taurus*), elephants (*Loxodonta africana*), gorillas (*Gorilla gorilla*), humans (*Homo sapiens*), orcas (*Orcinus orca*), minke whales (*Balaenoptera acutorostrata*), pigs (*Sus*), polar bears (*Ursus maritimus*), rabbits (*Oryctolagus cuniculus*), sheep (*Ovis aries*), tigers (*Panthera tigris*) and wolves (*Canis lupus*), and mined frequent patterns of protein-protein interactions among them. *UGRAP* and *MUSE* were too inefficient to be used for mining from the entire database, and consumed so much RAM that our computer crashed after running for hours. Thus, we used the random area selection sampling method used in [57] to select a small subset of the nodes and edges from each graph to enable us to gain results using all three approaches and perform comparative analysis.
- **NTP**[56]: The National Toxicology Program released this dataset documenting the effect of different chemical compounds on rodents. For our experiments, we used the version released in the Predictive Toxicology Challenge¹. The dataset has been widely used previously to evaluate the efficacy of various frequent subgraph mining approaches [25]. We injected uncertainty into its edges synthetically, following three different probability distributions- normal distributions with mean and standard deviation values set to (0.8, 0.1) and (0.4, 0.05) respectively, and an exponential distribution

¹paperswithcode.com/dataset/ptc

with $\lambda = 4.0$. Min-max normalization was applied on the probability values sampled from the exponential distribution.

- **Taxi**[19]: A trajectory dataset containing the routes of 106 taxi trips in Porto, Portugal. The tuples in the dataset consisted of locations of the taxis in successive timestamps. Locations that were within 20 metres of the locations appearing in previous tuples were replaced by their predecessors. Each trip was considered as a graph in the dataset, the points were treated as nodes and points in consecutive timestamps were considered to be connected by an edge. The nodes were labelled into 10 different groups, according to the deciles of the average velocity of the taxis in them, representing how busy the points usually were. Uncertainties were synthetically injected to the edges according to the same distributions followed in the NTP dataset, namely with normal distributions having mean and standard deviation value pairs of $(\mu=0.8, \sigma=0.1)$ and $(\mu=0.4, \sigma=0.05)$ respectively, and with an exponential distribution of $(\lambda = 4)$. However, this time around, we avoided normalizing the uncertainty values generated from the underlying exponential distribution.
- **AIDS Antiviral Screening Data**[58]: This graph repository, published by the Developmental Therapeutics Program (DTP), contains information about the structure of chemical compounds which have been tested for traces of antiviral activity. Each atom in the molecular structure of these compounds represents a vertex, while each bond between two atoms is represented by an edge between the corresponding vertices. The vertices and edges were labelled according to the types of their corresponding atoms and bonds respectively. For our experiments, we used the version provided by Morris et. al. [59]. Uncertainties were synthetically introduced to the edges using the same underlying distributions used for the Taxi dataset. Similar to what happened in the case of the STRING database, UGRAP and MUSE were too inefficient and consumed too much memory, causing our local device to

crash after hours of execution. In the interest of performing comparative analysis, we used random area selection sampling [57] to take a representative sample of each graph in the database, to enable UGRAP and MUSE to produce results.

Name of Dataset	# of Graphs	# of Nodes	# of Edges	# of Node Labels	# of Edge Labels	# of Distinct Label triplets
STRING	14	210	326	16	1	100
NTP	340	9189	9317	66	4	207
Taxi	106	3074	7844	10	1	98
AIDS	399	4106	3854	27	3	82

TABLE 4.1: Dataset Attributes

Table 4.1 summarizes details about different attributes of these datasets and Fig. 4.1 shows the distribution of uncertainty values in them. For the approximation schemes of *UGRAP* and *MUSE*, the values of ϵ and δ were both set to 0.1. Changing these values would result in either increased usage of runtime and memory or decreased accuracy of results.

While demonstrating our experimental results, we express all threshold values as percentages of the total number of graphs in the database. That is, a threshold value listed as 5% in the figures that follow for a database consisting of $|D|$ graphs is equivalent to a threshold value $\tau = 0.05 \times |D|$, as specified in the problem statement in section 2.1. For example, a threshold value of 10% in the NTP dataset means that the algorithms are to mine the frequent patterns that have an expected value of at least $0.10 \times 340 = 34$ in the dataset.

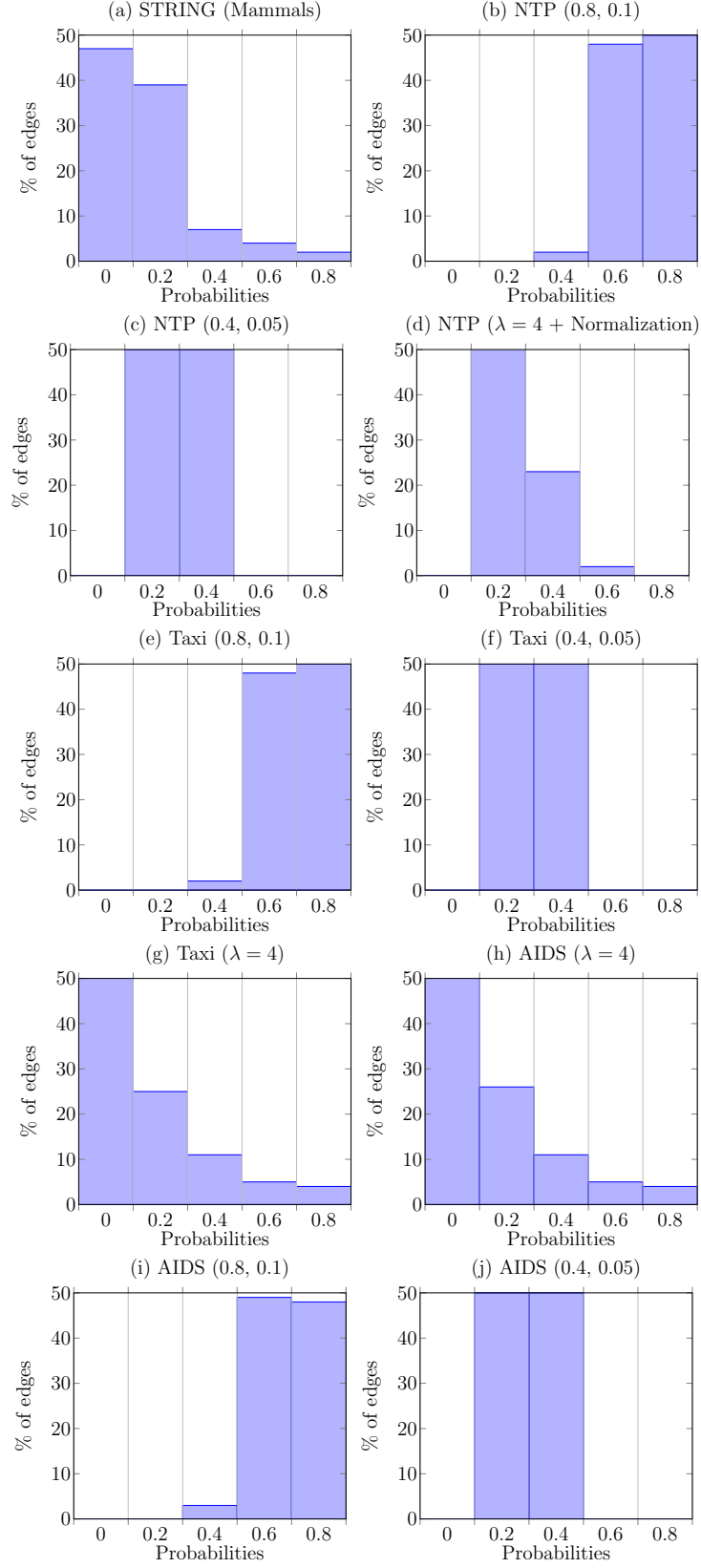


FIGURE 4.1: Distribution of Uncertainty values in the datasets

4.2 Runtime Comparisons

We first compared the runtimes of the respective approaches, since time complexity remains at the heart of the challenges faced in our problem. Fig. 4.2 shows that *UFreS* outperforms *MUSE* and *UGRAP* by some distance in this respect. For lower thresholds, as the number of frequent subgraphs keep increasing, the runtime of the algorithms increase along with it. In several instances, *MUSE* and *UGRAP* failed to terminate even after running for hours for low thresholds. *UFreS* encountered no such issues on those same threshold values and produced results in a matter of seconds. As Fig. 4.2 shows, our experiments proved *UFreS* to be several orders of magnitude faster than the existing approaches *MUSE* and *UGRAP*.

To investigate why this significant improvement in runtime occurs, we compare our expected support evaluation schemes with the exact and approximate-dual approach adopted by both *UGRAP* and *MUSE*. As displayed by the red dots in the upper half of the scatterplots in Fig. 4.3-4.9, the expected support evaluation scheme of *UGRAP* and *MUSE* requires a significantly higher number of CPU clock cycles when the number of embeddings of a pattern in an uncertain graph is high. On the other hand, our edge-embedding graph based support evaluation scheme remains almost unaffected even when there are many embeddings high, as evidenced by the low-lying blue dots in the Figures 4.3- 4.12.

Aside from being highly efficient while evaluating the expected support of each pattern, scheduling and early termination also helps to reduce the number of times the expected support module needs to be called. *UGRAP* introduced these approaches in the field of uncertain frequent subgraph mining, and thus requires a lower number of expected support evaluations than *MUSE*, as shown in Fig 4.13. We adopted and modified these steps for *UFreS*, and the number of evaluations required by *UFreS* is similar to, and in many instances even lower

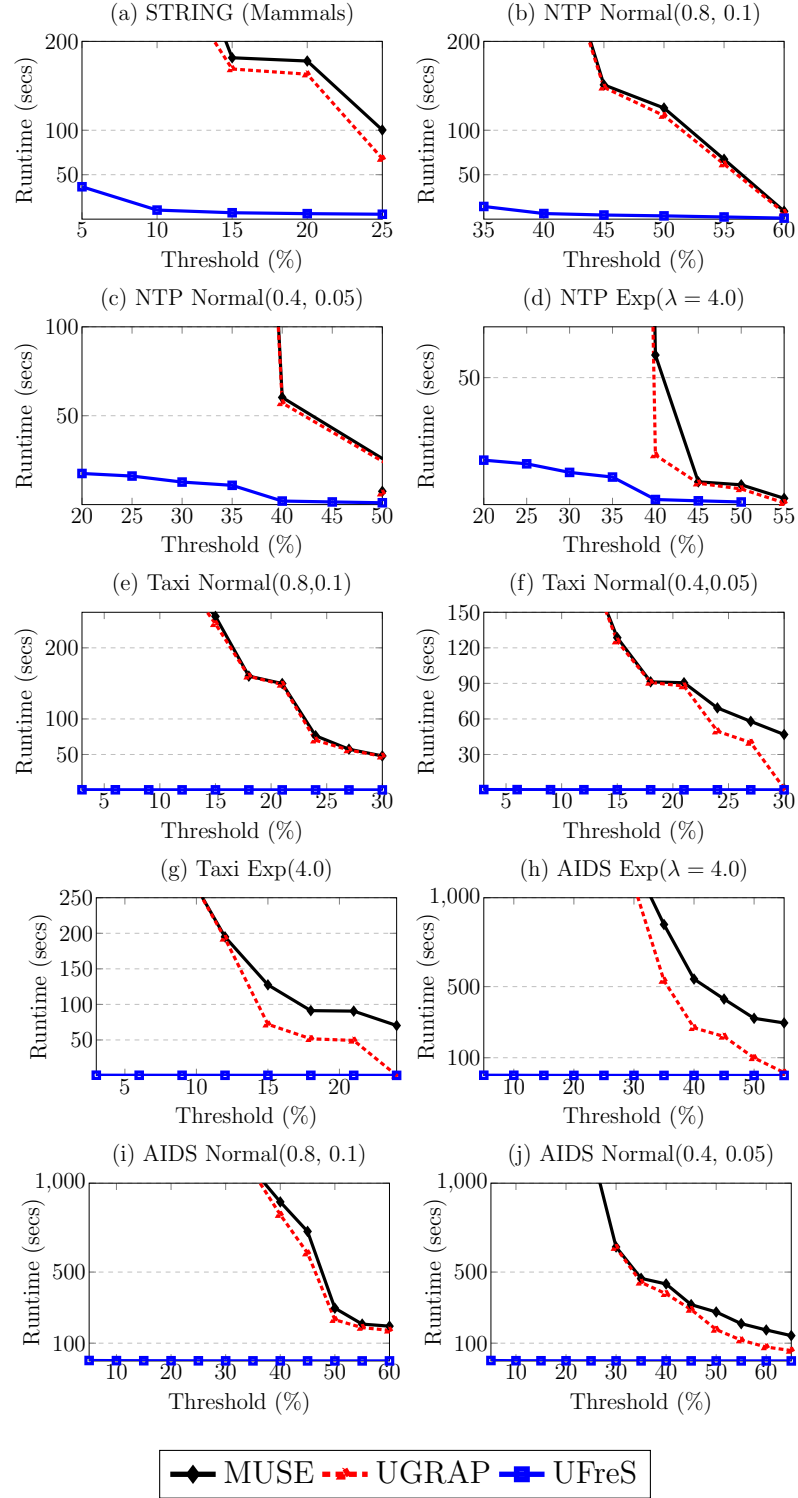


FIGURE 4.2: Runtime comparison between MUSE, UGRAP and UFreS

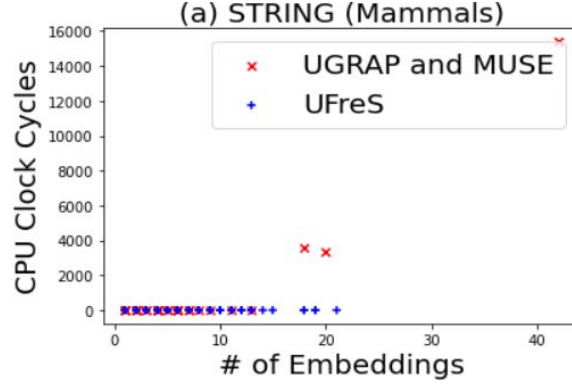


FIGURE 4.3: Comparison of time taken by the Expected Support Evaluation Schemes in the STRING Database [$\tau = 15\%$]

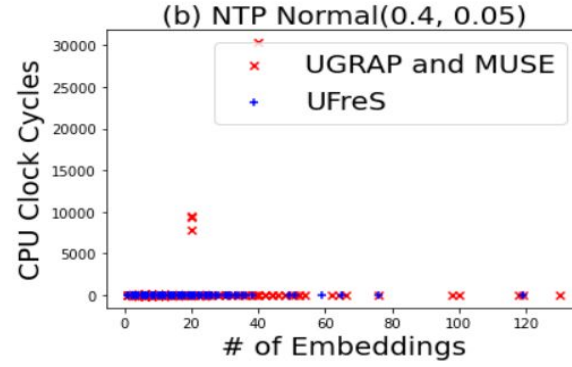


FIGURE 4.4: Comparison of time taken by the Expected Support Evaluation Schemes in the NTP Normal(0.8, 0.1) Dataset [$\tau = 45\%$]

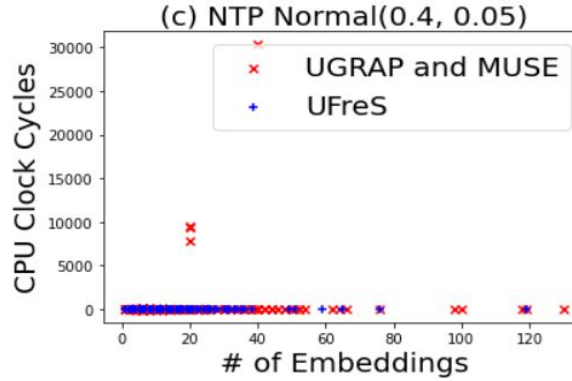


FIGURE 4.5: Comparison of time taken by the Expected Support Evaluation Schemes in the NTP Normal(0.4, 0.05) Dataset [$\tau = 40\%$]

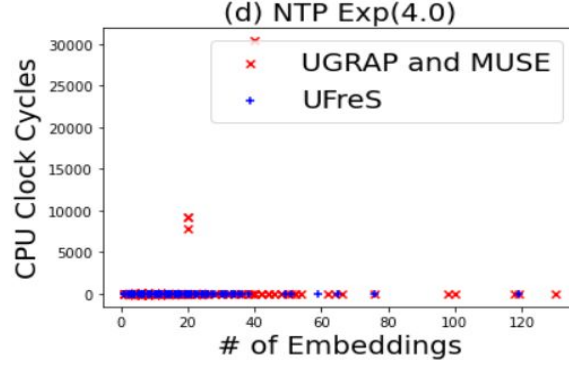


FIGURE 4.6: Comparison of time taken by the Expected Support Evaluation Schemes in the NTP Exp($\lambda = 4.0$) Dataset [$\tau = 40\%$]

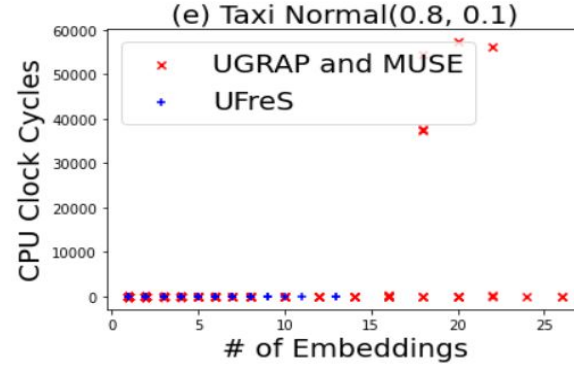


FIGURE 4.7: Comparison of time taken by the Expected Support Evaluation Schemes in the Taxi Normal(0.8, 0.1) Dataset [$\tau = 15\%$]

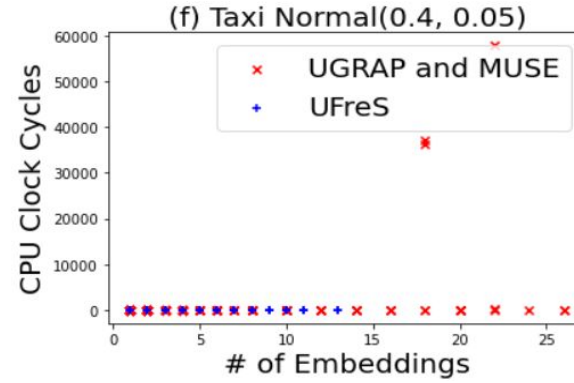


FIGURE 4.8: Comparison of time taken by the Expected Support Evaluation Schemes in the Taxi Normal(0.4, 0.05) Dataset [$\tau = 15\%$]

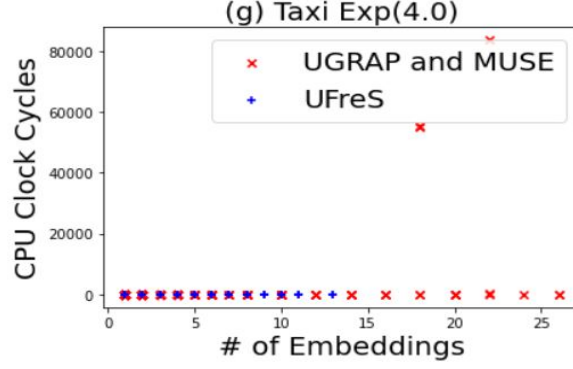


FIGURE 4.9: Comparison of time taken by the Expected Support Evaluation Schemes in the Taxi Exp($\lambda = 4.0$) Dataset [$\tau = 12\%$]

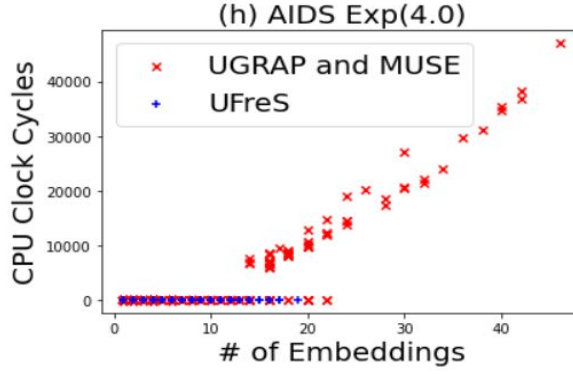


FIGURE 4.10: Comparison of time taken by the Expected Support Evaluation Schemes in the AIDS Exp($\lambda = 4.0$) Dataset [$\tau = 35\%$]

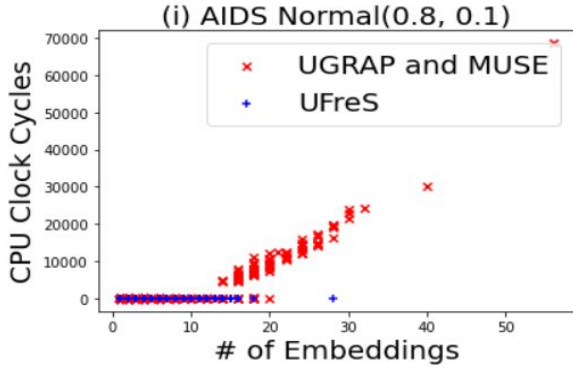


FIGURE 4.11: Comparison of time taken by the Expected Support Evaluation Schemes in the AIDS Normal(0.8, 0.1) Dataset [$\tau = 35\%$]

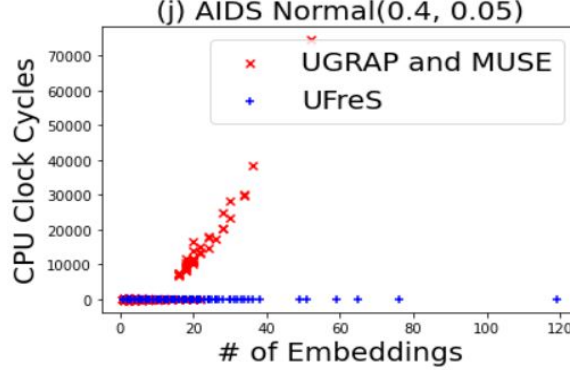


FIGURE 4.12: Comparison of time taken by the Expected Support Evaluation Schemes in the AIDS Normal(0.4, 0.05) Dataset [$\tau = 30\%$]

than the number required by UGRAP. Interestingly, the number of required evaluations sometimes decreases for smaller thresholds. This happens because when the threshold is lower, patterns may be deemed frequent earlier due to early termination. The combination of better evaluation schemes and a lower number of required evaluations thus helps UFreS to be several orders of magnitude faster than its predecessors.

For instance, in the normally distributed variant of the AIDS dataset where the underlying probability distribution of the uncertainty values has a mean of 0.4 and a standard deviation of 0.05, when the threshold is set to 30%, Fig. 4.11 shows that for the same number of embeddings of a pattern in a graph, the number of clock cycles required for determining the expected support using the method employed by both MUSE and UGRAP is four orders of magnitude slower than our edge-embedding graph based technique. Furthermore, the total number of evaluations required by UFreS for said value of τ is 1764, which is almost 12% less than the number of evaluations required by UGRAP and almost 22% less than that required by MUSE, which does not employ any form of early termination and scheduling whatsoever. UFreS thus only requires half a second to run in this case, while both UGRAP and MUSE required over 10 minutes to terminate in our experimental setup.

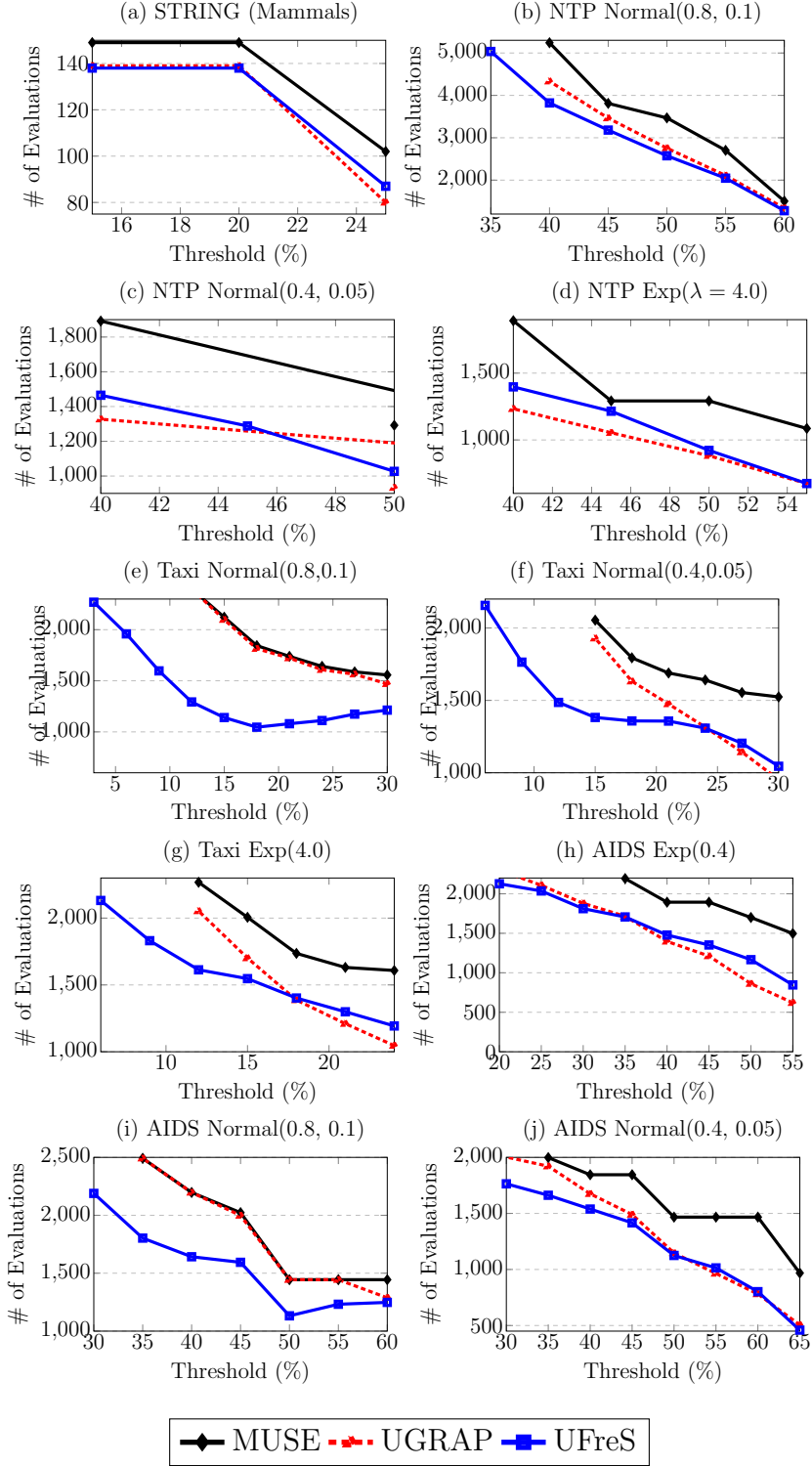


FIGURE 4.13: No. of expected support evaluations required by MUSE, UGRAP and UFreS

4.3 Memory Usage Comparisons

Having compared the runtimes of UFreS with that of the existing state-of-the-art approaches, we now turn our attention to comparing the peak memory requirement of these algorithms. While memory requirements are often traditionally construed as less of a concern than runtime and the accuracy, we note here that the memory requirement of UGRAP and MUSE rises exponentially with the number of embeddings a pattern has. This exponential rise in memory requirement remains a vulnerability of these approaches, and hinders the application of these approaches in large-scale databases.

The results of our comparative empirical analysis on the memory requirements of these algorithms is seen in Fig. 4.14, the amount of memory required by UGRAP is the highest of the three algorithms, due to its use of indexing. However, in most cases, this difference between the two pre-existing algorithms is not too significant.

The memory usage of both UGRAP and MUSE increases drastically for lower threshold values. For instance, for a threshold value of 10% in the STRING database, the memory required by both UGRAP and MUSE exceeded 5.5 GB of RAM, and yet, the algorithms did not terminate. The reason for this drastic increase in memory consumption is that MUSE and UGRAP adopt a caching-based optimization technique to speed up their exact expected support evaluation scheme. While this does help reduce the runtime to an extent, the costs paid in terms of memory is huge. UFreS' memory requirement, however, does not suffer from such sudden spikes, making it a much lighter and memory-friendly approach.

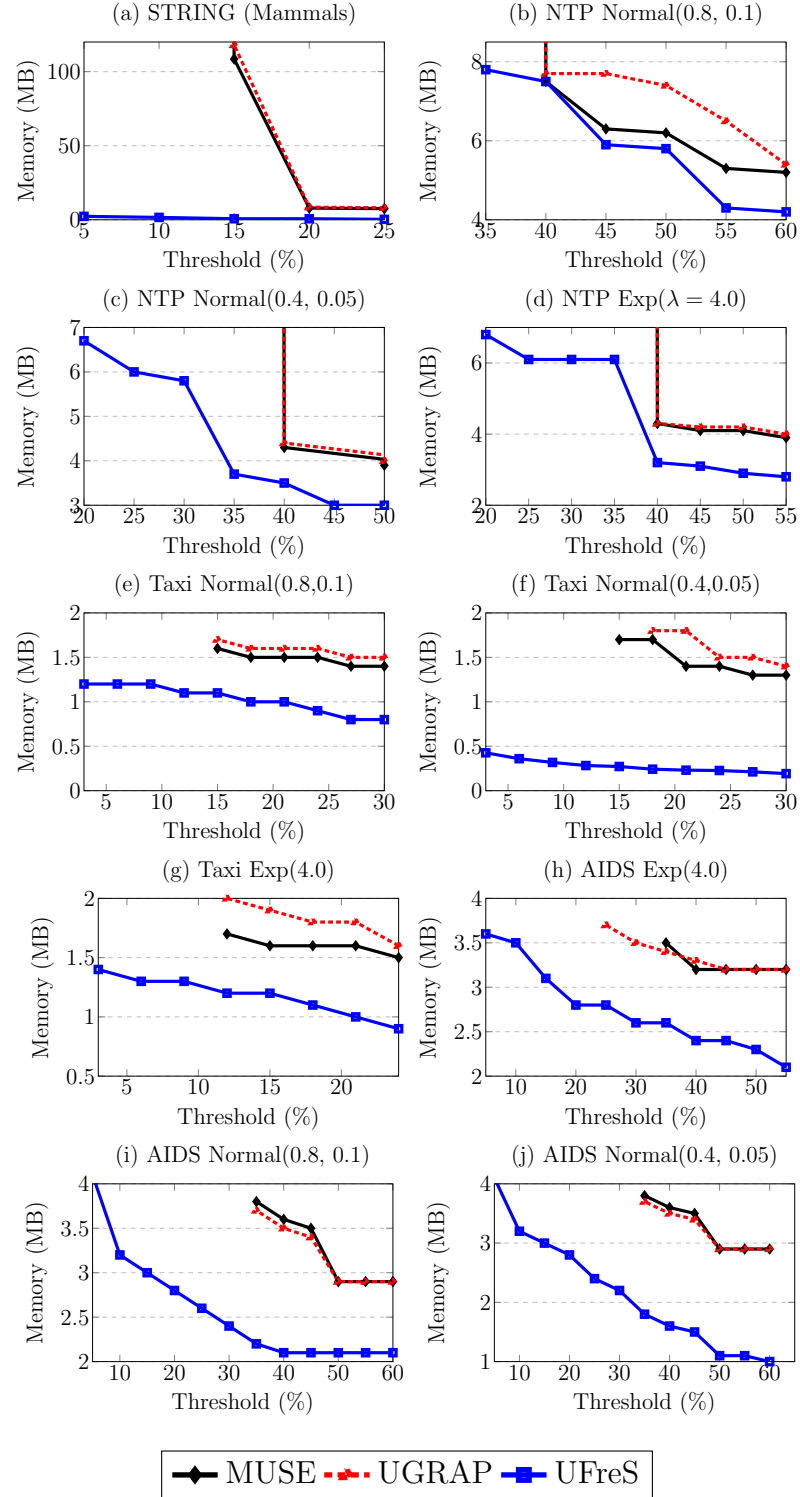


FIGURE 4.14: Peak Memory Usage Comparison between MUSE, UGRAP and UFreS

4.4 Accuracy of results

We now compare our proposed approach UFreS, with the existing approaches on the basis of the soundness and completeness of their results. Being completely non-reliant on any sort of approximation scheme allows UFreS to always produce a sound and complete set of frequent subgraphs. The same cannot be said for UGRAP and MUSE, however, as their dependence on FPRAS makes them susceptible to both false positives and false negatives. For the same threshold values, the output of UGRAP and MUSE remains the same[25]. In the STRING database, both these approaches failed to mine more than 3 subgraph patterns even after running them for over a day. Table 4.2 shows the precision and recall values of the set of patterns mined by UGRAP and MUSE for different threshold values. In contrast to our approach, which shall always have precision and recall values of 100%, Table 4.2 shows that these approaches do miss out on mining subgraph patterns that are actually frequent and falsely identify infrequent patterns as frequent. The accuracy of UGRAP and MUSE would decrease further if greater values of ϵ and λ were used for our experiments.

NTP(0.8, 0.1)		NTP(0.4, 0.05)	
Threshold	Recall (%)	Threshold	Recall (%)
60	100	50	100
50	90.9	45	75
40	100	40	66.67
AIDS Exp(4.0)		Taxi(0.4, 0.05)	
Threshold	Precision (%)	Threshold	Recall (%)
55	50	30	100
45	50	21	100
35	33.33	15	96.97

TABLE 4.2: Sample Precision and Recall Values of UGRAP and MUSE in our experiments. Values marked red represent instances where the pre-existing approaches produced false results

4.5 Scalability of UFreS

With the incredible volume of data being that is now being increasingly available in data repositories around the world nowadays, scalability remains a major concern for all novel data mining algorithms that are being proposed. In this section, we explore how the performance of UFreS is affected by an increase in the number of graphs in the database. While the runtime and memory consumption of any algorithm that does not have a constant time or memory complexity is expected to increase with the size of the input, a worse than linear increase in runtime or memory consumption typically indicates that an algorithm is not well-suited for being feasibly used in large datasets.

To test the scalability of our approach, we adopt the same approach followed in [24] and [25]. We constructed derived databases consisting of duplicate copies of all graphs in the original datasets. The number of duplications of each graph was varied from 1 to 10. The advantage of this duplication technique is that it allows us to measure how the increase in the number of graphs affects the performance of the algorithm, while keeping all other aspects of the data unchanged. It is to be noted that the number of duplications does not affect the number of subgraphs that are mined and the number of candidates that are generated.

The plots in Fig. 4.15 and 4.16 show that the runtime and memory requirement of our approach increases linearly with the number of duplications in all datasets, and in some cases shows lower-than-linear growth in runtime and memory, providing evidence of how well UFreS can scale for large inputs.

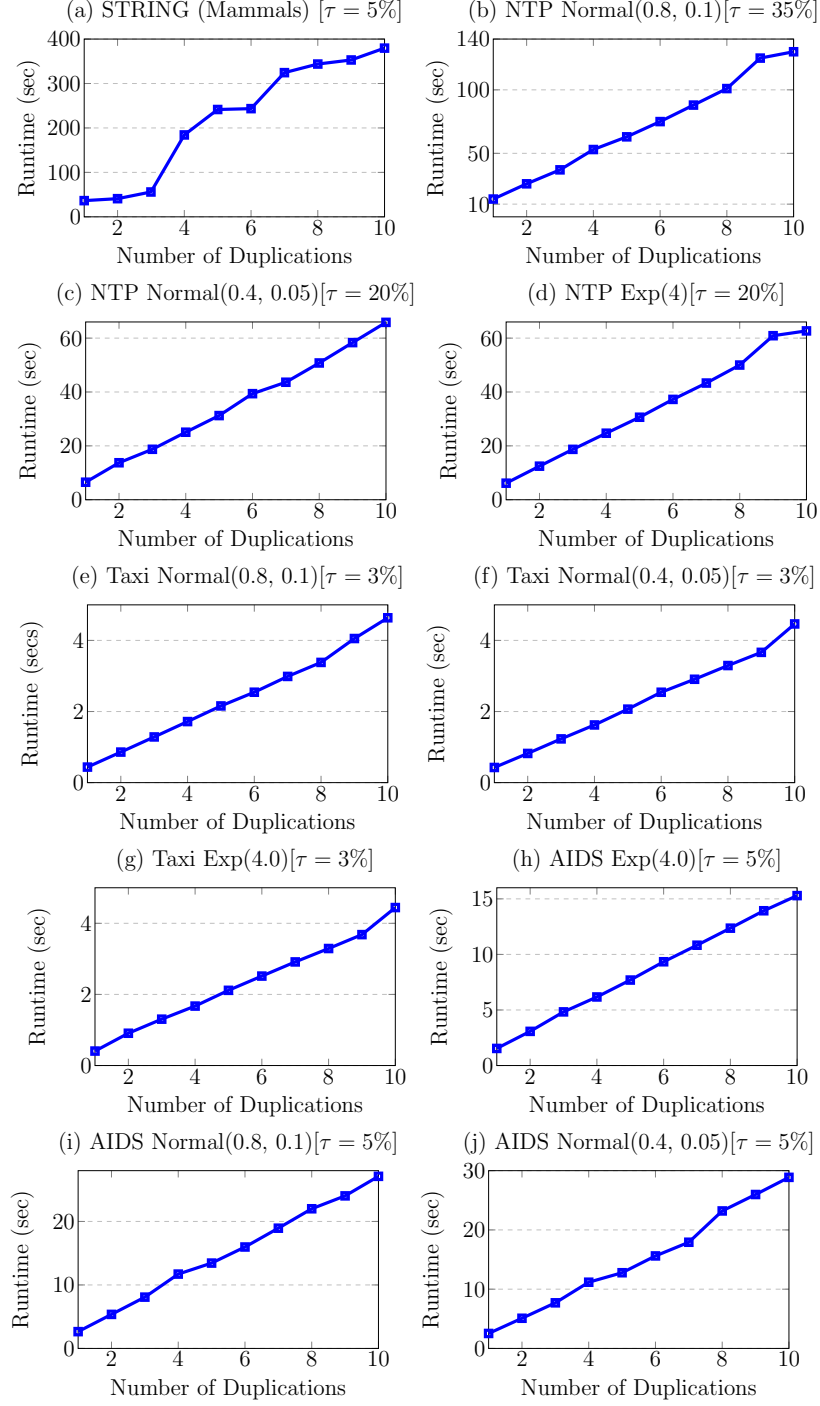


FIGURE 4.15: Scalability of UFreS in terms of Runtime

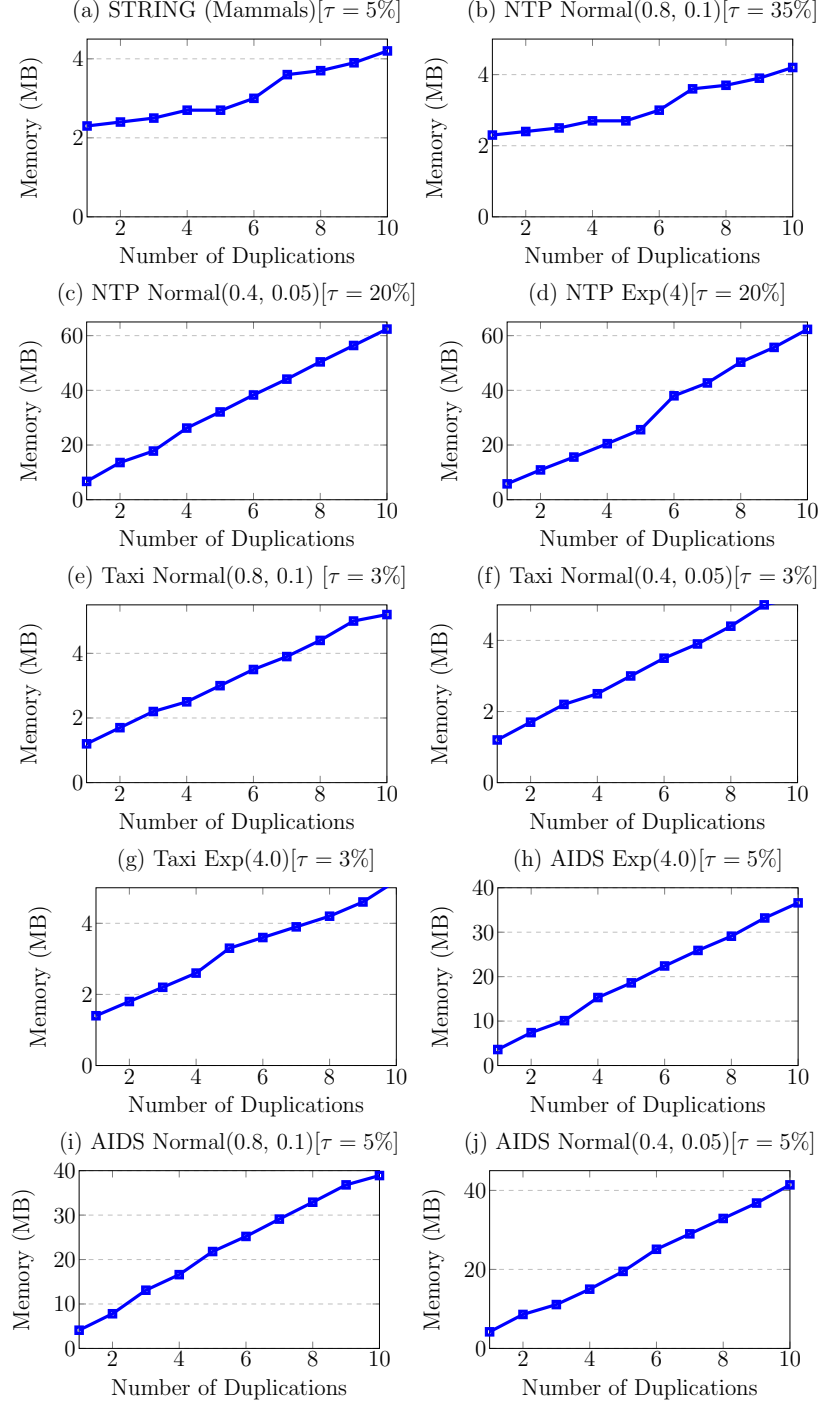


FIGURE 4.16: Scalability of UFreS in terms of Memory

4.6 Summary of the Experimental Results

We have experimented on 10 variants of four real-world graph databases from three different application domains and have compared the performances of our proposed approach UFreS with that of the existing approaches UGRAP and MUSE for a total of 82 different threshold values. For all threshold values, in all variants of all datasets, we have observed that the runtime of UFreS is several orders of magnitude lower than that of the existing approaches. For threshold values for which both MUSE and UGRAP failed to produce outputs even after running for hours, UFreS produced results in a matter of seconds.

Our investigations revealed that this improvement in runtime is achieved due to the robustness of our edge-embedding graph based expected support evaluation technique. While the performance of the exact approach for determining the expected support followed by both MUSE and UGRAP deteriorates sharply in graphs where the number of embeddings is high, the edge-embedding graph technique's performance remains almost unaffected if the number of overlapping edges between the embeddings is not too high. Scheduling and early termination allow UFreS to produce the same results despite conducting a lower number of expected support evaluations than MUSE, often allowing UFreS to avoid computationally difficult evaluations.

The exact approach followed by UGRAP and MUSE also has another glaring weakness-its peak memory consumption increases exponentially with the number of embeddings in a graph. This exponential increase in memory is so drastic, that it causes the device used in our experimental setup to run out of available RAM and crash for several threshold values while running those algorithms. UFreS suffers from no such explosive growth of peak memory usage, making it much more feasible for usage when attempting to mine from large databases whose graphs may contain potentially hundreds of embeddings.

While the exact approach of the existing approaches has weaknesses in terms of runtime and memory consumption, the alternative Monte Carlo approach they propose using when the number of embeddings is high causes the quality of the output generated by UGRAP and MUSE to deteriorate. Due to their reliance on approximation schemes, both UGRAP and MUSE (which produce the same output for the same threshold values), produced false positives and false negatives in our experiments. UFreS does not rely on any approximation scheme whatsoever, and thus never produces false outputs. Thus, UFreS, being an exact approach, unsurprisingly outperforms UGRAP and MUSE in terms of accuracy.

Finally, to test how the size of the database affects the performance of UFreS, we performed scalability tests by making as many as 10 duplicate copies for all variants of all datasets and testing how UFreS handles the increased size of the dataset. Although the runtime and memory consumption of UFreS increased with the number of duplicate copies, the growth was linear in all our experiments, showing that UFreS can handle large databases in an efficient and effective manner. This proved that UFreS is scalable, and can be employed to produce results in the extremely large data repositories that are seen in many different applications nowadays.

In all, our experiments show that UFreS outperforms its predecessors in terms of runtime, memory consumption and accuracy, and is well-equipped to handle large graph databases containing thousands of graphs as transactions.

Chapter 5

Conclusions

The usage of graphs remains indispensable for representing many real world datasets. However, the data derived from real world scenarios is often laden with uncertainties, and this uncertainty often leads to the nature of the graphs being non-deterministic as well. To effectively mine frequent patterns from such uncertain graphs, it is necessary to design specialized frequent subgraph mining algorithms that are well-equipped to handle uncertainty in the graph data. Due to the considerable computational difficulty of the problem, and the increasingly large volumes of data becoming available in many applications, the uncertain frequent subgraph mining techniques need to be highly efficient. However, the state-of-the-art approaches for mining frequent subgraph patterns from uncertain graph data are not efficient in terms of runtime or memory, and our experiments show that they often require hours to mine a relatively low number of patterns. Furthermore, their reliance on approximation schemes has a detrimental effect on the quality of their results, making their results unreliable. To overcome this difficulty, we have proposed UFreS, a new approach that mines frequent subgraph patterns both effectively and efficiently.

5.1 Research Summary

In this work, we have tackled the challenging, yet important problem of frequent subgraph pattern mining from uncertain graph databases. We have shown in chapter 1 that frequently appearing patterns mined from uncertain graph databases can be useful in a wide range of application domains such as bioinformatics, social network analysis and trajectory data analytics. However, considering uncertainty in the data makes frequent subgraph mining a considerably more difficult task. As we have shown in Chapter 2, each uncertain graph implies an exponential number of implied graphs with differing probabilities, and the concept of support, a term very frequently used in pattern mining literature, becomes a random variable instead of a deterministic real number. This considerably increases the computational difficulty of the frequent subgraph mining process, and existing works determine the expected support of subgraph patterns by resorting to Monte-Carlo approximation schemes. However, using Monte-Carlo approximation schemes while mining frequent subgraph patterns is risky, since incorrectly classifying a frequent pattern as infrequent can have a domino effect which, in turn, can lead to missing out on mining many truly frequent patterns. Furthermore, the Monte-Carlo approaches and the alternate exact approach used by these algorithms are inefficient, and this severely limits the application of frequent subgraph mining in the presence of uncertain graph data.

In chapter 3, we have described how our work helps to remediate these weaknesses in the existing approaches. We propose a new data representation technique, the edge-embedding graph, and describe how it can be used to elegantly overcome the challenge of determining the expected value of the support of a frequent subgraph pattern. We then proposed *UFreS*, a novel algorithm for mining frequent subgraph patterns from uncertain graph databases. The usage of edge-embedding graphs, combined with optimizations such as scheduling and early termination, allow *UFreS* to work faster and consume less memory than its predecessors. Our

experimental analysis, detailed in Chapter 4, also shows that state-of-the-art approaches such as MUSE and UGRAP fail to mine a complete and sound set of frequent patterns due to their reliance on approximation schemes. *UFreS* overcomes this deficiency by avoiding approximate algorithms and instead relying on an exact edge-embedding graph based approach to determine the expected support of a pattern. Through experiments on a total of 4 variants of two biological datasets, three variants of a chemical dataset and three variants of a trajectory dataset, we showed that *UFreS* can be applied on a large variety of application domains. Upon further empirical analysis, we found *UFreS* to be highly scalable, both in terms of runtime and memory consumption. All these factors strongly indicate that *UFreS* can be widely used in numerous applications for frequent subgraph mining even when the data is filled with uncertainties.

5.2 Future Research Scopes

For our future research work, we are exploring ways in which edge-embedding graphs can be used to solve the classical problem of propositional model counting. Since the problems of propositional model counting and the computation of expected support of subgraph patterns have been previously proven to be reducible to one other in polynomial time, we are optimistic that our edge-embedding-graph based approach can improve upon the existing solutions to the model counting problem in terms of runtime, accuracy and memory consumption. Furthermore, we are interested in extending our work to handle dynamic graph databases, where graphs may be inserted, replaced or removed from the database sporadically.

Bibliography

- [1] McAuley, J.J. and Leskovec, J., 2012, December. Learning to discover social circles in ego networks. In NIPS (Vol. 2012, pp. 548-56).
- [2] Rossi, R. and Ahmed, N., 2015, March. The network data repository with interactive graph analytics and visualization. In Proceedings of the AAAI Conference on Artificial Intelligence (Vol. 29, No. 1).
- [3] Nuutila, E. and Torma, S., 2004, September. Text graphs: Accurate concept mapping with well-defined meaning. In proceedings of the First International Conference on Concept Mapping.
- [4] Liang, Y., Jiang, Z. and Zheng, Y., 2017, November. Inferring traffic cascading patterns. In Proceedings of the 25th acm sigspatial international conference on advances in geographic information systems (pp. 1-10).
- [5] Asthana, S., King, O.D., Gibbons, F.D. and Roth, F.P., 2004. Predicting protein complex membership using probabilistic network reliability. Genome research, 14(6), pp.1170-1175.
- [6] Tian, Y., Hankins, R.A. and Patel, J.M., 2008, June. Efficient aggregation for graph summarization. In Proceedings of the 2008 ACM SIGMOD international conference on Management of data (pp. 567-580).
- [7] Yan, X., Yu, P.S. and Han, J., 2004, June. Graph indexing: A frequent structure-based approach. In Proceedings of the 2004 ACM SIGMOD international conference on Management of data (pp. 335-346).

-
- [8] Santos, Frances A., et al. "Context-aware vehicle route recommendation platform: Exploring open and crowdsourced data." 2018 IEEE International Conference on Communications (ICC). IEEE, 2018.
 - [9] Moosavi, S.A., Jalali, M., Misaghian, N., Shamshirband, S. and Anisi, M.H., 2017. Community detection in social networks using user frequent pattern mining. *Knowledge and Information Systems*, 51(1), pp.159-186.
 - [10] Li, X., Cheng, Y., Cong, G. and Chen, L., 2017, August. Discovering pollution sources and propagation patterns in urban area. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1863-1872).
 - [11] Zhao, X., Rao, Y., Cai, J. and Ma, W., 2020. Abnormal trajectory detection based on a sparse subgraph. *IEEE Access*, 8, pp.29987-30000.
 - [12] Liu, S., Wang, S., Jayarajah, K., Misra, A. and Krishnan, R., 2013, October. TODMIS: Mining communities from trajectories. In *Proceedings of the 22nd ACM international conference on Information Knowledge Management* (pp. 2109-2118).
 - [13] Zhang, D., Huang, J., Jie, B., Du, J., Tu, L. and Liu, M., 2018. Ordinal pattern: A new descriptor for brain connectivity networks. *IEEE transactions on medical imaging*, 37(7), pp.1711-1722.
 - [14] Yan, X. and Han, J., 2002, December. gspan: Graph-based substructure pattern mining. In *2002 IEEE International Conference on Data Mining, 2002. Proceedings.* (pp. 721-724). IEEE.
 - [15] Nijssen, S. and Kok, J.N., 2005. The gaston tool for frequent subgraph mining. *Electronic Notes in Theoretical Computer Science*, 127(1), pp.77-87.
 - [16] Fournier-Viger, P., Cheng, C., Lin, J.C.W., Yun, U. and Kiran, R.U., 2019, December. Tkg: Efficient mining of top-k frequent subgraphs. In *International Conference on Big Data Analytics* (pp. 209-226). Springer, Cham.

-
- [17] Mrzic, A., Meysman, P., Bittremieux, W., Moris, P., Cule, B., Goethals, B. and Laukens, K., 2018. Grasping frequent subgraph mining for bioinformatics applications. *BioData mining*, 11(1), pp.1-24.
- [18] Liakos, P. and Papakonstantinou, K., 2016, March. On the impact of social cost in opinion dynamics. In *Proceedings of the International AAAI Conference on Web and Social Media* (Vol. 10, No. 1).
- [19] Moreira-Matias, L., Gama, J., Ferreira, M., Mendes-Moreira, J. and Damas, L., 2013. Predicting taxi-passenger demand using streaming data. *IEEE Transactions on Intelligent Transportation Systems*, 14(3), pp.1393-1402.
- [20] Punzo, V., Borzacchiello, M.T. and Ciuffo, B., 2011. On the assessment of vehicle trajectory data accuracy and application to the Next Generation Simulation (NGSIM) program data. *Transportation Research Part C: Emerging Technologies*, 19(6), pp.1243-1262.
- [21] Santamaría-Gómez, A., Bouin, M.N., Collilieux, X. and Wöppelmann, G., 2011. Correlated errors in GPS position time series: Implications for velocity estimates. *Journal of Geophysical Research: Solid Earth*, 116(B1).
- [22] Muckell, J., Hwang, J.H., Lawson, C.T. and Ravi, S.S., 2010, November. Algorithms for compressing GPS trajectory data: an empirical evaluation. In *Proceedings of the 18th SIGSPATIAL International Conference on Advances in Geographic Information Systems* (pp. 402-405).
- [23] Boldi, P., Bonchi, F., Gionis, A. and Tassa, T., 2012. Injecting uncertainty in graphs for identity obfuscation. *arXiv preprint arXiv:1208.4145*.
- [24] Zou, Z., Li, J., Gao, H. and Zhang, S., 2010. Mining frequent subgraph patterns from uncertain graph data. *IEEE Transactions on Knowledge and Data Engineering*, 22(9), pp.1203-1218.

- [25] Papapetrou, O., Ioannou, E. and Skoutas, D., 2011, March. Efficient discovery of frequent subgraph patterns in uncertain graph databases. In *Proceedings of the 14th International Conference on Extending Database Technology* (pp. 355-366).
- [26] Karp, R.M., Luby, M. and Madras, N., 1989. Monte-Carlo approximation algorithms for enumeration problems. *Journal of algorithms*, 10(3), pp.429-448.
- [27] Chen, Y., Zhao, X., Lin, X. and Wang, Y., 2015, November. Towards frequent subgraph mining on single large uncertain graphs. In *2015 IEEE International Conference on Data Mining* (pp. 41-50). IEEE.
- [28] Acosta-Mendoza, N., Gago-Alonso, A. and Medina-Pagola, J.E., 2012. Frequent approximate subgraphs as features for graph-based image classification. *Knowledge-Based Systems*, 27, pp.381-392.
- [29] Jiang, M., Cui, P., Beutel, A., Faloutsos, C. and Yang, S., 2016. Catching synchronized behaviors in large networks: A graph mining approach. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 10(4), pp.1-27.
- [30] Kong, X., Yu, P.S., Wang, X. and Ragin, A.B., 2013, May. Discriminative feature selection for uncertain graph classification. In *Proceedings of the 2013 SIAM International Conference on Data Mining* (pp. 82-93). Society for Industrial and Applied Mathematics.
- [31] Akbarinasaji, S., Kavaklioglu, C., Başar, A. and Neal, A., 2020. Partially observable Markov decision process to generate policies in software defect management. *Journal of Systems and Software*, 163, p.110518.
- [32] Nguyen, D., Luo, W., Nguyen, T.D., Venkatesh, S. and Phung, D., 2018, May. Learning graph representation via frequent subgraphs. In *Proceedings of the 2018 SIAM International Conference on Data Mining* (pp. 306-314). Society for Industrial and Applied Mathematics.

-
- [33] Szklarczyk, D., Gable, A.L., Lyon, D., Junge, A., Wyder, S., Huerta-Cepas, J., Simonovic, M., Doncheva, N.T., Morris, J.H., Bork, P. and Jensen, L.J., 2019. STRING v11: protein–protein association networks with increased coverage, supporting functional discovery in genome-wide experimental datasets. *Nucleic acids research*, 47(D1), pp.D607-D613.
- [34] Han, J., Pei, J., Yin, Y. and Mao, R., 2004. Mining frequent patterns without candidate generation: A frequent-pattern tree approach. *Data mining and knowledge discovery*, 8(1), pp.53-87.
- [35] Lin, J.C.W., Gan, W., Fournier-Viger, P., Hong, T.P. and Tseng, V.S., 2016. Weighted frequent itemset mining over uncertain databases. *Applied Intelligence*, 44(1), pp.232-250.
- [36] Nguyen, H.L., 2015. An efficient algorithm for mining weighted frequent itemsets using adaptive weights. *International Journal of Intelligent Systems and Applications*, 7(11), p.41.
- [37] Bernecker, T., Kriegel, H.P., Renz, M., Verhein, F. and Zuefle, A., 2009, June. Probabilistic frequent itemset mining in uncertain databases. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining* (pp. 119-128).
- [38] Tong, Y., Chen, L. and Yu, P.S., 2012, August. Ufimt: an uncertain frequent itemset mining toolbox. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining* (pp. 1508-1511).
- [39] Yun, U. and Leggett, J.J., 2005, April. WFIM: weighted frequent itemset mining with a weight range and a minimum weight. In *Proceedings of the 2005 SIAM international conference on data mining* (pp. 636-640). Society for Industrial and Applied Mathematics.

-
- [40] Leung, C.K.S., Khan, Q.I., Li, Z. and Hoque, T., 2007. CanTree: a canonical-order tree for incremental frequent-pattern mining. *Knowledge and Information Systems*, 11(3), pp.287-311.
- [41] Sun, J., Xun, Y., Zhang, J. and Li, J., 2019. Incremental frequent itemsets mining with FCFP tree. *IEEE Access*, 7, pp.136511-136524.
- [42] Hernández-León, R., Hernández-Palancar, J., Carrasco-Ochoa, J.A. and Martínez-Trinidad, J.F., 2010. Algorithms for mining frequent itemsets in static and dynamic datasets. *Intelligent Data Analysis*, 14(3), pp.419-435.
- [43] Patil, S.D., Deshmukh, R.R. and Kirange, D.K., 2016, November. Adaptive Apriori Algorithm for frequent itemset mining. In *2016 International Conference System Modeling Advancement in Research Trends (SMART)* (pp. 7-13). IEEE.
- [44] Mozafari, B., Thakkar, H. and Zaniolo, C., 2008, April. Verifying and mining frequent patterns from large windows over data streams. In *2008 IEEE 24th International Conference on Data Engineering* (pp. 179-188). IEEE.
- [45] Li, H., Zhang, N., Zhu, J., Wang, Y. and Cao, H., 2018. Probabilistic frequent itemset mining over uncertain data streams. *Expert Systems with Applications*, 112, pp.274-287.
- [46] Tanbeer, S.K., Ahmed, C.F., Jeong, B.S. and Lee, Y.K., 2009. Efficient single-pass frequent pattern mining using a prefix-tree. *Information Sciences*, 179(5), pp.559-583.
- [47] Rizvee, R.A., Arefin, M.F. and Ahmed, C.F., 2020, May. Tree-Miner: Mining Sequential Patterns from SP-Tree. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining* (pp. 44-56). Springer, Cham.
- [48] Nishi, M.A., Ahmed, C.F., Samiullah, M. and Jeong, B.S., 2013. Effective periodic pattern mining in time series databases. *Expert Systems with Applications*, 40(8), pp.3015-3027.

- [49] Rizvee, R.A., Shahin, M.S.H., Ahmed, C.F., Leung, C.K., Deng, D. and Mai, J.J., 2019. Sliding window based weighted periodic pattern mining over time series data. ibai publishing.
- [50] Elseidy, M., Abdelhamid, E., Skiadopoulos, S. and Kalnis, P., 2014. Grami: Frequent subgraph and pattern mining in a single large graph. *Proceedings of the VLDB Endowment*, 7(7), pp.517-528.
- [51] Ashraf, N., Haque, R.R., Islam, M., Ahmed, C.F., Leung, C.K., Mai, J.J. and Wodi, B.H., 2019. WeFreS: weighted frequent subgraph mining in a single large graph. ibai publishing.
- [52] Fournier-Viger, P., Lin, J.C.W., Nkambou, R., Vo, B. and Tseng, V.S., 2019. High-utility pattern mining. Springer.
- [53] Ahmed, C.F., Tanbeer, S.K., Jeong, B.S. and Lee, Y.K., 2009. Efficient tree structures for high utility pattern mining in incremental databases. *IEEE Transactions on Knowledge and Data Engineering*, 21(12), pp.1708-1721.
- [54] Meel, K.S., Shrotri, A.A. and Vardi, M.Y., 2019. Not all FPRASs are equal: demystifying FPRASs for DNF-counting. *Constraints*, 24(3), pp.211-233.
- [55] Agarwal, R. and Srikant, R., 1994, September. Fast algorithms for mining association rules. In *Proc. of the 20th VLDB Conference* (Vol. 487, p. 499).
- [56] Srinivasan, A., King, R.D., Muggleton, S.H. and Sternberg, M.J., 1997, August. The predictive toxicology evaluation challenge. In *IJCAI* (1) (pp. 4-9).
- [57] Zou, R. and Holder, L.B., 2010, July. Frequent subgraph mining on a single large graph using sampling techniques. In *Proceedings of the eighth workshop on mining and learning with graphs* (pp. 171-178).
- [58] Riesen, K. and Bunke, H.: IAM Graph Database Repository for Graph Based Pattern Recognition and Machine Learning. In: da Vitoria Lobo, N. et al. (Eds.), *SSPRSPR 2008, LNCS*, vol. 5342, pp. 287-297, 2008.

-
- [59] TUDataset: A collection of benchmark datasets for learning with graphs C Morris, NM Kriege, F Bause, K Kersting, P Mutzel, M Neumann ICML 2020 Workshop on Graph Representation Learning and Beyond (GRL+ 2020)

List of Notations

KDD - Knowledge Discovery from Data

UFreS - Uncertain Frequent Subgraph Miner

τ - Minimum Expected Support Threshold

min_sup - Minimum Support

DFS - Depth First Search

Edge-Embedding Graph - A novel data structure we propose to infer the probability of a subgraph pattern being subgraph isomorphic to an uncertain graph.

PPI - Protein-protein Interactions

Connected Component - A maximal subset of nodes in a graph where a path exists between all pairs of nodes.

Bipartite Graph - A graph whose nodes can be partitioned into two sets, such that no edge connects two nodes in the same set.

CBR -Cost Benefit Ratio